

Day01_Ajax入门

知识点自测

1. 如下对象取值的方式哪个正确?

```
let obj = {  
  name: '黑马'  
}
```

A: obj.a

B: obj()a

► 答案

2. 哪个赋值会让浏览器解析成标签显示?

```
let ul = document.querySelector('#ul')  
let str = `<span>我是span标签</span>`
```

A: ul.innerText = str

B: ul.innerHTML = str

► 答案

3. 哪个是获取输入框值的方式?

```
let theInput = document.querySelector('#input')
```

A: theInput.innerHTML

B: theInput.value

► 答案

4. 哪个是用于获取标签内容?

```
let theP = document.querySelector('#p')
```

A: theP.innerHTML = '内容'

B: theP.innerHTML

► 答案

5. 哪个是数组的映射方法?

A: arr.forEach

B: arr.map

► 答案

6. 数组转字符串并指定拼接符的是哪个?

A: arr.join()

B: arr.split()

► 答案

7. 函数传参的方式哪个是正确的?

```
function showAlert(msg, className) {}
```

A: showAlert('消息', '类名')

B: showAlert()

► 答案

8. 以下哪套代码可以实现对象属性的简写?

A:

```
const username = '老李'
let obj = {
  username: username
}
```

B:

```
const user = '老李'
let obj = {
  username: user
}
```

► 答案

9. 以下代码的值是多少?

```
const age = 10
const result = age > 18 ? '成年了' : '未成年'
```

A: '成年了'

B: '未成年'

► 答案

10. 以下哪个方法可以添加一个额外类名?

A: 标签对象.classList.add()

B: 标签对象.classList.contains()

► 答案

目录

- AJAX 概念和 axios 使用
- 认识 URL
- URL 查询参数
- 常用请求方法和数据提交
- HTTP协议-报文
- 接口文档
- 案例 - 用户登录
- form-serialize 插件

学习目标

1. 掌握 axios 相关参数，从服务器获取并解析展示数据
2. 掌握接口文档的查看和使用
3. 掌握在浏览器的 network 面板中查看请求和响应的内容
4. 了解请求和响应报文的组成部分

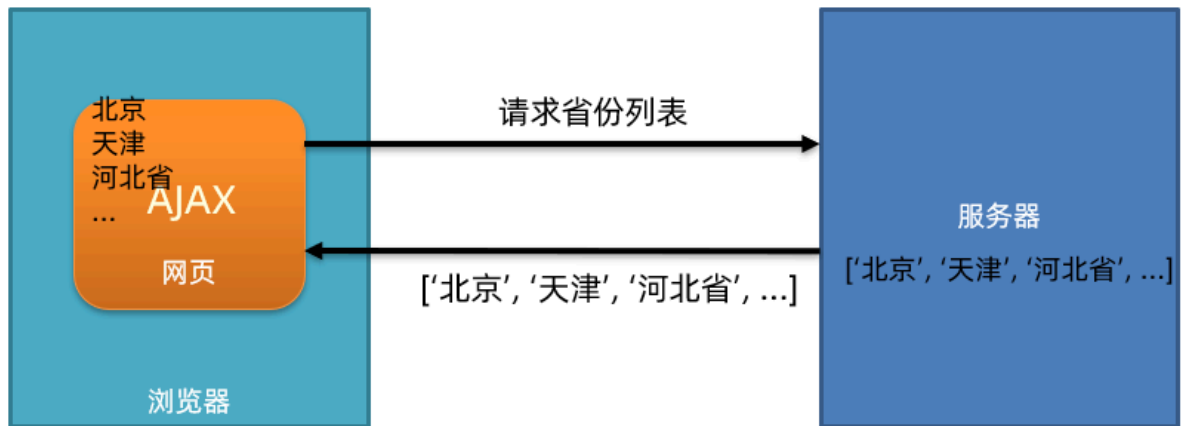
01.AJAX 概念和 axios 使用

目标

了解 AJAX 概念并掌握 axios 库基本使用

讲解

1. 什么是 AJAX ? [mdn](#)
 - 使用浏览器的 XMLHttpRequest 对象 与服务器通信
 - 浏览器网页中，使用 AJAX技术（XHR对象）发起获取省份列表数据的请求，服务器代码响应准备好的省份列表数据给前端，前端拿到数据数组以后，展示到网页



2. 什么是服务器？

- 可以暂时理解为提供数据的一台电脑

3. 为何学 AJAX ？

- 以前我们的数据都是写在代码里固定的, 无法随时变化
- 现在我们的数据可以从服务器上进行获取，让数据变活

4. 怎么学 AJAX ？

- 这里使用一个第三方库叫 axios, 后续在学习 XMLHttpRequest 对象了解 AJAX 底层原理
- 因为 axios 库语法简单，让我们有更多精力关注在与服务器通信上，而且后续 Vue, React 学习中，也使用 axios 库与服务器通信

5. 需求：从服务器获取省份列表数据，展示到页面上（体验 axios 语法的使用）

获取省份列表数据 - 目标资源地址：<http://hmajax.itheima.net/api/province>

- 完成效果：



6. 接下来讲解 axios 语法，步骤：

7. 引入 axios.js 文件到自己的网页中

axios.js文件链接: <https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js>

8. 明确axios函数的使用语法

```
axios({
  url: '目标资源地址'
}).then((result) => {
  // 对服务器返回的数据做后续处理
})
```

注意: 请求的 url 地址, 就是标记资源的网址

注意: then 方法这里先体验使用, 由来后续会讲到

9. 对应代码

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AJAX概念和axios使用</title>
</head>

<body>
  <!--
    axios库地址: https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js
    省份数据地址: http://hmajax.itheima.net/api/province

    目标: 使用axios库, 获取省份列表数据, 展示到页面上
    1. 引入axios库
  -->
  <p class="my-p"></p>
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <script>
    // 2. 使用axios函数
    axios({
      url: 'http://hmajax.itheima.net/api/province'
    }).then(result => {
      console.log(result)
      // 好习惯: 多打印, 确认属性名
      console.log(result.data.list)
      console.log(result.data.list.join('<br>'))
      // 把准备好省份列表, 插入到页面
      document.querySelector('.my-p').innerHTML = result.data.list.join('<br>')
    })
  </script>
</body>
```

```
</html>
```

小结

1. AJAX 有什么用？
 - ▶ 答案
2. AJAX 如何学？
 - ▶ 答案
3. 这一节 axios 体验步骤（语法）？
 - ▶ 答案

02.认识 URL

目标

了解 URL 的组成和作用

讲解

1. 为什么要认识 URL？[mdn](#)
 - 虽然是后端给我的一个地址，但是哪部分标记的是服务器电脑，哪部分标记的是资源呢？所以为了和服务 器有效沟通我们要认识一下
2. 什么是 URL？
 - 统一资源定位符，简称网址，用于定位网络中的资源（资源指的是：网页，图片，数据，视频，音频等 等）

统一资源定位符（英语：Uniform Resource Locator，缩写：URL，或称统一资源定位器、定位地址、URL地址^[1]） 俗称网页地址，简称网址，是因特网^[1]上标准的资源的地址（Address），如同在网络上的门牌。它最初是由蒂姆·伯纳 斯-李发明用来作为万维网的地址，现在它已经被万维网联盟编制为因特网标准RFC 1738^[2]。
3. URL 的组成？
 - 协议，域名，资源路径（URL 组成有很多部分，我们先掌握这3个重要的部分即可）



钥匙



地址



物品位置

`http://hmajax.itheima.net/api/province`



协议



域名



资源路径

4. 什么是 http 协议？

- 叫超文本传输协议，规定了浏览器和服务器传递数据的格式（而格式具体有哪些稍后我们会学到）

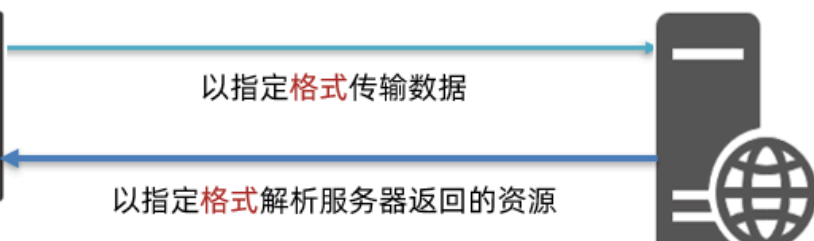
`http://hmajax.itheima.net/api/province`



协议



浏览器



服务器

5. 什么是域名？

- 标记服务器在互联网当中的方位，网络中有很多服务器，你想访问哪一台，就需要知道它的域名才可以

<http://hmajax.itheima.net/api/province>

域名(必须)



百度服务器
baidu.com

淘宝服务器
www.taobao.com

黑马服务器
hmajax.itheima.net

互联网中的其它服务器

6. 什么是资源路径？

- 一个服务器内有多个资源，用于标识你要访问的资源具体的位置

<http://hmajax.itheima.net/api/province>

资源路径



/api/books 图书数据

/api/province 省份数据

/api/news 新闻数据

服务器

7. 接下来做个需求，访问新闻列表的 URL 网址，打印新闻数据

- 效果图如下：

```
▼ data: Array(9)
  ► 0: {id: 1, title: '5G商用在即，三大运营商营收持续下降', source: '新京报经济新闻', cmtcount: 58, img: 'http://ajax-api.itheima.net/public/images/0.webp', ...}
  ► 1: {id: 2, title: '国际媒体头条速览：特朗普宣布杀死巴格达迪时，有人想起了奥巴马', source: '环视全球', cmtcount: 56, img: 'http://ajax-api.itheima.net/public/images/1.webp', ...}
  ► 2: {id: 3, title: '华为5G随行WiFi发布，5G布局全面领先苹果、三星', source: '驱动中国网络传媒', cmtcount: 32, img: 'http://ajax-api.itheima.net/public/images/2.webp', ...}
  ► 3: {id: 4, title: '沈义人：首批5G手机最好选集成SoC和双模5A+NSA', source: '手机中国', cmtcount: 18, img: 'http://ajax-api.itheima.net/public/images/3.webp', ...}
  ► 4: {id: 5, title: '为什么说中美阶段性协议再进一步，读懂周末的这些关键信息', source: '澎湃新闻', cmtcount: 131, img: 'http://ajax-api.itheima.net/public/images/4.webp', ...}
  ► 5: {id: 6, title: '阿根廷大选结果揭晓：反对派候选人费尔南德斯当选下任总统', source: '海外网', cmtcount: 99, img: 'http://ajax-api.itheima.net/public/images/5.webp', ...}
  ► 6: {id: 7, title: '赵薇谈演员整容：不是一件坏事情，但劝大家别整太像', source: '腾讯视频Big磅来了', cmtcount: 154, img: 'http://ajax-api.itheima.net/public/images/6.webp', ...}
  ► 7: {id: 8, title: 'LV母公司史上最大并购：报价145亿美元购Tiffany', source: '澎湃新闻', cmtcount: 119, img: 'http://ajax-api.itheima.net/public/images/7.webp', ...}
  ► 8: {id: 9, title: '黄峥1350亿蝉联80后白手起家首富：过去1年财富每天涨1个亿', source: '胡润百富', cmtcount: 676, img: 'http://ajax-api.itheima.net/public/images/8.webp', ...}
  length: 9
  ► [[Prototype]]: Array(0)
  message: "获取新闻列表成功"
```

新闻列表数据 URL 网址：<http://hmajax.itheima.net/api/news>

```
axios({
  url: 'http://hmajax.itheima.net/api/news'
}).then(result => {
  console.log(result)
})
```

url解释：从黑马服务器使用http协议，访问/api/news路径下的新闻列表资源

小结

1. URL 是什么？

► 答案

2. 请解释这个 URL，每个部分作用？

<http://hmajax.itheima.net/api/news>

► 答案

03.URL 查询参数

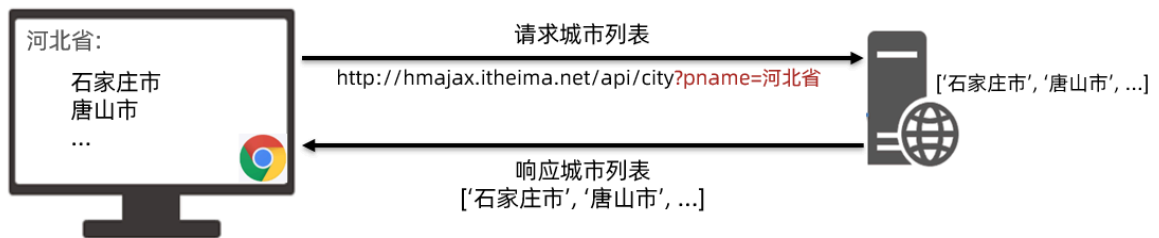
目标

掌握-通过URL传递查询参数，获取匹配的数据

讲解

1. 什么是查询参数？

- 携带给服务器额外信息，让服务器返回我想要的某一部分数据而不是全部数据
- 举例：查询河北省下属的城市列表，需要先把河北省传递给服务器



2. 查询参数的语法？

- 在 url 网址后面用?拼接格式：<http://xxxx.com/xxx/xxx?参数名1=值1&参数名2=值2>
- 参数名一般是后端规定的，值前端看情况传递即可

3. axios 如何携带查询参数？

- 使用 params 选项即可

```
axios({
  url: '目标资源地址',
  params: {
    参数名: 值
  }
}).then(result => {
  // 对服务器返回的数据做后续处理
})
```

查询城市列表的 url 地址：<http://hmajax.itheima.net/api/city>

参数名：pname （值要携带省份名字）

4. 需求：获取“河北省”下属的城市列表，展示到页面，对应代码：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>查询参数</title>
</head>
<body>
  <!--
    城市列表: http://hmajax.itheima.net/api/city
    参数名: pname
    值: 省份名字
  -->
  <p></p>
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <script>
    axios({
      url: 'http://hmajax.itheima.net/api/city',
      // 查询参数
      params: {
        pname: '辽宁省'
      }
    })
```

```
}).then(result => {
  console.log(result.data.list)
  document.querySelector('p').innerHTML = result.data.list.join('<br>')
})
</script>
</body>
</html>
```

小结

1. URL 查询参数有什么用?
▶ 答案
2. axios 要如何携带查询参数?
▶ 答案

04.案例-查询-地区列表

目标

巩固查询参数的使用，并查看多对查询参数如何传递

讲解

1. 需求：根据输入的省份名字和城市名字，查询下属地区列表
 - 完成效果如下：

省份名字	城市名字
北京	北京市

[查询](#)

地区列表:

东城区
西城区
朝阳区
丰台区

- 相关参数

查询地区: <http://hmajax.itheima.net/api/area>

参数名:

pname: 省份名字

cname: 城市名字

2. 正确代码如下:

```
/*
    获取地区列表: http://hmajax.itheima.net/api/area
    查询参数:
        pname: 省份或直辖市名字
        cname: 城市名字
*/
// 目标: 根据省份和城市名字, 查询地区列表
// 1. 查询按钮-点击事件
document.querySelector('.sel-btn').addEventListener('click', () => {
    // 2. 获取省份和城市名字
    let pname = document.querySelector('.province').value
    let cname = document.querySelector('.city').value

    // 3. 基于axios请求地区列表数据
    axios({
        url: 'http://hmajax.itheima.net/api/area',
        params: {
            pname,
            cname
        }
    }).then(result => {
        // console.log(result)
        // 4. 把数据转li标签插入到页面上
        let list = result.data.list
        console.log(list)
        let theLi = list.map(areaName => `<li class="list-group-item">${areaName}</li>`).join('')
        console.log(theLi)
        document.querySelector('.list-group').innerHTML = theLi
    })
})
```

小结

1. ES6 对象属性和值简写的前提是什么?

► 答案

05.常用请求方法和数据提交

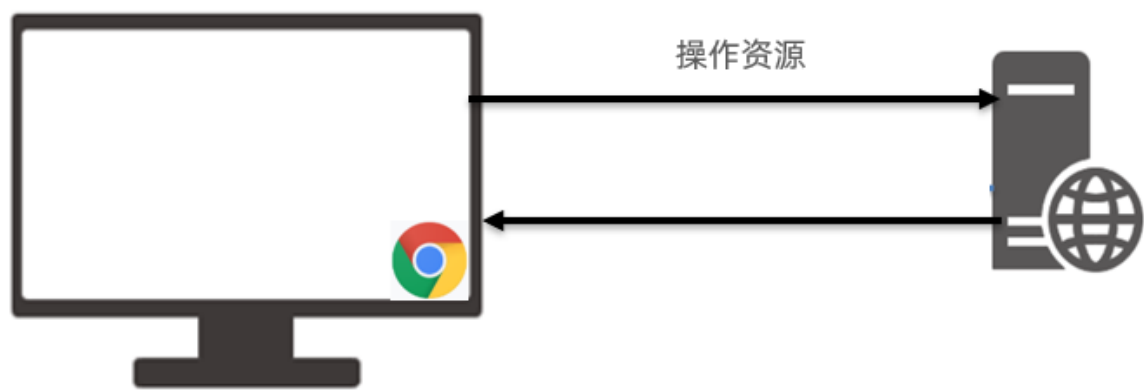
目标

掌握如何向服务器提交数据，而不单单是获取数据

讲解

- 1. 想要提交数据，先来了解什么是请求方法
 - 请求方法是一些固定单词的英文，例如：GET，POST，PUT，DELETE，PATCH（这些都是http协议规定的），每个单词对应一种对服务器资源要执行的操作

请求方法	操作
GET	获取数据
POST	数据提交
PUT	修改数据（全部）
DELETE	删除数据
PATCH	修改数据（部分）



- 前面我们获取数据其实用的就是GET请求方法，但是axios内部设置了默认请求方法就是GET，我们就没有写
 - 但是提交数据需要使用POST请求方法
2. 什么时候进行数据提交呢？
- 例如：多端要查看同一份订单数据，或者使用同一个账号进行登录，那订单/用户名+密码，就需要保存在服务器上，随时随地进行访问

全部订单	待付款	待发货	待收货	待评价	已完成	已取消
下单时间：2022-11-01 17:16:40 订单编号：1587373040889499650						
	女式头层牛皮经典切尔西靴 颜色:黑色牛犊（长絨里） 尺码:36	¥275.00 x1	待发货	¥278.00 (含运费: ¥3.00) 在线支付	查看详情 再次购买	
下单时间：2022-11-01 17:11:50 订单编号：1587371828051972098						
	新潮运动底，女式保暖皮毛一体雪地靴 颜色:杏色 尺码:38	¥319.00 x1	已取消	¥319.00 (含运费: ¥0.00) 在线支付	查看详情	

欢迎注册

已有帐号? [登录](#)

用户名

手机号

密 码 

验证码

[获取验证码](#)

[注册](#)

☐ 阅读并接受 [《百度用户协议》](#)、[《儿童个人信息保护声明》](#) 及 [《百度隐私权保护声明》](#)

3. axios 如何提交数据到服务器呢?

- 需要学习, method 和 data 这2个新的选项了 (大家不用担心, 这2个学完, axios常用的选项就都学完了)

```

axios({
  url: '目标资源地址',
  method: '请求方法',
  data: {
    参数名: 值
  }
}).then(result => {
  // 对服务器返回的数据做后续处理
})

```

4. 需求：注册账号，提交用户名和密码到服务器保存

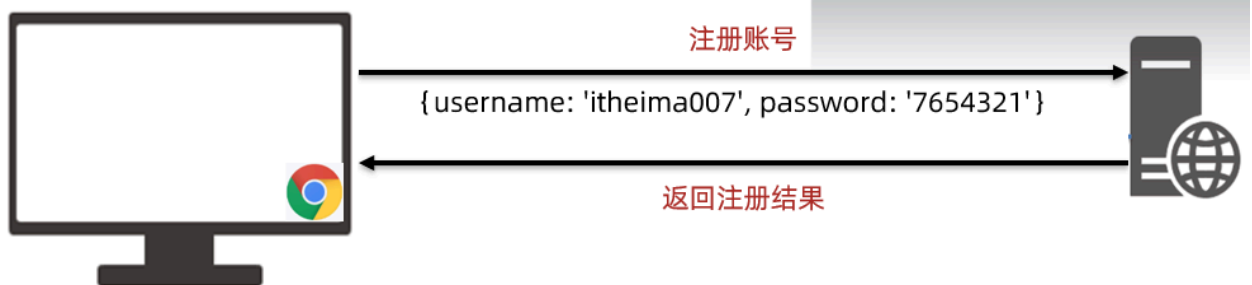
注册用户 URL 网址： <http://hmajax.itheima.net/api/register>

请求方法：POST

参数名：

username：用户名（要求中英文和数字组成，最少8位）

password：密码（最少6位）



5. 正确代码如下：

```

/*
  注册用户: http://hmajax.itheima.net/api/register
  请求方法: POST
  参数名:
    username: 用户名（中英文和数字组成，最少8位）
    password: 密码 （最少6位）

  目标: 点击按钮，通过axios提交用户和密码，完成注册
*/
document.querySelector('.btn').addEventListener('click', () => {
  axios({
    url: 'http://hmajax.itheima.net/api/register',
    method: 'POST',
    data: {
      username: 'itheima007',
      password: '7654321'
    }
  })
})

```

小结

1. 请求方法最常用的是哪2个，分别有什么作用？

► 答案

2. axios 的核心配置项？

► 答案

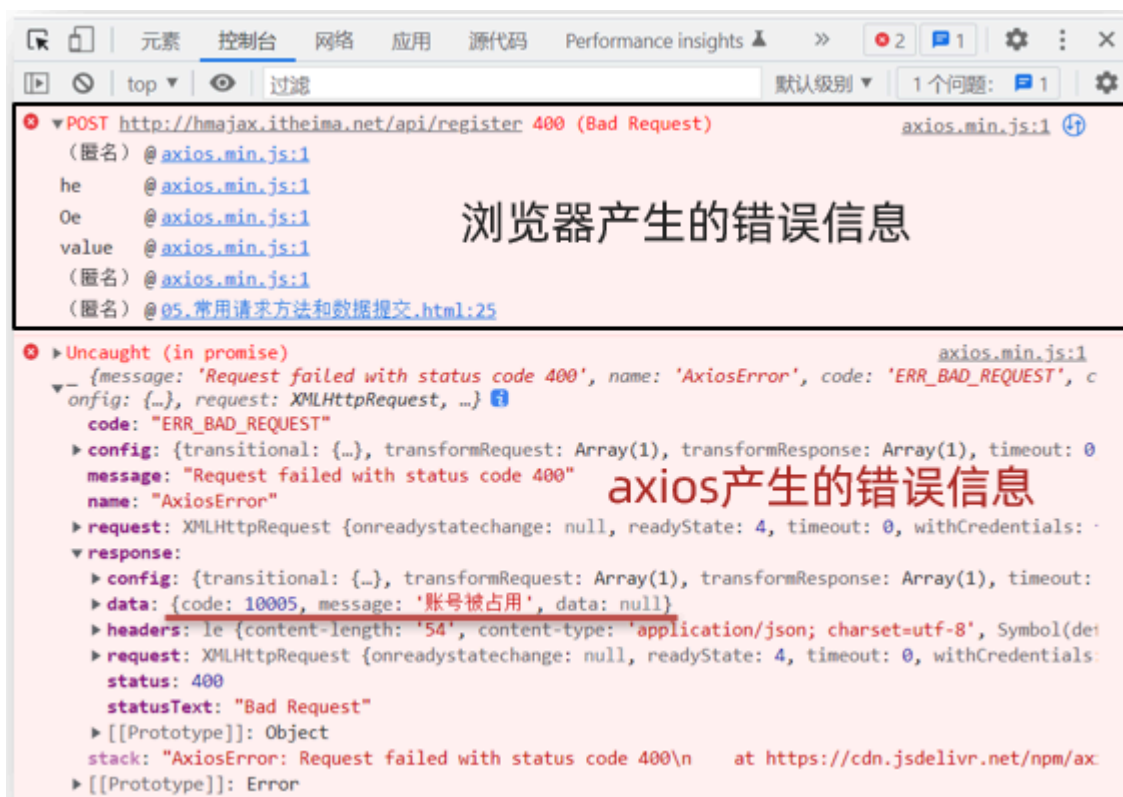
06.axios 错误处理

目标

掌握接收 axios 响应错误信息的处理语法

讲解

1. 如果注册相同的用户名，则会遇到注册失败的请求，也就是 axios 请求响应失败了，你会在控制台看到如图的错误：



2. 在 axios 语法中要如何处理呢？

◦ 因为，普通用户不会去控制台里看错误信息，我们要编写代码拿到错误并展示给用户在页面上

3. 使用 axios 的 catch 方法，捕获这次请求响应的错误并做后续处理，语法如下：

```

axios({
  // ...请求选项
}).then(result => {
  // 处理成功数据
}).catch(error => {
  // 处理失败错误
})

```

4. 需求：再次重复注册相同用户名，提示用户注册失败的原因



5. 对应代码

```

document.querySelector('.btn').addEventListener('click', () => {
  axios({
    url: 'http://hmajax.itheima.net/api/register',
    method: 'post',
    data: {
      username: 'itheima007',
      password: '7654321'
    }
  }).then(result => {
    // 成功
    console.log(result)
  }).catch(error => {
    // 失败
    // 处理错误信息
    console.log(error)
    console.log(error.response.data.message)
    alert(error.response.data.message)
  })
})

```

小结

1. axios 如何拿到请求响应失败的信息？

► 答案

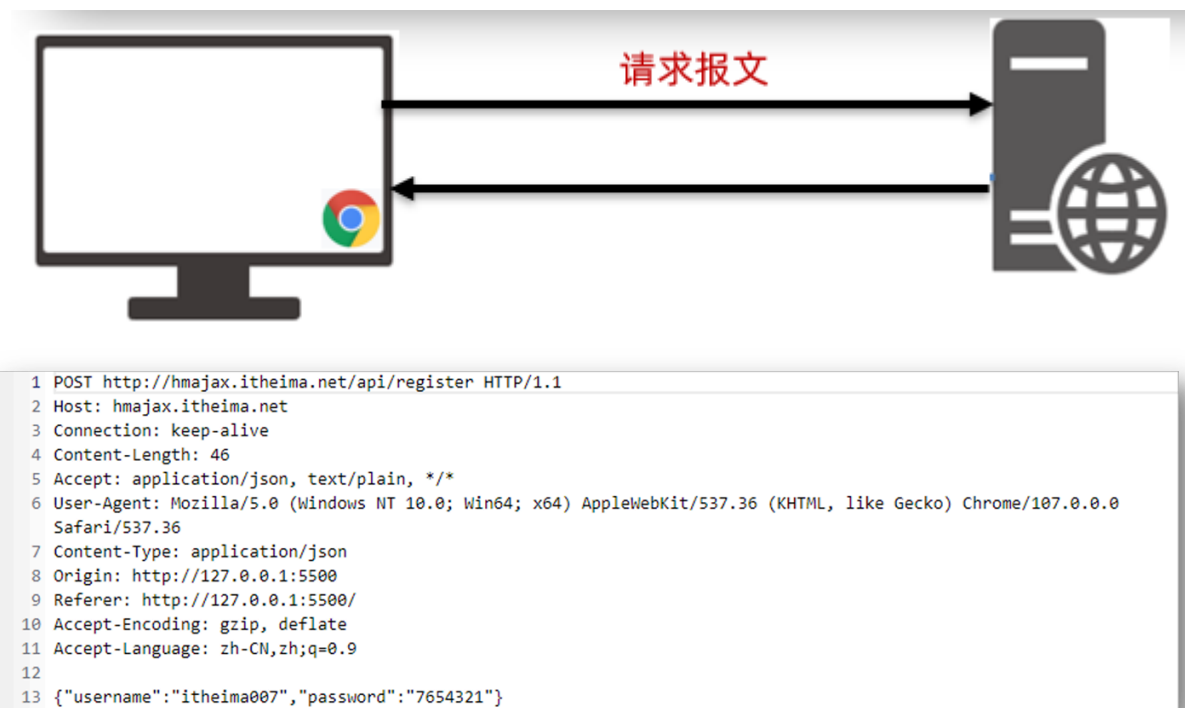
07.HTTP 协议-请求报文

目标

了解 HTTP 协议中，请求报文的组成和作用

讲解

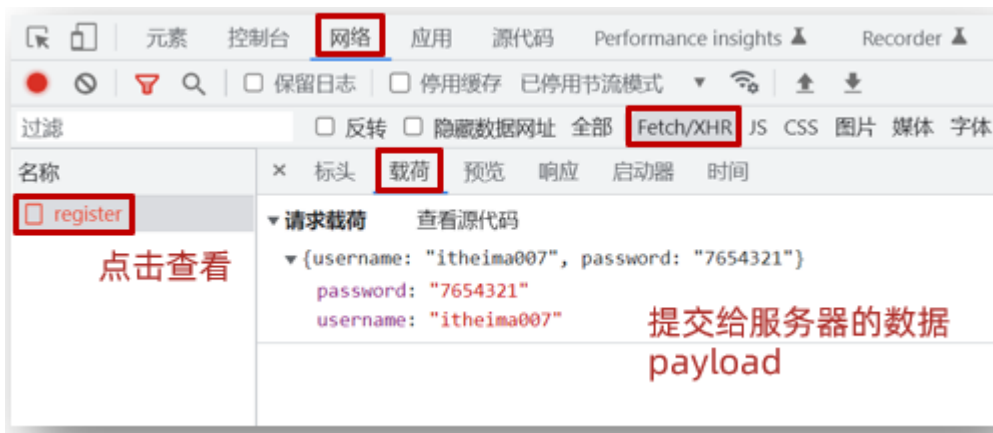
1. 首先，HTTP 协议规定了浏览器和服务端返回内容的格式
2. 请求报文：是浏览器按照协议规定发送给服务器的内容，例如刚刚注册用户时，发起的请求报文：



3. 这里的格式包含：
 - 请求行：请求方法，URL，协议
 - 请求头：以键值对的格式携带的附加信息，比如：Content-Type（指定了本次传递的内容类型）
 - 空行：分割请求头，空行之后的是发送给服务器的资源
 - 请求体：发送的资源
4. 我们切换到浏览器中，来看看刚才注册用户发送的这个请求报文以及内容去哪里查看呢
5. 代码：直接在上个代码基础上复制，然后运行查看请求报文对应关系即可

小结

- 1. 浏览器发送给服务器的内容叫做，请求报文
- 2. 请求报文的组成是什么？
 - ▶ 答案
- 3. 通过 Chrome 的网络面板如何查看请求体？



08.请求报文-错误排查

目标

了解学习了查看请求报文之后的作用，可以用来辅助错误排查

讲解

- 1. 学习了查看请求报文有什么用呢？
 - 可以用来确认我们代码发送的请求数据是否真的正确
- 2. 配套模板代码里，对应 08 标题文件夹里是我同桌的代码，它把登录也写完了，但是无法登录，我们来到模板代码中，找到运行后，在 **不逐行查看代码的情况下**，查看请求报文，看看它登录提交的相关信息对不对，帮他找找问题出现的原因
- 3. 发现请求体数据有问题，往代码中定位，找到类名写错误了
- 4. 代码：在配套文件夹素材里，找到需要对应代码，直接运行，根据报错信息，找到错误原因

小结

1. 学会了查看请求报文，对实际开发有什么帮助呢？

► 答案

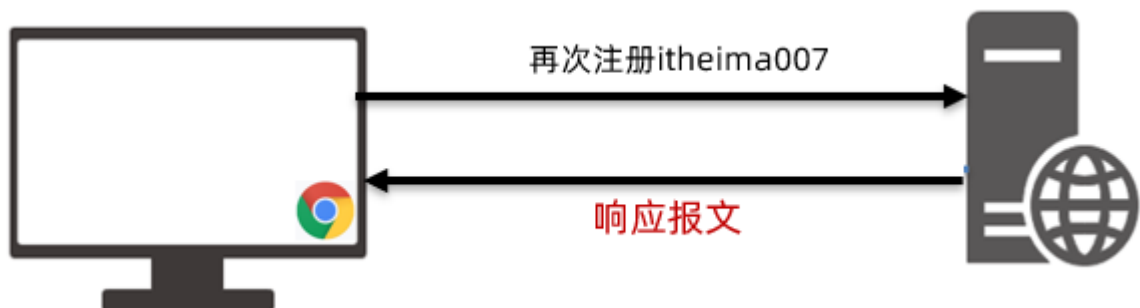
09.HTTP 协议-响应报文

目标

了解响应报文的组成

讲解

1. 响应报文：是服务器按照协议固定的格式，返回给浏览器的内容



```
1 HTTP/1.1 400 Bad Request
2 Server: nginx
3 Date: Wed, 09 Nov 2022 13:26:06 GMT
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 54
6 Connection: keep-alive
7 Vary: Origin
8 Access-Control-Allow-Origin: http://127.0.0.1:5500
9 Access-Control-Allow-Credentials: true
10 x-frame-options: SAMEORIGIN
11 x-xss-protection: 1; mode=block
12 x-content-type-options: nosniff
13 x-download-options: noopen
14 x-readtime: 16
15
16 {"code":10005,"message":"账号被占用","data":null}
```

2. 响应报文的组成：
 - 响应行（状态行）：协议，HTTP响应状态码，状态信息

- 响应头：以键值对的格式携带的附加信息，比如：Content-Type（告诉浏览器，本次返回的内容类型）
- 空行：分割响应头，控制之后的是服务器返回的资源
- 响应体：返回的资源

3. HTTP 响应状态码：

- 用来表明请求是否成功完成
- 例如：404（客户端要找的资源，在服务器上不存在）

状态码	说明
1xx	信息
2xx	成功
3xx	重定向消息
4xx	客户端错误
5xx	服务端错误

小结

1. 响应报文的组成？

► 答案

2. HTTP 响应状态码是做什么的？

► 答案

10.接口文档

目标

掌握接口文档的使用，配合 axios 与服务器进行数据交互

讲解

- 1. 接口文档：描述接口的文章（一般是后端工程师，编写和提供）
- 2. 接口：指的使用 AJAX 和 服务器通讯时，使用的 URL，请求方法，以及参数，例如：[AJAX阶段接口文档](#)
- 3. 例如：获取城市列表接口样子

获取-城市列表

GET

http://hmajax.itheima.net/api/city

获取-城市列表

请求参数

参数名	位置	类型	必填	说明
pname	query	string	是	示例值：辽宁省

- 4. 需求：打开 AJAX 阶段接口文档，查看登录接口，并编写代码，完成一次登录的效果吧
- 5. 代码如下：

```
document.querySelector('.btn').addEventListener('click', () => {
  // 用户登录
  axios({
    url: 'http://hmajax.itheima.net/api/login',
    method: 'post',
    data: {
      username: 'itheima007',
      password: '7654321'
    }
  })
})
```

小结

1. 接口文档是什么?
 - ▶ 答案
2. 接口文档里包含什么?
 - ▶ 答案

11.案例-用户登录-主要业务

目标

尝试通过页面获取用户名和密码，进行登录

讲解

1. 先来到备课代码中，运行完成的页面，查看要完成的登录效果（登录成功和失败）
2. 需求：编写代码，查看接口文档，填写相关信息，完成登录业务
3. 分析实现的步骤
 1. 点击登录，获取并判断用户名和长度
 2. 提交数据和服务器通信
 3. 提示信息，反馈给用户（这节课先来完成前 2 个步骤）

欢迎-登录

登录成功

账号名

itheima007

密码

.....

登 录

4. 代码如下：

```
// 目标1：点击登录时，用户名和密码长度判断，并提交数据和服务器通信

// 1.1 登录-点击事件
document.querySelector('.btn-login').addEventListener('click', () => {
  // 1.2 获取用户名和密码
  const username = document.querySelector('.username').value
  const password = document.querySelector('.password').value
  // console.log(username, password)

  // 1.3 判断长度
  if (username.length < 8) {
    console.log('用户名必须大于等于8位')
    return // 阻止代码继续执行
  }
  if (password.length < 6) {
    console.log('密码必须大于等于6位')
    return // 阻止代码继续执行
  }
})
```

```
// 1.4 基于axios提交用户名和密码
// console.log('提交数据到服务器')
axios({
  url: 'http://hmajax.itheima.net/api/login',
  method: 'POST',
  data: {
    username,
    password
  }
}).then(result => {
  console.log(result)
  console.log(result.data.message)
}).catch(error => {
  console.log(error)
  console.log(error.response.data.message)
})
})
```

小结

1. 总结下用户登录案例的思路？

► 答案

12.案例-用户登录-提示信息

目标

根据准备好的提示标签和样式，给用户反馈提示

讲解

1. 需求：使用提前准备好的提示框，来把登录成功/失败结果提示给用户

欢迎-登录

登录成功

账号名

itheima007

密码

.....

登录

欢迎-登录

用户名或密码错误

账号名

itheima007

密码

••••••••

登 录

2. 使用提示框，反馈提示消息，因为有4处地方需要提示框，所以封装成函数

1. 获取提示框

2. 封装提示框函数，重复调用，满足提示需求

功能：

1. 显示提示框

2. 不同提示文字msg，和成功绿色失败红色isSuccess参数（true成功，false失败）

3. 过2秒后，让提示框自动消失

3. 对应提示框核心代码：

/**

* 2.2 封装提示框函数，重复调用，满足提示需求

* 功能：

* 1. 显示提示框


```
* 2. 不同提示文字msg, 和成功绿色失败红色isSuccess (true成功, false失败)
* 3. 过2秒后, 让提示框自动消失
*/
function alertFn(msg, isSuccess) {
  // 1> 显示提示框
  myAlert.classList.add('show')

  // 2> 实现细节
  myAlert.innerText = msg
  const bgStyle = isSuccess ? 'alert-success' : 'alert-danger'
  myAlert.classList.add(bgStyle)

  // 3> 过2秒隐藏
  setTimeout(() => {
    myAlert.classList.remove('show')
    // 提示: 避免类名冲突, 重置背景色
    myAlert.classList.remove(bgStyle)
  }, 2000)
}
```

小结

1. 我们什么时候需要封装函数?
 - ▶ 答案
2. 如何封装一个函数呢?
 - ▶ 答案
3. 我们的提示框是如何控制出现/隐藏的?
 - ▶ 答案

13.form-serialize 插件

目标

使用 form-serialize 插件, 快速收集目标表单范围内表单元素的值

讲解

1. 我们前面收集表单元素的值, 是一个个标签获取的

账号名

itheima007

密码

●●●●●●●●

2. 如果一套表单里有很多很多表单元素，如何一次性快速收集出来呢？

新增员工

* 姓名

请输入姓名

* 手机号

请输入手机号

* 入职时间

请选择入职时间

* 聘用形式

请选择聘用形式

* 工号

请输入工号

* 部门

请选择部门

转正时间

请选择转正时间

确定

取消

3. 使用 form-serialize 插件提供的 serialize 函数就可以办到

4. form-serialize 插件语法：

1. 引入 form-serialize 插件到自己网页中

2. 使用 serialize 函数

- 参数1：要获取的 form 表单标签对象（要求表单元素需要有 name 属性-用来作为收集的数据中属性名）
- 参数2：配置对象

- hash:
 - true - 收集出来的是一个 JS 对象结构
 - false - 收集出来的是一个查询字符串格式
- empty:
 - true - 收集空值
 - false - 不收集空值

5. 需求：收集登录表单里用户名和密码

6. 对应代码：

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>form-serialize插件使用</title>
</head>

<body>
  <form action="javascript:;" class="example-form">
    <input type="text" name="username">
    <br>
    <input type="text" name="password">
    <br>
    <input type="button" class="btn" value="提交">
  </form>
  <!--
    目标：在点击提交时，使用form-serialize插件，快速收集表单元素值
    1. 把插件引入到自己网页中
  -->
  <script src="./lib/form-serialize.js"></script>
  <script>
    document.querySelector('.btn').addEventListener('click', () => {
      /**
       * 2. 使用serialize函数，快速收集表单元素的值
       * 参数1：要获取哪个表单的数据
       * 表单元素设置name属性，值会作为对象的属性名
       * 建议name属性的值，最好和接口文档参数名一致
       * 参数2：配置对象
       * hash 设置获取数据结构
       *   - true: JS对象（推荐）一般请求体里提交给服务器
       *   - false: 查询字符串
       * empty 设置是否获取空值
       *   - true: 获取空值（推荐）数据结构和标签结构一致
       *   - false: 不获取空值
       */
      const form = document.querySelector('.example-form')
      const data = serialize(form, { hash: true, empty: true })
      // const data = serialize(form, { hash: false, empty: true })
      // const data = serialize(form, { hash: true, empty: false })
    })
  </script>
</body>
</html>
```

```
        console.log(data)
    })
</script>
</body>

</html>
```

小结

1. 我们什么时候使用 form-serialize 插件?
 - ▶ 答案
2. 如何使用 form-serialize 插件?
 - ▶ 答案
3. 配置对象中 hash 和 empty 有什么用?
 - ▶ 答案

14.案例-用户登录-form-serialize

目标

尝试通过 form-serialize 重新修改用户登录案例-收集用户名和密码

讲解

1. 基于模板代码，使用 form-serialize 插件来收集用户名和密码
2. 在原来的代码基础上修改即可
 1. 先引入插件

```
<!-- 3.1 引入插件 -->
<script src="./lib/form-serialize.js"></script>
```

2. 然后修改代码

```
// 3.2 使用serialize函数，收集登录表单里用户名和密码
const form = document.querySelector('.login-form')
const data = serialize(form, { hash: true, empty: true })
console.log(data)
// {username: 'itheima007', password: '7654321'}
const { username, password } = data
```

小结

1. 如何把一个第三方插件使用在已完成的案例中？

► 答案

今日重点(必须会)

1. axios 的配置项有哪几个，作用分别是什么？
2. 接口文档都包含哪些信息？
3. 在浏览器中如何查看查询参数/请求体，以及响应体数据？
4. 请求报文和响应报文由几个部分组成，每个部分的作用？

今日作业(必完成)

参考作业文件夹的md要求

参考文献

1. 客户端->百度百科
2. 浏览器解释->百度百科
3. 服务器解释->百度百科
4. url解释->百度百科
5. http协议->百度百科
6. 主机名->百度百科
7. 端口号->百度百科
8. Ajax解释->百度-懂啦
9. Ajax解释->MDN解释Ajax是与服务器通信而不只是请求
10. axios->百度(可以点击播报听读音)
11. axios(github)地址
12. axios官方推荐官网
13. axios(npmjs)地址
14. GET和POST区别->百度百科
15. 报文讲解->百度百科
16. HTTP状态码->百度百科

17. [接口概念->百度百科](#)

Day02_AJAX综合案例

知识点自测

1. 以下代码运行结果是什么？（考察扩展运算符的使用）

```
const result = {  
  name: '老李',  
  age: 18  
}  
  
const obj = {  
  ...result  
}  
  
console.log(obj.age)
```

A: 报错

B: 18

► 答案

2. 什么是事件委托？

A: 只能把单击事件委托给父元素绑定

B: 可以把能冒泡的事件，委托给已存在的向上的任意标签元素绑定

► 答案

3. 事件对象e.target作用是什么？

A: 获取到这次触发事件相关的信息

B: 获取到这次触发事件目标标签元素

► 答案

4. 如果获取绑定在标签上自定义属性的值10？

```
<div data-code="10">西游记</div>
```

A: div标签对象.innerHTML

B: div标签对象.dataset.code

C: div标签对象.code

► 答案

5. 哪个方法可以判断目标标签是否包含指定的类名?

```
<div class="my-div title info"></div>
```

A: div标签对象.className === 'title'

B: div标签对象.classList.contains('title')

► 答案

6. 伪数组取值哪种方式是正确的?

```
let obj = { 0: '老李', 1: '老刘' }
```

A: obj.0

B: obj[0]

► 答案

7. 以下哪个选项可以，往本地存储键为‘bgImg’，值为图片url网址的代码

A: localStorage.setItem('bgImg')

B: localStorage.getItem('bgImg')

C: localStorage.setItem('bgImg', '图片url网址')

D: localStorage.getItem('bgImg', '图片url网址')

► 答案

8. 以下代码运行结果是?

```
const obj = {  
  username: '老李',  
  age: 18,  
  sex: '男'  
}  
Object.keys(obj)
```

A: 代码报错

B: [username, age, sex]

C: ["username", "age", "sex"]

D: ["老李", 18, "男"]

► 答案

9. 下面哪个选项可以把数字字符串转成数字类型?

A: +'10'

B: '10' + 0

► 答案

10. 以下代码运行后的结果是什么？（考察逻辑与的短路特性）

```
const age = 18
const result1 = (age || '有年龄')

const sex = ''
const result2 = sex || '没有性别'
```

- A: 报错，报错
- B: 18，没有性别
- C: 有年龄，没有性别
- D: 18, ''

► 答案

目录

- 案例-图书管理
- 图片上传
- 案例-网站换肤
- 案例-个人信息设置

学习目标

今天主要就是练，巩固 axios 的使用

1. 完成案例-图书管理系统（增删改查）经典业务
2. 掌握图片上传的思路
3. 完成案例-网站换肤并实现图片地址缓存
4. 完成案例-个人信息设置

01.案例_图书管理-介绍

目标

案例-图书管理-介绍（介绍要完成的效果和练习到的思维）

讲解

- 1. 打开备课代码运行图书管理案例效果-介绍要完成的增删改查业务效果和 Bootstrap 弹框使用



- 2. 分析步骤和对应的视频模块
 - 先学习 Bootstrap 弹框的使用（因为添加图书和编辑图书需要这个窗口来承载图书表单）
 - 先做渲染图书列表（这样做添加和编辑以及删除可以看到数据变化，所以先做渲染）
 - 再做新增图书功能
 - 再做删除图书功能
 - 再做编辑图书功能（注意：编辑和新增图书是2套弹框-后续做项目我们再用同1个弹框）

小结

- 1. 做完这个案例我们将会有什么收获呢？
 - ▶ 答案

02.Bootstrap 弹框_属性控制

目标

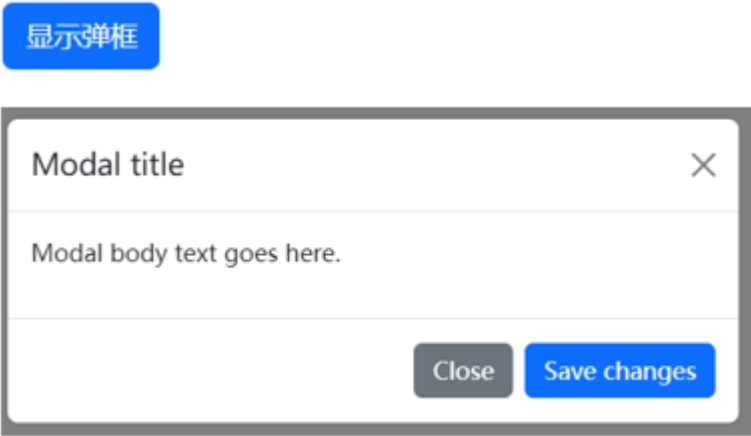
使用属性方式控制 Bootstrap 弹框的显示和隐藏

讲解

- 1. 什么是 Bootstrap 弹框？
 - 不离开当前页面，显示单独内容，供用户操作



- 2. 需求：使用 Bootstrap 弹框，先做个简单效果，点击按钮，让弹框出现，点击 X 和 Close 让弹框隐藏



- 3. 如何使用 Bootstrap 弹框呢？
 - 1. 先引入 bootstrap.css 和 bootstrap.js 到自己网页中

2. 准备弹框标签，确认结构（可以从 Bootstrap 官方文档的 Modal 里复制基础例子）- 运行到网页后，逐一
对应标签和弹框每个部分对应关系
3. 通过自定义属性，通知弹框的显示和隐藏，语法如下：

```
<button data-bs-toggle="modal" data-bs-target="#css选择器">
  显示弹框
</button>

<button data-bs-dismiss="modal">Close</button>
```

4. 去代码区实现一下

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bootstrap 弹框</title>
  <!-- 引入bootstrap.css -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>
  <!--
    目标：使用Bootstrap弹框
    1. 引入bootstrap.css 和 bootstrap.js
    2. 准备弹框标签，确认结构
    3. 通过自定义属性，控制弹框的显示和隐藏
  -->

  <button type="button" class="btn btn-primary" data-bs-toggle="modal" data-bs-target=".my-
box">
    显示弹框
  </button>

  <!--
    弹框标签
    bootstrap的modal弹框，添加modal类名（默认隐藏）
  -->
  <div class="modal my-box" tabindex="-1">
    <div class="modal-dialog">
      <!-- 弹框-内容 -->
      <div class="modal-content">
        <!-- 弹框-头部 -->
        <div class="modal-header">
          <h5 class="modal-title">Modal title</h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal" aria-
label="Close"></button>
        </div>
        <!-- 弹框-身体 -->
```

```
<div class="modal-body">
  <p>Modal body text goes here.</p>
</div>
<!-- 弹框-底部 -->
<div class="modal-footer">
  <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Close</button>
  <button type="button" class="btn btn-primary">Save changes</button>
</div>
</div>
</div>

<!-- 引入bootstrap.js -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.2/dist/js/bootstrap.min.js">
</script>
</body>

</html>
```

小结

1. 用哪个属性绑定来控制弹框显示呢?
 - ▶ 答案
2. 用哪个属性来控制隐藏弹框呢?
 - ▶ 答案

03.Bootstrap 弹框_JS控制

目标

使用 JS 方式控制 Bootstrap 弹框的显示和隐藏

讲解

1. 为什么需要 JS 方式控制呢?
 - 当我显示之前, 隐藏之前, 需要执行一些 JS 逻辑代码, 就需要引入 JS 控制弹框显示/隐藏的方式了
 - 例如:
 - 点击编辑姓名按钮, 在弹框显示之前, 在输入框填入默认姓名
 - 点击保存按钮, 在弹框隐藏之前, 获取用户填入的名字并打印



2. 所以在现实和隐藏之前，需要执行 JS 代码逻辑，就使用 JS 方式 控制 Bootstrap 弹框显示和隐藏语法如下：

```
// 创建弹框对象
const modalDom = document.querySelector('css选择器')
const modal = new bootstrap.Modal(modalDom)

// 显示弹框
modal.show()
// 隐藏弹框
modal.hide()
```

3. 去代码区实现一下

```
// 1. 创建弹框对象
const modalDom = document.querySelector('.name-box')
const modal = new bootstrap.Modal(modalDom)

// 编辑姓名->点击->赋予默认姓名->弹框显示
document.querySelector('.edit-btn').addEventListener('click', () => {
  document.querySelector('.username').value = '默认姓名'

  // 2. 显示弹框
  modal.show()
})

// 保存->点击->获取姓名打印->弹框隐藏
document.querySelector('.save-btn').addEventListener('click', () => {
  const username = document.querySelector('.username').value
  console.log('模拟把姓名保存到服务器上', username)

  // 2. 隐藏弹框
  modal.hide()
})
```

小结

- 1. 什么时候用属性控制，什么时候用 JS 控制 Bootstrap 弹框的显示/隐藏?
▶ 答案

04.案例_图书管理_渲染列表

目标

完成图书管理案例-图书列表数据渲染效果

讲解

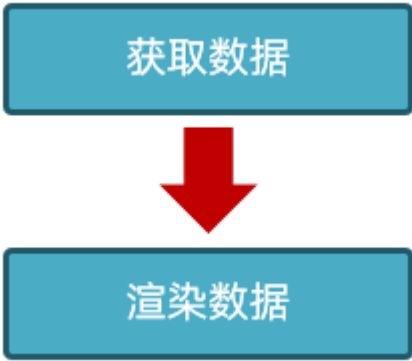
- 1. 需求：基于 axios 获取到图书列表数据，并用 JS 代码渲染数据，到准备好的模板标签中

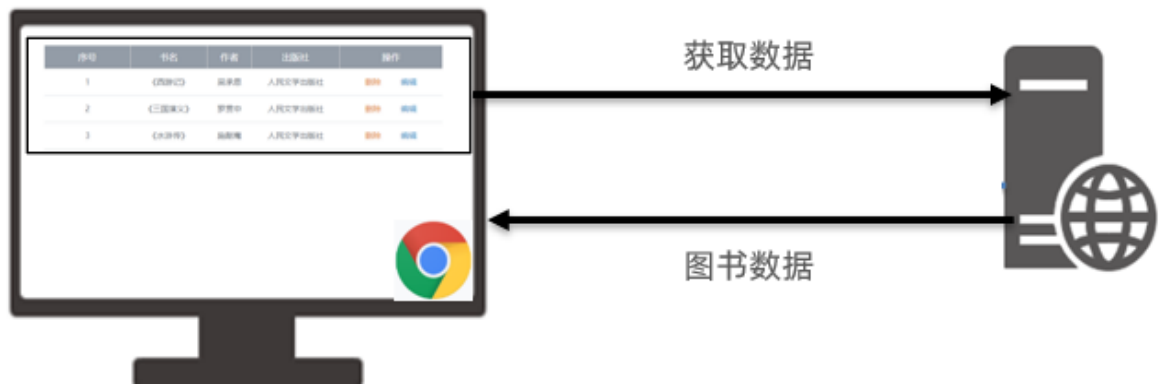
图书管理

+ 添加

序号	书名	作者	出版社	操作	
1	《西游记》	吴承恩	人民文学出版社	删除	编辑
2	《三国演义》	罗贯中	人民文学出版社	删除	编辑
3	《水浒传》	施耐庵	人民文学出版社	删除	编辑

- 2. 步骤：
 - 1. 获取数据
 - 2. 渲染数据





3. 获取数据的时候，需要给自己起一个外号，为什么需要给自己起一个外号呢？

- 我们所有人数据都来自同一个服务器上，为了区分每个同学不同的数据，需要大家设置一个外号告诉服务器，服务器就会返回你对应的图书数据了

4. 核心代码如下：

因为默认展示列表，新增，修改，删除后都要重新获取并刷新列表，所以把获取数据渲染数据的代码封装在一个函数内，方便复用

```
/**
 * 目标1：渲染图书列表
 * 1.1 获取数据
 * 1.2 渲染数据
 */
const creator = '老张'
// 封装-获取并渲染图书列表函数
function getBooksList() {
  // 1.1 获取数据
  axios({
    url: 'http://hmajax.itheima.net/api/books',
    params: {
      // 外号：获取对应数据
      creator
    }
  }).then(result => {
    // console.log(result)
    const bookList = result.data.data
    // console.log(bookList)
    // 1.2 渲染数据
    const htmlStr = bookList.map((item, index) => {
      return `<tr>
        <td>${index + 1}</td>
        <td>${item.bookname}</td>
        <td>${item.author}</td>
        <td>${item.publisher}</td>
        <td data-id=${item.id}>
          <span class="del">删除</span>
          <span class="edit">编辑</span>
        </td>
      </tr>`
    }).join('')
```



```
// console.log(htmlStr)
document.querySelector('.list').innerHTML = htmlStr
})
}
// 网页加载运行，获取并渲染列表一次
getBooksList()
```

小结

1. 渲染数据列表的2个步骤是什么？

► 答案

05.案例_图书管理_新增图书

目标

完成图书管理案例-新增图书需求

讲解

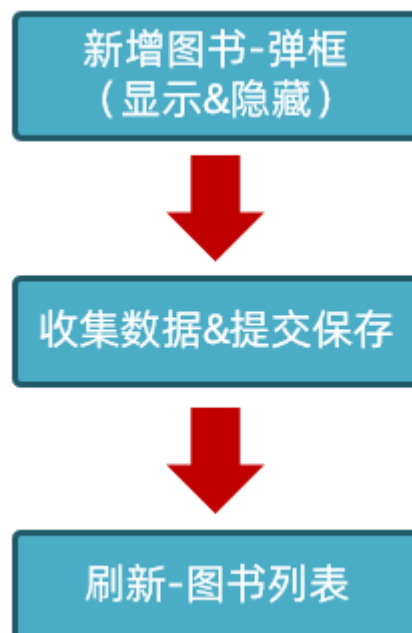
1. 需求：点击添加按钮，出现准备好的新增图书弹框，填写图书信息提交到服务器保存，并更新图书列表





2. 步骤：

1. 新增弹框（控制显示和隐藏）（基于 Bootstrap 弹框和准备好的表单-用属性和 JS 方式控制）
2. 在点击保存按钮时，收集数据&提交保存
3. 刷新-图书列表（重新调用下之前封装的获取并渲染列表的函数）



3. 核心代码如下：

```

/**
 * 目标2：新增图书
 * 2.1 新增弹框->显示和隐藏
 * 2.2 收集表单数据，并提交到服务器保存
 * 2.3 刷新图书列表
 */
// 2.1 创建弹框对象
const addModalDom = document.querySelector('.add-modal')
const addModal = new bootstrap.Modal(addModalDom)
// 保存按钮->点击->隐藏弹框
document.querySelector('.add-btn').addEventListener('click', () => {
  // 2.2 收集表单数据，并提交到服务器保存
  const addForm = document.querySelector('.add-form')
  const bookObj = serialize(addForm, { hash: true, empty: true })
  // console.log(bookObj)
  // 提交到服务器
  axios({

```

```
url: 'http://hmajax.itheima.net/api/books',
method: 'POST',
data: {
  ...bookObj,
  creator
}
}).then(result => {
  // console.log(result)
  // 2.3 添加成功后，重新请求并渲染图书列表
  getBooksList()
  // 重置表单
  addForm.reset()
  // 隐藏弹框
  addModal.hide()
})
})
```

小结

1. 新增数据的3个步骤是什么？

► 答案

06.案例_图书管理_删除图书

目标

完成图书管理案例-删除图书需求

讲解

1. 需求：点击图书删除元素，删除当前图书数据

图书管理

+ 添加

序号	书名	作者	出版社	操作	
1	前端JS	黑马程序员	北京出版社	删除	编辑
2	Java从入门到精通	黑马程序员	北京出版社	删除	编辑
3	前端开发	黑马	人民文学出版社	删除	编辑
4	前端与移动开发	黑马程序员	人民邮电出版社	删除	编辑
5	《西游记》	吴承恩	人民文学出版社	删除	编辑
6	《三国演义》	罗贯中	人民文学出版社	删除	编辑
7	《水浒传》	施耐庵	人民文学出版社	删除	编辑



2. 步骤：

1. 给删除元素，绑定点击事件（事件委托方式并判断点击的是删除元素才走删除逻辑代码），并获取到要删除的数据id
2. 基于 axios 和接口文档，调用删除接口，让服务器删除这条数据
3. 重新获取并刷新图书列表



3. 核心代码如下：

```

/**
 * 目标3：删除图书
 * 3.1 删除元素绑定点击事件->获取图书id
 * 3.2 调用删除接口
 * 3.3 刷新图书列表
 */
// 3.1 删除元素->点击（事件委托）
document.querySelector('.list').addEventListener('click', e => {
  // 获取触发事件目标元素
  // console.log(e.target)
  // 判断点击的是删除元素
  if (e.target.classList.contains('del')) {
    // console.log('点击删除元素')
    // 获取图书id（自定义属性id）
    const theId = e.target.parentNode.dataset.id
    // console.log(theId)
    // 3.2 调用删除接口
    axios({
      url: `http://hmajax.itheima.net/api/books/${theId}`,
      method: 'DELETE'
    }).then(() => {
      // 3.3 刷新图书列表
      getBooksList()
    })
  }
})

```

小结

1. 删除数据的步骤是什么？

► 答案

07-09.案例_图书管理_编辑图书

目标

完成图书管理案例-编辑图书需求

讲解

- 1. 因为编辑图书要做回显等，比较复杂，所以分了3个视频来讲解
- 2. 需求：完成编辑图书回显当前图书数据到编辑表单，在用户点击修改按钮，收集数据提交到服务器保存，并刷新列表



- 3. 编辑数据的核心思路：
 - 1. 给编辑元素，绑定点击事件（事件委托方式并判断点击的是编辑元素才走编辑逻辑代码），并获取到要编辑的数据id
 - 2. 基于 axios 和接口文档，调用查询图书详情接口，获取正在编辑的图书数据，并回显到表单中（页面上的数据是在用户的浏览器中不够准备，所以只要是查看数据都要从服务器获取）



- 3. 收集并提交保存修改数据，并重新从服务器获取列表刷新页面



4. 核心代码如下：

```
/**
 * 目标4：编辑图书
 * 4.1 编辑弹框->显示和隐藏
 * 4.2 获取当前编辑图书数据->回显到编辑表单中
 * 4.3 提交保存修改，并刷新列表
 */
// 4.1 编辑弹框->显示和隐藏
const editDom = document.querySelector('.edit-modal')
const editModal = new bootstrap.Modal(editDom)
// 编辑元素->点击->弹框显示
document.querySelector('.list').addEventListener('click', e => {
  // 判断点击的是否为编辑元素
  if (e.target.classList.contains('edit')) {
    // 4.2 获取当前编辑图书数据->回显到编辑表单中
    const theId = e.target.parentNode.dataset.id
    axios({
      url: `http://hmajax.itheima.net/api/books/${theId}`
    }).then(result => {
      const bookObj = result.data.data
      // document.querySelector('.edit-form .bookname').value = bookObj.bookname
      // document.querySelector('.edit-form .author').value = bookObj.author
      // 数据对象“属性”和标签“类名”一致
      // 遍历数据对象，使用属性去获取对应的标签，快速赋值
      const keys = Object.keys(bookObj) // ['id', 'bookname', 'author', 'publisher']
      keys.forEach(key => {
        document.querySelector(`.edit-form .${key}`).value = bookObj[key]
      })
    })
    editModal.show()
  }
})
// 修改按钮->点击->隐藏弹框
```

```
document.querySelector('.edit-btn').addEventListener('click', () => {
  // 4.3 提交保存修改，并刷新列表
  const editForm = document.querySelector('.edit-form')
  const { id, bookname, author, publisher } = serialize(editForm, { hash: true, empty: true })
  // 保存正在编辑的图书id，隐藏起来：无需让用户修改
  // <input type="hidden" class="id" name="id" value="84783">
  axios({
    url: `http://hmajax.itheima.net/api/books/${id}`,
    method: 'PUT',
    data: {
      bookname,
      author,
      publisher,
      creator
    }
  }).then(() => {
    // 修改成功以后，重新获取并刷新列表
    getBooksList()

    // 隐藏弹框
    editModal.hide()
  })
})
```

小结

1. 编辑数据的步骤是什么？

► 答案

10.案例_图书管理_总结

目标

总结下增删改查的核心思路

讲解

1. 因为增删改查的业务在前端实际开发中非常常见，思路是可以通用的，所以总结下思路

- 1.渲染列表（查）
- 2.新增图书（增）

3.删除图书（删）

4.编辑图书（改）

图书管理

+ 添加

序号	书名	作者	出版社	操作	
1	《西游记》	吴承恩	人民文学出版社	删除	编辑
2	《三国演义》	罗贯中	人民文学出版社	删除	编辑
3	《水浒传》	施耐庵	人民文学出版社	删除	编辑

2. 渲染数据（查）

核心思路：获取数据 -> 渲染数据

```
// 1.1 获取数据
axios({...}).then(result => {
  const bookList = result.data.data
  // 1.2 渲染数据
  const htmlStr = bookList.map((item, index) => {
    return `<tr>
      <td>${index + 1}</td>
      <td>${item.bookname}</td>
      <td>${item.author}</td>
      <td>${item.publisher}</td>
      <td data-id=${item.id}>
        <span class="del">删除</span>
        <span class="edit">编辑</span>
      </td>
    </tr>`
  }).join('')
  document.querySelector('.list').innerHTML = htmlStr
})
```

3. 新增数据（增）

核心思路：准备页面标签 -> 收集数据提交（必须） -> 刷新页面列表（可选）

```
// 2.1 创建弹框对象
const addModalDom = document.querySelector('.add-modal')
const addModal = new bootstrap.Modal(addModalDom)
document.querySelector('.add-btn').addEventListener('click', () => {
  // 2.2 收集表单数据，并提交到服务器保存
})
```

```
const addForm = document.querySelector('.add-form')
const bookObj = serialize(addForm, { hash: true, empty: true })
axios({...}).then(result => {
  // 2.3 添加成功后，重新请求并渲染图书列表
  getBooksList()
  addForm.reset()
  addModal.hide()
})
})
```

添加图书

书名

请输入书籍名称

作者

请输入作者名称

出版社

请输入出版社名称

取消

保存

4. 删除图书（删）

核心思路：绑定点击事件（获取要删除的图书唯一标识） -> 调用删除接口（让服务器删除此数据） -> 成功后重新获取并刷新列表

```
// 3.1 删除元素->点击（事件委托）
document.querySelector('.list').addEventListener('click', e => {
  if (e.target.classList.contains('del')) {
    // 获取图书id（自定义属性id）
    const theId = e.target.parentNode.dataset.id
    // 3.2 调用删除接口
    axios({...}).then(() => {
      // 3.3 刷新图书列表
      getBooksList()
    })
  }
})
```

图书管理

[+ 添加](#)

序号	书名	作者	出版社	操作
1	《西游记》	吴承恩	人民文学出版社	删除 编辑
2	《三国演义》	罗贯中	人民文学出版社	删除 编辑
3	《水浒传》	施耐庵	人民文学出版社	删除 编辑

5. 编辑图书（改）

核心思路：准备编辑图书表单 -> 表单回显正在编辑的数据 -> 点击修改收集数据 -> 提交到服务器保存 -> 重新获取并刷新列表

```
// 4.1 编辑弹框->显示和隐藏
const editDom = document.querySelector('.edit-modal')
const editModal = new bootstrap.Modal(editDom)
document.querySelector('.list').addEventListener('click', e => {
  if (e.target.classList.contains('edit')) {
    // 4.2 获取当前编辑图书数据->回显到编辑表单中
    const theId = e.target.parentNode.dataset.id
    axios({...}).then(result => {
      const bookObj = result.data.data
      // 遍历数据对象，使用属性去获取对应的标签，快速赋值
      const keys = Object.keys(bookObj)
      keys.forEach(key => {
        document.querySelector(`.edit-form .${key}`).value = bookObj[key]
      })
    })
    editModal.show()
  }
})

document.querySelector('.edit-btn').addEventListener('click', () => {
  // 4.3 提交保存修改，并刷新列表
  const editForm = document.querySelector('.edit-form')
  const { id, bookname, author, publisher } = serialize(editForm, { hash: true, empty: true })
  // 保存正在编辑的图书id，隐藏起来：无需让用户修改
  // <input type="hidden" class="id" name="id" value="84783">
  axios({...}).then(() => {
```

```
getBooksList()  
    editModal.hide()  
  })  
})
```

编辑图书

书名

《西游记》

作者

吴承恩

出版社

人民文学出版社

取消

修改

小结

1. 学完图书管理案例，我们收货了什么？

► 答案

11. 图片上传

目标

把本地图片上传到网页上显示

讲解

1. 什么是图片上传？
 - 就是把本地的图片上传到网页上显示
2. 图片上传怎么做？
 - 先依靠文件选择元素获取用户选择的本地文件，接着提交到服务器保存，服务器会返回图片的 url 网址，然后把网址加载到 img 标签的 src 属性中即可显示
3. 为什么不直接显示到浏览器上，要放到服务器上呢？
 - 因为浏览器保存是临时的，如果你想随时随地访问图片，需要上传到服务器上
4. 图片上传怎么做呢？
 1. 先获取图片文件对象
 2. 使用 FormData 表单数据对象装入（因为图片是文件而不是以前的数字和字符串了所以传递文件一般需要放入 FormData 以键值对-文件流的数据传递（可以查看请求体-确认请求体结构）

```
const fd = new FormData()  
fd.append(参数名, 值)
```

3. 提交表单数据对象，使用服务器返回图片 url 网址
5. 核心代码如下：

```
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>图片上传</title>  
</head>  
  
<body>  
  <!-- 文件选择元素 -->  
  <input type="file" class="upload">  
  <img src="" alt="" class="my-img">  
  
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>  
  <script>  
    /**  
     * 目标：图片上传，显示到网页上  
     * 1. 获取图片文件  
     * 2. 使用 FormData 携带图片文件  
     * 3. 提交到服务器，获取图片url网址使用  
     */  
    // 文件选择元素->change改变事件  
    document.querySelector('.upload').addEventListener('change', e => {  
      // 1. 获取图片文件  
      console.log(e.target.files[0])  
      // 2. 使用 FormData 携带图片文件  
      const fd = new FormData()
```

```
fd.append('img', e.target.files[0])
// 3. 提交到服务器，获取图片url网址使用
axios({
  url: 'http://hmajax.itheima.net/api/uploadimg',
  method: 'POST',
  data: fd
}).then(result => {
  console.log(result)
  // 取出图片url网址，用img标签加载显示
  const imgUrl = result.data.data.url
  document.querySelector('.my-img').src = imgUrl
})
})
</script>
</body>

</html>
```

小结

1. 图片上传的思路是什么？

► 答案

12.案例_网站-更换背景图

目标

实现更换网站背景图的效果

讲解

1. 需求：先运行备课代码，查看要完成的效果，点击右上角选择本机中提供的素材图片，更换网站背景图



2. 网站更换背景图如何实现呢，并且保证刷新后背景图还在？具体步骤：

1. 先获取到用户选择的背景图片，上传并把服务器返回的图片 url 网址设置给 body 背景
2. 上传成功时，保存图片 url 网址到 localStorage 中
3. 网页运行后，获取 localStorage 中的图片的 url 网址使用（并判断本地有图片 url 网址字符串才设置）

3. 核心代码如下：

```
/**
 * 目标：网站-更换背景
 * 1. 选择图片上传，设置body背景
 * 2. 上传成功时，"保存"图片url网址
 * 3. 网页运行后，"获取"url网址使用
 * */
document.querySelector('.bg-ipt').addEventListener('change', e => {
  // 1. 选择图片上传，设置body背景
  console.log(e.target.files[0])
  const fd = new FormData()
  fd.append('img', e.target.files[0])
  axios({
    url: 'http://hmajax.itheima.net/api/uploading',
    method: 'POST',
    data: fd
  }).then(result => {
    const imgUrl = result.data.data.url
    document.body.style.backgroundImage = `url(${imgUrl})`

    // 2. 上传成功时，"保存"图片url网址
    localStorage.setItem('bgImg', imgUrl)
  })
})

// 3. 网页运行后，"获取"url网址使用
const bgUrl = localStorage.getItem('bgImg')
console.log(bgUrl)
bgUrl && (document.body.style.backgroundImage = `url(${bgUrl})`)
```

小结

1. localStorage 取值和赋值的语法分别是什么？

► 答案

13.案例_个人信息设置-介绍

目标

介绍个人信息设置案例-需要完成哪些效果，分几个视频讲解

讲解

1. 需求：先运行备课代码，查看要完成的效果

The screenshot displays a user profile settings interface. On the left, a sidebar contains links for '基本设置' (Basic Settings), '安全设置' (Security Settings), '账号绑定' (Account Binding), and '新消息通知' (New Message Notifications). The '基本设置' section is active. The main content area includes a green '操作成功' (Operation Successful) notification at the top. Below it, the '基本设置' (Basic Settings) section contains the following fields: '邮箱' (Email) with the value 'itheima@itcast.cn', '昵称' (Nickname) with the value 'itheima', '性别' (Gender) with '男' (Male) selected via a radio button, and '个人简介' (Bio) with the text '我是播仔'. To the right of these fields is the '头像' (Avatar) section, which shows a cartoon character and a '更换头像' (Change Avatar) button. At the bottom of the form is a blue '提交' (Submit) button.

2. 本视频分为，信息回显 + 头像修改 + 信息修改+ 提示框反馈 4 部分
 1. 先完成信息回显
 2. 再做头像修改-立刻就更新给此用户
 3. 收集个人信息表单-提交保存

- 提交后反馈结果给用户（提示框）

小结

暂无

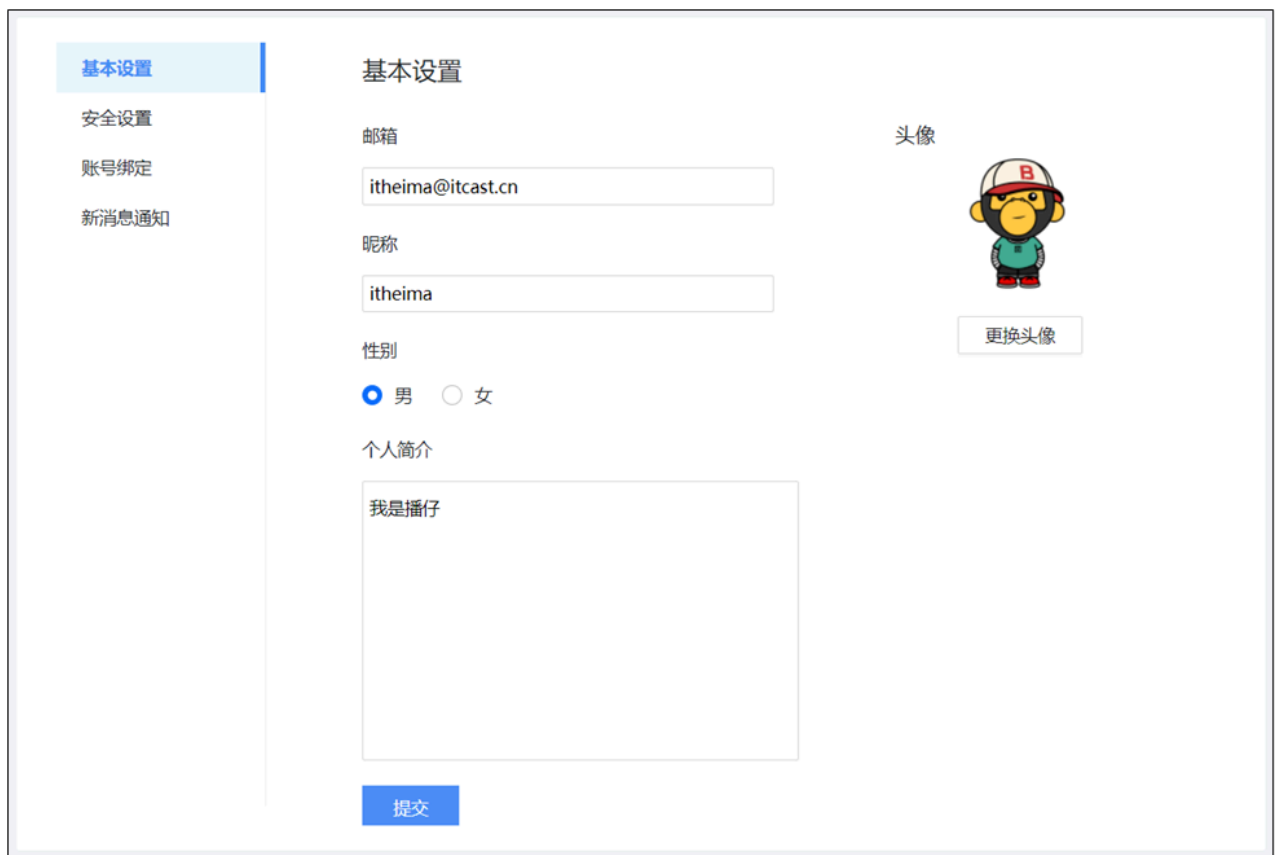
14.案例_个人信息设置-信息渲染

目标

把外号对应的用户信息渲染到页面上

讲解

- 需求：把外号对应的个人信息和头像，渲染到页面表单和头像标签上。



- 注意：还是需要准备一个外号，因为想要查看自己对应的用户信息，不想被别人影响
- 步骤：
 - 获取数据
 - 渲染数据到页面
- 代码如下：

```

/**
 * 目标1：信息渲染
 * 1.1 获取用户的数据
 * 1.2 回显数据到标签上
 */
const creator = '播仔'
// 1.1 获取用户的数据
axios({
  url: 'http://hmajax.itheima.net/api/settings',
  params: {
    creator
  }
}).then(result => {
  const userObj = result.data.data
  // 1.2 回显数据到标签上
  Object.keys(userObj).forEach(key => {
    if (key === 'avatar') {
      // 赋予默认头像
      document.querySelector('.prew').src = userObj[key]
    } else if (key === 'gender') {
      // 赋予默认性别
      // 获取性别单选框：[男radio元素，女radio元素]
      const gRadioList = document.querySelectorAll('.gender')
      // 获取性别数字：0男，1女
      const gNum = userObj[key]
      // 通过性别数字，作为下标，找到对应性别单选框，设置选中状态
      gRadioList[gNum].checked = true
    } else {
      // 赋予默认内容
      document.querySelector(`.${key}`).value = userObj[key]
    }
  })
})
})

```

小结

1. 渲染数据和图书列表的渲染思路是否一样呢，是什么？

► 答案

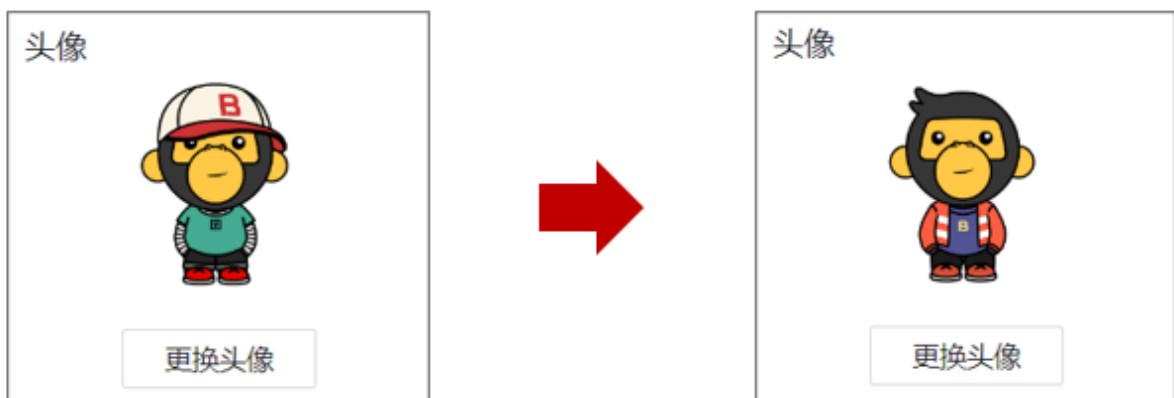
15.案例_个人信息设置-头像修改

目标

修改用户的头像并立刻生效

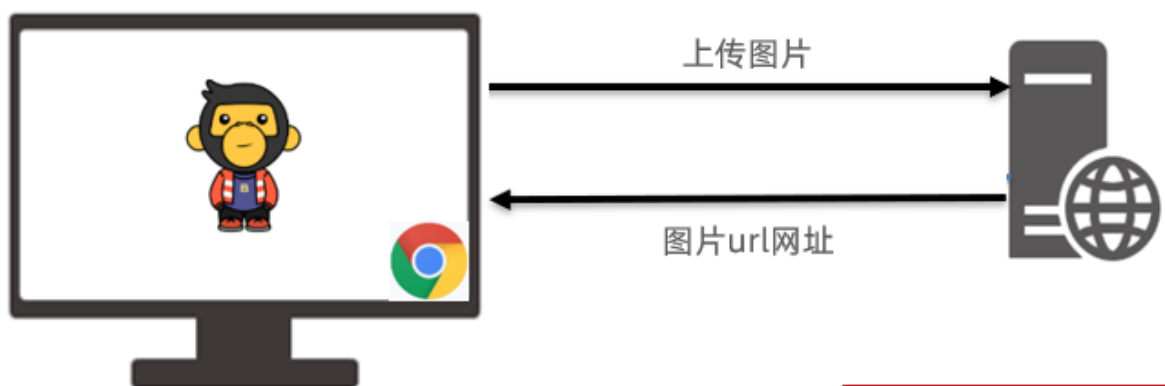
讲解

1. 需求：点击修改用户头像



2. 实现步骤如下：

1. 获取到用户选择的头像文件
2. 调用头像修改接口，并除了头像文件外，还要在 FormData 表单数据对象中携带外号
3. 提交到服务器保存此用户对应头像文件，并把返回的头像图片 url 网址设置在页面上



3. 注意：重新刷新重新获取，已经是修改后的头像了（证明服务器那边确实保存成功）
4. 核心代码：

```
/**
 * 目标2：修改头像
 * 2.1 获取头像文件
 * 2.2 提交服务器并更新头像
 * */
// 文件选择元素->change事件
document.querySelector('.upload').addEventListener('change', e => {
```

```
// 2.1 获取头像文件
console.log(e.target.files[0])
const fd = new FormData()
fd.append('avatar', e.target.files[0])
fd.append('creator', creator)
// 2.2 提交服务器并更新头像
axios({
  url: 'http://hmajax.itheima.net/api/avatar',
  method: 'PUT',
  data: fd
}).then(result => {
  const imgUrl = result.data.data.avatar
  // 把新的头像回显到页面上
  document.querySelector('.preview').src = imgUrl
})
})
```

小结

1. 为什么这次上传头像，需要携带外号呢？

► 答案

16.案例_个人信息设置-信息修改

目标

把用户修改的信息提交到服务器保存

讲解

1. 需求：点击提交按钮，收集个人信息，提交到服务器保存（无需重新获取刷新，因为页面已经是最新的数据了）
 1. 收集表单数据
 2. 提交到服务器保存-调用用户信息更新接口（注意请求方法是 PUT）代表数据更新的意思

基本设置

邮箱

itheima@itcast.cn

昵称

itheima

性别

☒ 男 ☐ 女

个人简介

我是播仔

提交

2. 核心代码如下：

```
/**
 * 目标3：提交表单
 * 3.1 收集表单信息
 * 3.2 提交到服务器保存
 */
// 保存修改->点击
document.querySelector('.submit').addEventListener('click', () => {
  // 3.1 收集表单信息
  const userForm = document.querySelector('.user-form')
  const userObj = serialize(userForm, { hash: true, empty: true })
```

```
userObj.creator = creator
// 性别数字字符串，转成数字类型
userObj.gender = +userObj.gender
console.log(userObj)
// 3.2 提交到服务器保存
axios({
  url: 'http://hmajax.itheima.net/api/settings',
  method: 'PUT',
  data: userObj
}).then(result => {
})
})
```

小结

1. 信息修改数据和以前增删改查哪个实现的思路比较接近呢？

► 答案

17.案例_个人信息设置-提示框

目标

把用户更新个人信息结果，用提示框反馈给用户

讲解

1. 需求：使用 bootstrap 提示框，提示个人信息设置后的结果

基本设置

安全设置

账号绑定

新消息通知

基本设置

操作成功

邮箱

itheima@itcast.cn

昵称

itheima

性别

☒ 男 ☐ 女

个人简介

我是播仔

头像

更换头像

提交

2. bootstrap 的 toast 提示框和 modal 弹框使用很像，语法如下：

1. 先准备对应的标签结构（模板里已有）
2. 设置延迟自动消失的时间

```
<div class="toast" data-bs-delay="1500">
  提示框内容
</div>
```

3. 使用 JS 的方式，在 axios 请求响应成功时，展示结果

```
// 创建提示框对象
const toastDom = document.querySelector('css选择器')
const toast = new bootstrap.Toast(toastDom)

// 显示提示框
toast.show()
```

3. 核心代码：

```
/**
 * 目标3：提交表单
 * 3.1 收集表单信息
 * 3.2 提交到服务器保存
 */
/**
 * 目标4：结果提示
 * 4.1 创建toast对象
 * 4.2 调用show方法->显示提示框
```

```

*/
// 保存修改->点击
document.querySelector('.submit').addEventListener('click', () => {
  // 3.1 收集表单信息
  const userForm = document.querySelector('.user-form')
  const userObj = serialize(userForm, { hash: true, empty: true })
  userObj.creator = creator
  // 性别数字字符串, 转成数字类型
  userObj.gender = +userObj.gender
  console.log(userObj)
  // 3.2 提交到服务器保存
  axios({
    url: 'http://hmajax.itheima.net/api/settings',
    method: 'PUT',
    data: userObj
  }).then(result => {
    // 4.1 创建toast对象
    const toastDom = document.querySelector('.my-toast')
    const toast = new bootstrap.Toast(toastDom)

    // 4.2 调用show方法->显示提示框
    toast.show()
  })
})

```

小结

1. bootstrap 弹框什么时候用 JS 方式控制显示呢?

► 答案

今日重点(必须会)

1. 掌握增删改查数据的思路
2. 掌握图片上传的思路和流程
3. 理解调用接口时, 携带外号的作用
4. 了解 bootstrap 弹框的使用

今日作业(必完成)

在配套作业文件夹的md内

参考文献

1. 表单概念->百度百科
2. accept属性->mdn
3. accept属性->菜鸟教程
4. FormData->mdn
5. BS的Model文档
6. axios请求方式别名

Day03_AJAX原理

知识点自测

1. 以下哪个方法可以把 JS 数据类型转成 JSON 字符串类型?

A. JSON.stringify()

B. JSON.parse()

► 答案

2. 以下哪个方法，会延迟一段时间后，再执行函数体，并执行一次就停止?

A. setTimeout(函数体, 毫秒值)

B. setInterval(函数体, 毫秒值)

► 答案

3. 下面代码 result 结果是多少?

```
let obj = {  
  status: 240  
}  
const result = obj.status >= 200 && obj.status < 300
```

A. true

B. 大于

C. 240

D. false

► 答案

4. 下面代码运行结果是多少?

```
let result = 'http://www.baidu.com'  
result += '?a=10'  
result += '&b=20'
```

A: 'http://www.baidu.com'

B: '?a=10'

C: '&b=20'

D: 'http://www.baidu.com?a=10&b=20'

► 答案

5. 哪个事件能实时检测到输入框值的变化？

A: input 事件

B: change 事件

► 答案

目录

- XMLHttpRequest 的学习
- Promise
- 封装简易版 axios
- 案例 - 天气预报

学习目标

1. 了解原生 AJAX 语法 - XMLHttpRequest (XHR)
2. 了解 Promise 的概念和使用
3. 了解 axios 内部工作的大概过程 (XHR + Promise)
4. 案例 - 天气预报

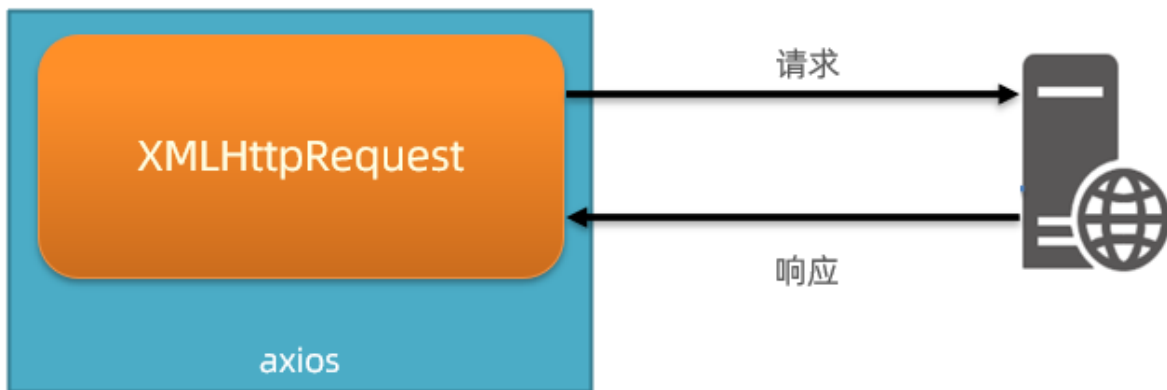
01.XMLHttpRequest - 基础使用

目标

了解 AJAX 原理 XHR 的基础使用

讲解

1. AJAX 是浏览器与服务器通信的技术，采用 XMLHttpRequest 对象相关代码
2. axios 是对 XHR 相关代码进行了封装，让我们只关心传递的接口参数
3. 学习 XHR 也是了解 axios 内部与服务器交互过程的真正原理



4. 语法如下：

```
const xhr = new XMLHttpRequest()
xhr.open('请求方法', '请求url网址')
xhr.addEventListener('loadend', () => {
  // 响应结果
  console.log(xhr.response)
})
xhr.send()
```



5. 需求：以一个需求来体验下原生 XHR 语法，获取所有省份列表并展示到页面上

6. 代码如下：

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>XMLHttpRequest_基础使用</title>
</head>

<body>
  <p class="my-p"></p>
  <script>
    /**
     * 目标：使用XMLHttpRequest对象与服务器通信
     * 1. 创建 XMLHttpRequest 对象
     * 2. 配置请求方法和请求 url 地址
```

```

    * 3. 监听 loadend 事件，接收响应结果
    * 4. 发起请求
*/
// 1. 创建 XMLHttpRequest 对象
const xhr = new XMLHttpRequest()

// 2. 配置请求方法和请求 url 地址
xhr.open('GET', 'http://hmajax.itheima.net/api/province')

// 3. 监听 loadend 事件，接收响应结果
xhr.addEventListener('loadend', () => {
  console.log(xhr.response)
  const data = JSON.parse(xhr.response)
  console.log(data.list.join('<br>'))
  document.querySelector('.my-p').innerHTML = data.list.join('<br>')
})

// 4. 发起请求
xhr.send()
</script>
</body>

</html>
```

小结

1. AJAX 原理是什么?
 - ▶ 答案
2. 为什么学习 XHR ?
 - ▶ 答案
3. XHR 使用步骤?
 - ▶ 答案

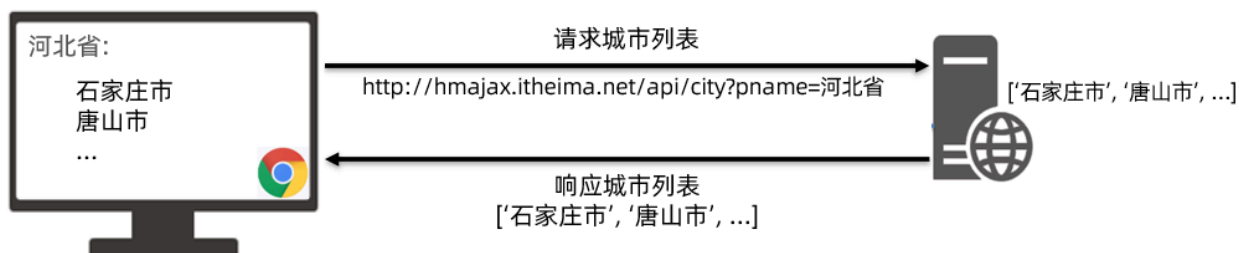
02.XMLHttpRequest - 查询参数

目标

使用 XHR 传递查询参数给服务器，获取匹配数据

讲解

1. 什么是查询参数：携带额外信息给服务器，返回匹配想要的数据库
2. 查询参数原理要携带的位置和语法：<http://xxxx.com/xxx/xxx?参数名1=值1&参数名2=值2>
3. 所以，原生 XHR 需要自己在 url 后面携带查询参数字符串，没有 axios 帮助我们 params 参数拼接到 url 字符串后面了
4. 需求：查询河北省下属的城市列表



5. 核心代码如下：

```
/**
 * 目标：使用XHR携带查询参数，展示某个省下属的城市列表
 */
const xhr = new XMLHttpRequest()
xhr.open('GET', 'http://hmajax.itheima.net/api/city?pname=辽宁省')
xhr.addEventListener('loadend', () => {
  console.log(xhr.response)
  const data = JSON.parse(xhr.response)
  console.log(data)
  document.querySelector('.city-p').innerHTML = data.list.join('<br>')
})
xhr.send()
```

小结

1. XHR 如何携带查询参数？

► 答案

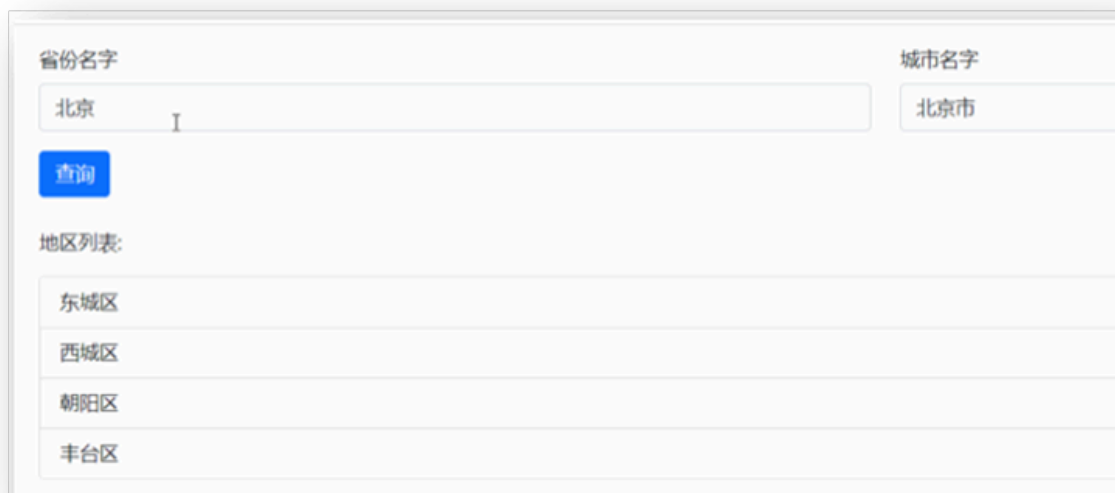
03.案例 - 地区查询

目标

使用 XHR 完成案例地区查询

讲解

1. 需求：和我们之前做的类似，就是不用 axios 而是用 XHR 实现，输入省份和城市名字后，点击查询，传递多对查询参数并获取地区列表的需求



省份名字	城市名字
北京	北京市

查询

地区列表:

东城区
西城区
朝阳区
丰台区

2. 但是多个查询参数，如果自己拼接很麻烦，这里用 URLSearchParams 把参数对象转成“参数名=值&参数名=值”格式的字符串，语法如下：

```
// 1. 创建 URLSearchParams 对象
const paramsObj = new URLSearchParams({
  参数名1: 值1,
  参数名2: 值2
})

// 2. 生成指定格式查询参数字符串
const queryString = paramsObj.toString()
// 结果: 参数名1=值1&参数名2=值2
```

小结

1. JS 对象如何转成查询参数格式字符串？

► 答案

04.XMLHttpRequest - 数据提交

目标

通过 XHR 提交用户名和密码，完成注册功能

讲解

- 1. 了解原生 XHR 进行数据提交的方式
- 2. 需求：通过 XHR 完成注册用户功能

注册账号

POST

http://hmajax.itheima.net/api/register

注册账号

请求参数

Body 参数(application/json)

数据结构

生成代码

username

string

用户名

必需

中英文和数字组成, 最少8位

password

string

密码

必需

最少6位

- 3. 步骤和语法：
 - 1. 注意1：但是这次没有 axios 帮我们了，我们需要自己设置请求头 Content-Type: application/json，来告诉服务器端，我们发过去的内容类型是 JSON 字符串，让他转成对应数据结构取值使用
 - 2. 注意2：没有 axios 了，我们前端要传递的请求体数据，也没人帮我把 JS 对象转成 JSON 字符串了，需要我们自己转换
 - 3. 注意3：原生 XHR 需要在 send 方法调用时，传入请求体携带


```

const xhr = new XMLHttpRequest()
xhr.open('请求方法', '请求url网址')
xhr.addEventListener('loadend', () => {
  console.log(xhr.response)
})

// 1. 告诉服务器，我传递的内容类型，是 JSON 字符串
xhr.setRequestHeader('Content-Type', 'application/json')
// 2. 准备数据并转成 JSON 字符串
const user = { username: 'itheima007', password: '7654321' }
const userStr = JSON.stringify(user)
// 3. 发送请求体数据
xhr.send(userStr)

```

4. 核心代码如下：

```

/**
 * 目标：使用xhr进行数据提交-完成注册功能
 */
document.querySelector('.reg-btn').addEventListener('click', () => {
  const xhr = new XMLHttpRequest()
  xhr.open('POST', 'http://hmajax.itheima.net/api/register')
  xhr.addEventListener('loadend', () => {
    console.log(xhr.response)
  })

  // 设置请求头-告诉服务器内容类型（JSON字符串）
  xhr.setRequestHeader('Content-Type', 'application/json')
  // 准备提交的数据
  const userObj = {
    username: 'itheima007',
    password: '7654321'
  }
  const userStr = JSON.stringify(userObj)
  // 设置请求体，发起请求
  xhr.send(userStr)
})

```

小结

1. XHR 如何提交请求体数据？

► 答案

05.认识_Promise

目标

认识 Promise 的作用和好处以及使用步骤

讲解

1. 什么是 Promise ?
 - Promise 对象用于表示一个异步操作的最终完成（或失败）及其结构值
2. Promise 的好处是什么？
 - 逻辑更清晰（成功或失败会关联后续的处理函数）
 - 了解 axios 函数内部运作的机制



- 能解决回调函数地狱问题（后面会讲到），今天先来看下它的基础使用
3. Promise 管理异步任务，语法怎么用？

```
// 1. 创建 Promise 对象
const p = new Promise((resolve, reject) => {
  // 2. 执行异步任务-并传递结果
  // 成功调用：resolve(值) 触发 then() 执行
  // 失败调用：reject(值) 触发 catch() 执行
})
// 3. 接收结果
p.then(result => {
  // 成功
}).catch(error => {
  // 失败
})
```

4. 示例代码：

```
/**
 * 目标：使用Promise管理异步任务
 */
// 1. 创建Promise对象
const p = new Promise((resolve, reject) => {
  // 2. 执行异步代码
```

```
setTimeout(() => {
  // resolve('模拟AJAX请求-成功结果')
  reject(new Error('模拟AJAX请求-失败结果'))
}, 2000)
})

// 3. 获取结果
p.then(result => {
  console.log(result)
}).catch(error => {
  console.log(error)
})
```

小结

1. 什么是 Promise ?
 - ▶ 答案
2. 为什么学习 Promise ?
 - ▶ 答案
3. Promise 使用步骤?
 - ▶ 答案

06.认识_Promise 的状态

目标

认识 Promise 的三种状态，知道如何关联成功/失败的处理函数

讲解

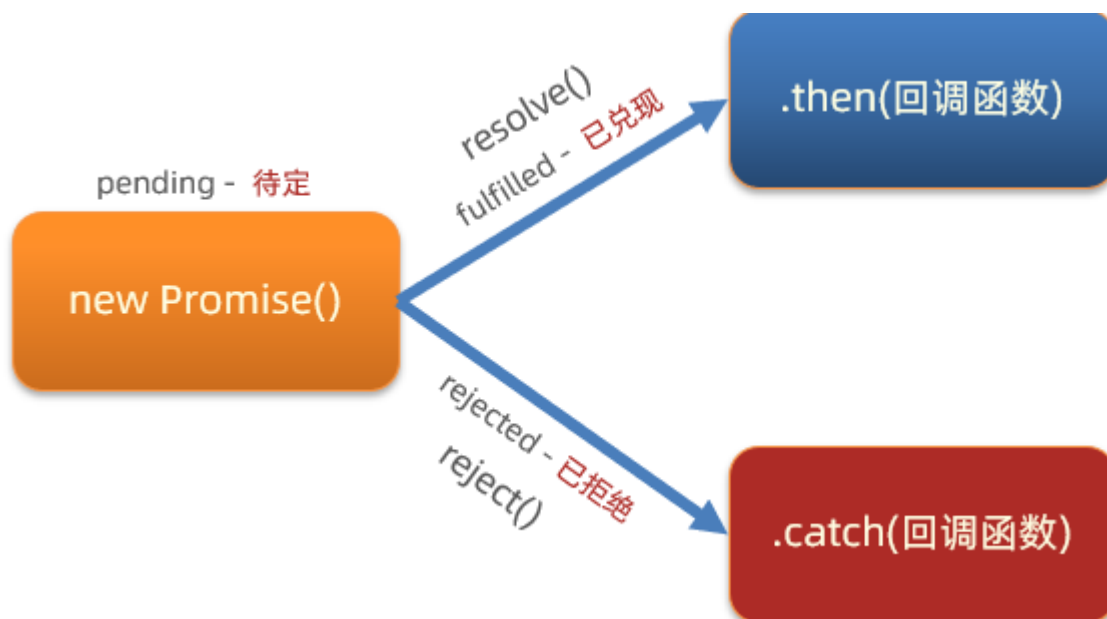
1. 为什么要了解 Promise 的三种状态 ?
 - 知道 Promise 对象如何关联的处理函数，以及代码的执行顺序
2. Promise 有哪三种状态?

每个 Promise 对象必定处于以下三种状态之一

1. 待定 (pending)：初始状态，既没有被兑现，也没有被拒绝
2. 已兑现 (fulfilled)：操作成功完成
3. 已拒绝 (rejected)：操作失败

状态的英文字符串，可以理解为 Promise 对象内的字符串标识符，用于判断什么时候调用哪一个处理函数

3. Promise 的状态改变有什么用：调用对应函数，改变 Promise 对象状态后，内部触发对应回调函数传参并执行



4. 注意：每个 Promise 对象一旦被兑现/拒绝，那就是已敲定了，状态无法再被改变

小结

1. Promise 对象有哪 3 种状态？

▶ 答案

2. Promise 状态有什么用？

▶ 答案

07.使用 Promise 和 XHR_获取省份列表

目标

尝试用 Promise 管理 XHR 异步任务

讲解

1. Promise 和 XHR 都已经学过基础语法了，我们可以来结合使用一下了

2. 需求：使用 Promise 和 XHR 请求省份列表数据并展示到页面上

北京
天津
河北省
山西省
内蒙古自治区
辽宁省
吉林省
黑龙江省
上海
江苏省
浙江省
安徽省
福建省
江西省
山东省
河南省
湖北省

3. 步骤：

1. 创建 Promise 对象
2. 执行 XHR 异步代码，获取省份列表数据
3. 关联成功或失败回调函数，做后续的处理

错误情况：用地址错了404演示

4. 核心代码如下：

```
/**
 * 目标：使用Promise管理XHR请求省份列表
 * 1. 创建Promise对象
 * 2. 执行XHR异步代码，获取省份列表
 * 3. 关联成功或失败函数，做后续处理
 */
// 1. 创建Promise对象
const p = new Promise((resolve, reject) => {
  // 2. 执行XHR异步代码，获取省份列表
  const xhr = new XMLHttpRequest()
  xhr.open('GET', 'http://hmajax.itheima.net/api/province')
  xhr.addEventListener('loadend', () => {
    // xhr如何判断响应成功还是失败的？
    // 2xx开头的都是成功响应状态码
    if (xhr.status >= 200 && xhr.status < 300) {
      resolve(JSON.parse(xhr.response))
    } else {
      reject(new Error(xhr.response))
    }
  })
  xhr.send()
})
```

```
// 3. 关联成功或失败函数，做后续处理
p.then(result => {
  console.log(result)
  document.querySelector('.my-p').innerHTML = result.list.join('<br>')
}).catch(error => {
  // 错误对象要用console.dir详细打印
  console.dir(error)
  // 服务器返回错误提示消息，插入到p标签显示
  document.querySelector('.my-p').innerHTML = error.message
})
```

小结

1. AJAX 如何判断是否请求响应成功了?

► 答案

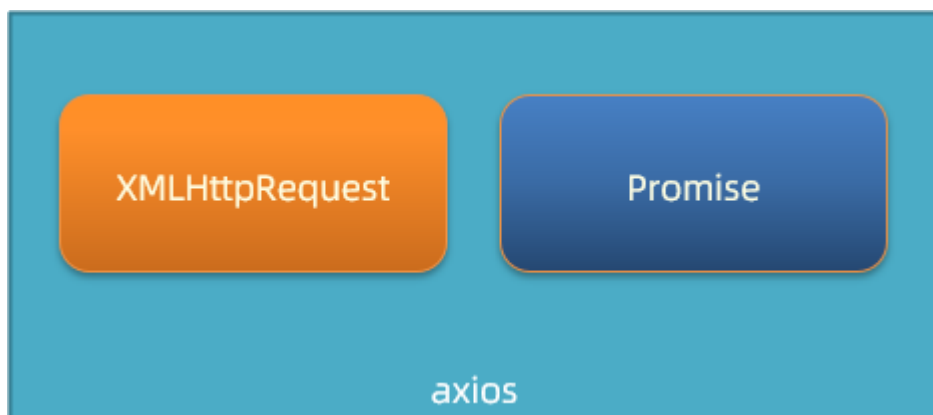
08.封装_简易axios-获取省份列表

目标

模拟 axios 函数封装，更深入了解 axios 内部运作原理

讲解

1. 需求：基于 Promise 和 XMLHttpRequest 封装 myAxios 函数，获取省份列表展示到页面



2. 核心语法：

```
function myAxios(config) {
  return new Promise((resolve, reject) => {
    // XHR 请求
    // 调用成功/失败的处理程序
  })
}
```

```

}

myAxios({
  url: '目标资源地址'
}).then(result => {

}).catch(error => {

})
})

```

3. 步骤：

1. 定义 myAxios 函数，接收配置对象，返回 Promise 对象
2. 发起 XHR 请求，默认请求方法为 GET
3. 调用成功/失败的处理程序
4. 使用 myAxios 函数，获取省份列表展示

4. 核心代码如下：

```

/**
 * 目标：封装_简易axios函数_获取省份列表
 * 1. 定义myAxios函数，接收配置对象，返回Promise对象
 * 2. 发起XHR请求，默认请求方法为GET
 * 3. 调用成功/失败的处理程序
 * 4. 使用myAxios函数，获取省份列表展示
 */
// 1. 定义myAxios函数，接收配置对象，返回Promise对象
function myAxios(config) {
  return new Promise((resolve, reject) => {
    // 2. 发起XHR请求，默认请求方法为GET
    const xhr = new XMLHttpRequest()
    xhr.open(config.method || 'GET', config.url)
    xhr.addEventListener('loadend', () => {
      // 3. 调用成功/失败的处理程序
      if (xhr.status >= 200 && xhr.status < 300) {
        resolve(JSON.parse(xhr.response))
      } else {
        reject(new Error(xhr.response))
      }
    })
    xhr.send()
  })
}

// 4. 使用myAxios函数，获取省份列表展示
myAxios({
  url: 'http://hmajax.itheima.net/api/province'
}).then(result => {
  console.log(result)
  document.querySelector('.my-p').innerHTML = result.list.join('<br>')
})

```

```
}).catch(error => {  
  console.log(error)  
  document.querySelector('.my-p').innerHTML = error.message  
})
```

小结

1. 自己封装的 myAxios 如何设置默认请求方法 GET?

► 答案

09.封装_简易axios-获取地区列表

目标

修改 myAxios 函数支持传递查询参数，获取辽宁省，大连市的地区列表

讲解

1. 需求：在上个封装的建议 axios 函数基础上，修改代码支持传递查询参数功能
2. 修改步骤：
 1. myAxios 函数调用后，判断 params 选项
 2. 基于 URLSearchParams 转换查询参数字符串
 3. 使用自己封装的 myAxios 函数显示地区列表
3. 核心代码：

```
function myAxios(config) {  
  return new Promise((resolve, reject) => {  
    const xhr = new XMLHttpRequest()  
    // 1. 判断有params选项，携带查询参数  
    if (config.params) {  
      // 2. 使用URLSearchParams转换，并携带到url上  
      const paramsObj = new URLSearchParams(config.params)  
      const queryString = paramsObj.toString()  
      // 把查询参数字符串，拼接在url? 后面  
      config.url += `?${queryString}`  
    }  
  
    xhr.open(config.method || 'GET', config.url)  
    xhr.addEventListener('loadend', () => {  
      if (xhr.status >= 200 && xhr.status < 300) {  
        resolve(JSON.parse(xhr.response))  
      }  
    })  
    xhr.send()  
  })  
}
```



```

        } else {
            reject(new Error(xhr.response))
        }
    })
    xhr.send()
})
}

// 3. 使用myAxios函数，获取地区列表
myAxios({
    url: 'http://hmajax.itheima.net/api/area',
    params: {
        pname: '辽宁省',
        cname: '大连市'
    }
}).then(result => {
    console.log(result)
    document.querySelector('.my-p').innerHTML = result.list.join('<br>')
})

```

小结

1. 外面传入查询参数对象，myAxios 函数内如何转查询参数字符串？

► 答案

10.封装_简易axios-注册用户

目标

修改 myAxios 函数支持传递请求体数据，完成注册用户

讲解

1. 需求：修改 myAxios 函数支持传递请求体数据，完成注册用户功能
2. 修改步骤：
 1. myAxios 函数调用后，判断 data 选项
 2. 转换数据类型，在 send 方法中发送
 3. 使用自己封装的 myAxios 函数完成注册用户功能
3. 核心代码：

```

function myAxios(config) {
    return new Promise((resolve, reject) => {
        const xhr = new XMLHttpRequest()

```

```

    if (config.params) {
      const paramsObj = new URLSearchParams(config.params)
      const queryString = paramsObj.toString()
      config.url += `?${queryString}`
    }
    xhr.open(config.method || 'GET', config.url)

    xhr.addEventListener('loadend', () => {
      if (xhr.status >= 200 && xhr.status < 300) {
        resolve(JSON.parse(xhr.response))
      } else {
        reject(new Error(xhr.response))
      }
    })
    // 1. 判断有data选项，携带请求体
    if (config.data) {
      // 2. 转换数据类型，在send中发送
      const jsonStr = JSON.stringify(config.data)
      xhr.setRequestHeader('Content-Type', 'application/json')
      xhr.send(jsonStr)
    } else {
      // 如果没有请求体数据，正常的发起请求
      xhr.send()
    }
  })
}

document.querySelector('.reg-btn').addEventListener('click', () => {
  // 3. 使用myAxios函数，完成注册用户
  myAxios({
    url: 'http://hmajax.itheima.net/api/register',
    method: 'POST',
    data: {
      username: 'itheima999',
      password: '666666'
    }
  }).then(result => {
    console.log(result)
  }).catch(error => {
    console.dir(error)
  })
})
})

```

小结

- 1. 外面传入 data 选项，myAxios 函数内如何携带请求体参数?
▶ 答案

11-12.案例_天气预报-默认数据

目标

把北京市的数据，填充到页面默认显示

讲解

- 1. 需求：介绍本项目要完成的效果，和要实现的步骤和分的步骤和视频



- 2. 步骤
 - 1. 先获取北京市天气预报，展示
 - 2. 搜索城市列表，展示
 - 3. 点击城市，切换显示对应天气数据
- 3. 本视频先封装函数，获取城市天气并设置页面内容
- 4. 核心代码如下：

```
/**
 * 目标1：默认显示-北京市天气
 * 1.1 获取北京市天气数据
 * 1.2 数据展示到页面
 */
```

```
// 获取并渲染城市天气函数
function getWeather(cityCode) {
  // 1.1 获取北京市天气数据
  myAxios({
    url: 'http://hmajax.itheima.net/api/weather',
    params: {
      city: cityCode
    }
  }).then(result => {
    console.log(result)
    const wObj = result.data
    // 1.2 数据展示到页面
    // 阳历和农历日期
    const dateStr = `${wObj.date}</span>
<span class="calendar">农历&nbsp;
  <span class="dateLunar">${wObj.dateLunar}</span>
</span>`
    document.querySelector('.title').innerHTML = dateStr
    // 城市名字
    document.querySelector('.area').innerHTML = wObj.area
    // 当天气温
    const nowWStr = `
```

```

<ul class="sun-list">
  <li>
    <span>紫外线</span>
    <span class="ultraviolet">${twObj.ultraviolet}</span>
  </li>
  <li>
    <span>湿度</span>
    <span class="humidity">${twObj.humidity}</span>%
  </li>
  <li>
    <span>日出</span>
    <span class="sunriseTime">${twObj.sunriseTime}</span>
  </li>
  <li>
    <span>日落</span>
    <span class="sunsetTime">${twObj.sunsetTime}</span>
  </li>
</ul>`
document.querySelector('.today-weather').innerHTML = todayWStr

// 7日天气预报数据展示
const dayForecast = wObj.dayForecast
const dayForecastStr = dayForecast.map(item => {
  return `<li class="item">
    <div class="date-box">
      <span class="dateFormat">${item.dateFormat}</span>
      <span class="date">${item.date}</span>
    </div>
    
    <span class="weather">${item.weather}</span>
    <div class="temp">
      <span class="temNight">${item.temNight}</span>-
      <span class="temDay">${item.temDay}</span>
      <span>℃</span>
    </div>
    <div class="wind">
      <span class="windDirection">${item.windDirection}</span>
      <span class="windPower">${item.windPower}</span>
    </div>
  </li>`
}).join('')
// console.log(dayForecastStr)
document.querySelector('.week-wrap').innerHTML = dayForecastStr
})
}

// 默认进入网页-就要获取天气数据（北京市城市编码：'110100'）
getWeather('110100')

```

小结

- 1. 做完这个项目会带来什么收货?
 - ▶ 答案

13.案例_天气预报-搜索城市列表

目标

根据关键字，展示匹配的城市列表

讲解

- 1. 介绍本视频要完成的效果：搜索匹配关键字相关城市名字，展示城市列表即可



- 2. 步骤
 - 1. 绑定 input 事件，获取关键字
 - 2. 获取展示城市列表数据
- 3. 核心代码如下：

```
/**
 * 目标2：搜索城市列表
 * 2.1 绑定input事件，获取关键字
 * 2.2 获取展示城市列表数据
 */
// 2.1 绑定input事件，获取关键字
```

```
document.querySelector('.search-city').addEventListener('input', (e) => {
  console.log(e.target.value)
  // 2.2 获取展示城市列表数据
  myAxios({
    url: 'http://hmajax.itheima.net/api/weather/city',
    params: {
      city: e.target.value
    }
  }).then(result => {
    console.log(result)
    const liStr = result.data.map(item => {
      return `<li class="city-item" data-code="${item.code}">${item.name}</li>`
    }).join('')
    console.log(liStr)
    document.querySelector('.search-list').innerHTML = liStr
  })
})
```

小结

1. 监听输入框实时改变的事件是什么?

► 答案

14.案例_天气预报-展示城市天气

目标

点击搜索框列表城市名字，切换对应城市天气数据

讲解

1. 介绍本视频要完成的效果：点击城市列表名字，切换当前页面天气数据



2. 步骤

1. 检测搜索列表点击事件，获取城市 code 值
2. 复用获取展示城市天气函数

3. 核心代码如下：

```
/**
 * 目标3：切换城市天气
 * 3.1 绑定城市点击事件，获取城市code值
 * 3.2 调用获取并展示天气的函数
 */
// 3.1 绑定城市点击事件，获取城市code值
document.querySelector('.search-list').addEventListener('click', e => {
  if (e.target.classList.contains('city-item')) {
    // 只有点击城市li才会走这里
    const cityCode = e.target.dataset.code
    console.log(cityCode)
    // 3.2 调用获取并展示天气的函数
    getWeather(cityCode)
  }
})
```

小结

1. 这次我们获取城市天气，传递的是城市名字还是 code 值？

► 答案

今日重点(必须会)

1. 了解 AJAX 原理之 XMLHttpRequest (XHR) 相关语法
2. 了解 Promise 的作用和三种状态
3. 了解 axios 内部运作的过程
4. 完成案例-天气预报

今日作业(必完成)

参考作业文件夹作用

参考文档

1. [Ajax原生-mdn](#)
2. [同步异步-mdn](#)
3. [回调函数-mdn](#)
4. [Promise-mdn](#)

Day04_AJAX进阶

知识点自测

1. 看如下标签回答如下问题？

```
<select>
  <option value="北京">北京市</option>
  <option value="南京">南京市</option>
  <option value="天津">天津市</option>
</select>
```

- 当选中第二个 option 时, JS 中获取下拉菜单 select 标签的 value 属性的值是多少?
▶ 答案
- 页面上看到的是北京, 还是北京市等?
▶ 答案

2. 我给 select 标签的 value 属性赋予"南京"会有什么效果？

▶ 答案

目录

- 同步代码和异步代码
- 回调函数地狱和 Promise 链式调用
- async 和 await 使用
- 事件循环-EventLoop
- Promise.all 静态方法
- 案例 - 商品分类
- 案例 - 学习反馈

学习目标

1. 区分异步代码, 回调函数地狱问题和所有解决防范 (Promise 链式调用)
2. 掌握 async 和 await 使用

3. 掌握 EventLoop 的概念
4. 了解 Promise.all 静态方法作用
5. 完成省市切换效果

01.同步代码和异步代码

目标

能够区分出哪些是异步代码

讲解

1. **同步代码**：逐行执行，需原地等待结果后，才继续向下执行
2. **异步代码**：调用后耗时，不阻塞代码继续执行（不必原地等待），在将来完成后触发回调函数传递结果
3. 回答代码打印顺序：发现异步代码接收结果，使用的都是回调函数

```
const result = 0 + 1
console.log(result)
setTimeout(() => {
  console.log(2)
}, 2000)
document.querySelector('.btn').addEventListener('click', () => {
  console.log(3)
})
document.body.style.backgroundColor = 'pink'
console.log(4)
```

结果：1， 4， 2

按钮点击一次打印一次 3

小结

1. 什么是同步代码?
 - ▶ 答案
2. 什么是异步代码?
 - ▶ 答案
3. JS 中有哪些异步代码?

► 答案

4. 异步代码如何接收结果?

► 答案

02.回调函数地狱

目标

了解回调函数地狱的概念和缺点

讲解

1. 需求：展示默认第一个省，第一个城市，第一个地区在下拉菜单中



2. 概念：在回调函数中嵌套回调函数，一直嵌套下去就形成了回调函数地狱

3. 缺点：可读性差，异常无法捕获，耦合性严重，牵一发动全身

```
axios({ url: 'http://hmajax.itheima.net/api/province' }).then(result => {
  const pname = result.data.list[0]
  document.querySelector('.province').innerHTML = pname
  // 获取第一个省份默认下属的第一个城市名字
  axios({ url: 'http://hmajax.itheima.net/api/city', params: { pname } }).then(result => {
    const cname = result.data.list[0]
    document.querySelector('.city').innerHTML = cname
    // 获取第一个城市默认下属第一个地区名字
    axios({ url: 'http://hmajax.itheima.net/api/area', params: { pname, cname }
  }).then(result => {
    document.querySelector('.area').innerHTML = result.data.list[0]
  })
})
})
})
```

小结

1. 什么是回调函数地狱?
 - ▶ 答案
2. 回调函数地狱问题?
 - ▶ 答案

03.Promise-链式调用

目标

了解 Promise 链式调用特点和语法

讲解

1. 概念：依靠 then() 方法会返回一个新生成的 Promise 对象特性，继续串联下一环任务，直到结束
2. 细节：then() 回调函数中的返回值，会影响新生成的 Promise 对象最终状态和结果
3. 好处：通过链式调用，解决回调函数嵌套问题



4. 按照图解，编写核心代码：

```
/**
 * 目标：掌握Promise的链式调用
 * 需求：把省市的嵌套结构，改成链式调用的线性结构
 */
// 1. 创建Promise对象-模拟请求省份名字
const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve('北京市')
  }, 2000)
})

// 2. 获取省份名字
```

```
const p2 = p.then(result => {
  console.log(result)
  // 3. 创建Promise对象-模拟请求城市名字
  // return Promise对象最终状态和结果，影响到新的Promise对象
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve(result + '--- 北京')
    }, 2000)
  })
})

// 4. 获取城市名字
p2.then(result => {
  console.log(result)
})

// then()原地的结果是一个新的Promise对象
console.log(p2 === p)
```

小结

1. 什么是 Promise 的链式调用?
 - ▶ 答案
2. then 回调函数中，return 的值会传给哪里?
 - ▶ 答案
3. Promise 链式调用有什么用?
 - ▶ 答案

04.Promise-链式调用_解决回调地狱

目标

了解 Promise 链式调用解决回调地狱

讲解

1. 目标：使用 Promise 链式调用，解决回调函数地狱问题
2. 做法：每个 Promise 对象中管理一个异步任务，用 then 返回 Promise 对象，串联起来



3. 按照图解思路，编写核心代码：

```
/**
 * 目标：把回调函数嵌套代码，改成Promise链式调用结构
 * 需求：获取默认第一个省，第一个市，第一个地区并展示在下拉菜单中
 */
let pname = ''
// 1. 得到-获取省份Promise对象
axios({url: 'http://hmajax.itheima.net/api/province'}).then(result => {
  pname = result.data.list[0]
  document.querySelector('.province').innerHTML = pname
  // 2. 得到-获取城市Promise对象
  return axios({url: 'http://hmajax.itheima.net/api/city', params: { pname }})
}).then(result => {
  const cname = result.data.list[0]
  document.querySelector('.city').innerHTML = cname
  // 3. 得到-获取地区Promise对象
  return axios({url: 'http://hmajax.itheima.net/api/area', params: { pname, cname }})
}).then(result => {
  console.log(result)
  const areaName = result.data.list[0]
  document.querySelector('.area').innerHTML = areaName
})
```

小结

1. Promise 链式调用如何解决回调函数地狱？

► 答案

05.async 函数和 await

目标

掌握 async 和 await 语法来编写简洁的异步代码

讲解

1. 概念：在 `async` 函数内，使用 `await` 关键字取代 `then` 函数，等待获取 `Promise` 对象成功状态的结果值
2. 做法：使用 `async` 和 `await` 解决回调地狱问题
3. 核心代码：

```
/**
 * 目标：掌握async和await语法，解决回调函数地狱
 * 概念：在async函数内，使用await关键字，获取Promise对象"成功状态"结果值
 * 注意：await必须用在async修饰的函数内（await会阻止"异步函数内"代码继续执行，原地等待结果）
 */
// 1. 定义async修饰函数
async function getData() {
  // 2. await等待Promise对象成功的结果
  const pObj = await axios({url: 'http://hmajax.itheima.net/api/province'})
  const pname = pObj.data.list[0]
  const cObj = await axios({url: 'http://hmajax.itheima.net/api/city', params: { pname }})
  const cname = cObj.data.list[0]
  const aObj = await axios({url: 'http://hmajax.itheima.net/api/area', params: { pname,
cname }})
  const areaName = aObj.data.list[0]

  document.querySelector('.province').innerHTML = pname
  document.querySelector('.city').innerHTML = cname
  document.querySelector('.area').innerHTML = areaName
}

getData()
```

使用 `await` 替代 `then` 的方法

小结

1. `await` 的作用是什么？

► 答案

06.async 函数和 await 捕获错误

目标

了解用 `try catch` 捕获同步流程的错误

讲解

1. try 和 catch 的作用：语句标记要尝试的语句块，并指定一个出现异常时抛出的响应

```
try {  
    // 要执行的代码  
} catch (error) {  
    // error 接收的是，错误消息  
    // try 里代码，如果有错误，直接进入这里执行  
}
```

try 里有报错的代码，会立刻跳转到 catch 中

2. 尝试把代码中 url 地址写错，运行观察 try catch 的捕获错误信息能力

```
/**  
 * 目标: async和await_错误捕获  
 */  
async function getData() {  
    // 1. try包裹可能产生错误的代码  
    try {  
        const pObj = await axios({ url: 'http://hmajax.itheima.net/api/province' })  
        const pname = pObj.data.list[0]  
        const cObj = await axios({ url: 'http://hmajax.itheima.net/api/city', params: { pname } })  
    })  
        const cname = cObj.data.list[0]  
        const aObj = await axios({ url: 'http://hmajax.itheima.net/api/area', params: { pname, cname } })  
        const areaName = aObj.data.list[0]  
  
        document.querySelector('.province').innerHTML = pname  
        document.querySelector('.city').innerHTML = cname  
        document.querySelector('.area').innerHTML = areaName  
    } catch (error) {  
        // 2. 接着调用catch块，接收错误信息  
        // 如果try里某行代码报错后，try中剩余的代码不会执行了  
        console.dir(error)  
    }  
}  
  
getData()
```

小结

1. try 和 catch 有什么作用？

► 答案

07.事件循环

目标

掌握事件循环模型是如何执行异步代码的

讲解

1. 事件循环（EventLoop）：掌握后知道 JS 是如何安排和运行代码的

请回答下面 2 段代码打印的结果，并说明原因

```
console.log(1)
setTimeout(() => {
  console.log(2)
}, 2000)
```

```
console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
console.log(3)
```

2. 作用：事件循环负责执行代码，收集和处理事件以及执行队列中的子任务
3. 原因：JavaScript 单线程（某一刻只能执行一行代码），为了让耗时代码不阻塞其他代码运行，设计了事件循环模型
4. 概念：执行代码和收集异步任务的模型，在调用栈空闲，反复调用任务队列里回调函数的执行机制，就叫事件循环

```
/**
 * 目标：阅读并回答执行的顺序结果
 */
console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
console.log(3)
setTimeout(() => {
  console.log(4)
}, 2000)
console.log(5)
```

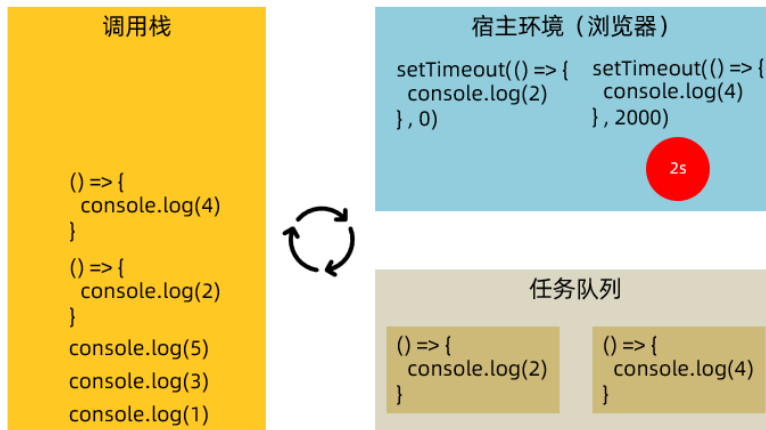
具体运行过程，请参考 PPT 动画和视频讲解

```

console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
console.log(3)
setTimeout(() => {
  console.log(4)
}, 2000)
console.log(5)

```

控制台输出: 1 3 5 2 4



小结

1. 什么是事件循环？
 - ▶ 答案
2. 为什么有事件循环？
 - ▶ 答案
3. JavaScript 内代码如何执行？
 - ▶ 答案

08.事件循环-练习

目标

了解事件循环的执行模型

讲解

1. 需求：请根据掌握的事件循环的模型概念，分析代码执行过程

```

/**
 * 目标：阅读并回答执行的顺序结果
 */
console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
function myFn() {
  console.log(3)
}
function ajaxFn() {
  const xhr = new XMLHttpRequest()

```

```

xhr.open('GET', 'http://hmajax.itheima.net/api/province')
xhr.addEventListener('loadend', () => {
  console.log(4)
})
xhr.send()
}
for (let i = 0; i < 1; i++) {
  console.log(5)
}
ajaxFn()
document.addEventListener('click', () => {
  console.log(6)
})
myFn()

```

```

console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
function myFn() {
  console.log(3)
}
function ajaxFn() {
  const xhr = new XMLHttpRequest()
  xhr.open('GET', 'http://hmajax.itheima.net/api/province')
  xhr.addEventListener('loadend', () => {
    console.log(4)
  })
  xhr.send()
}
ajaxFn()
for (let i = 0; i < 1; i++) {
  console.log(5)
}
document.addEventListener('click', () => {
  console.log(6)
})
myFn()

```

调用栈

宿主环境（浏览器）

任务队列

结果：1 5 3 2 4 点击一次document就会执行一次打印6

小结

暂无

09.宏任务与微任务

目标

掌握微任务和宏任务的概念和区分

讲解

- 1. ES6 之后引入了 Promise 对象，让 JS 引擎也可以发起异步任务
- 2. 异步任务划分为了
 - 宏任务：由浏览器环境执行的异步代码
 - 微任务：由 JS 引擎环境执行的异步代码
- 3. 宏任务和微任务具体划分：

任务（代码）	执行所在环境
JS脚本执行事件（script）	浏览器
setTimeout/setInterval	浏览器
AJAX请求完成事件	浏览器
用户交互事件等	浏览器

任务（代码）	执行所在环境
Promise对象.then()	JS 引擎

Promise 本身是同步的，而then和catch回调函数是异步的

4. 事件循环模型

具体运行效果，参考 PPT 动画或者视频

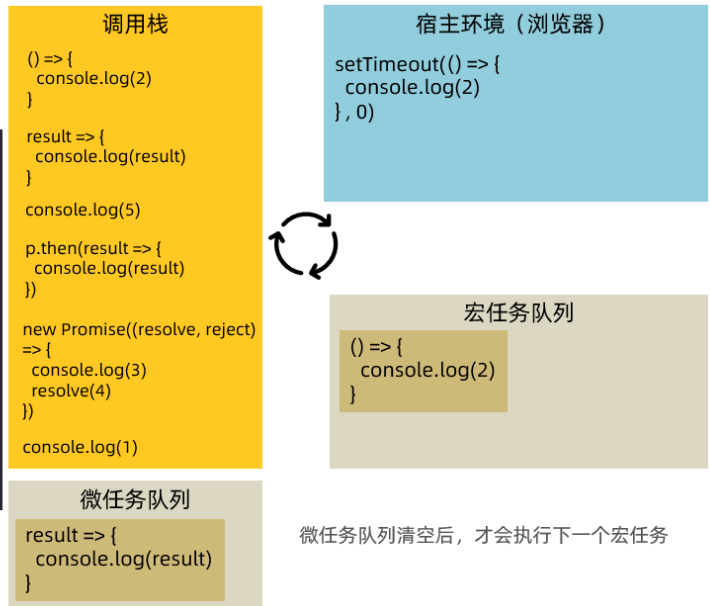
```
/**
 * 目标：阅读并回答打印的执行顺序
 */
console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
const p = new Promise((resolve, reject) => {
  resolve(3)
})
p.then(res => {
  console.log(res)
})
console.log(4)
```

宏任务与微任务 - 执行顺序

使用图解-分析代码执行顺序

```
<script>
console.log(1)
setTimeout(() => {
  console.log(2)
}, 0)
const p = new Promise((resolve, reject) => {
  console.log(3)
  resolve(4)
})
p.then(result => {
  console.log(result)
})
console.log(5)
</script>
```

控制台打印: 1 3 5 4 2



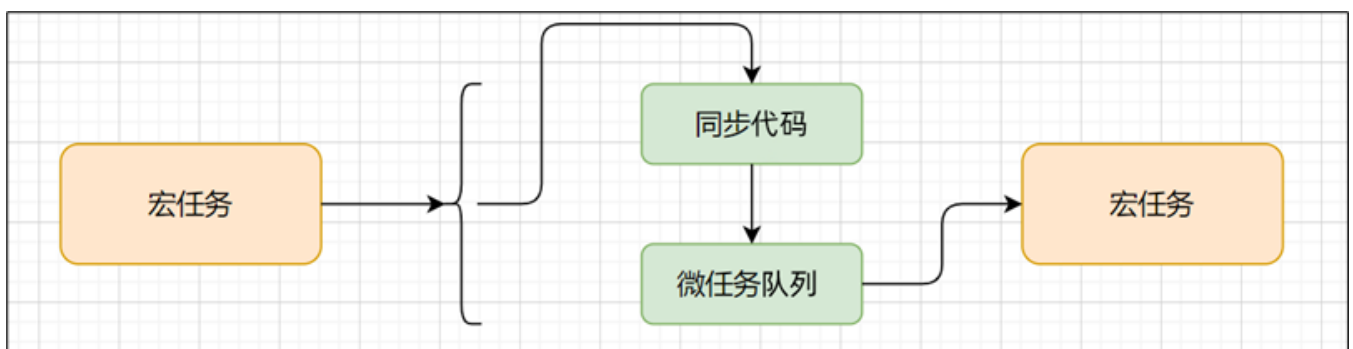
注意: 宏任务每次在执行同步代码时, 产生微任务队列, 清空微任务队列任务后, 微任务队列空间释放!

下一次宏任务执行时, 遇到微任务代码, 才会再次申请微任务队列空间放入回调函数消息排队

总结: 一个宏任务包含微任务队列, 他们之间是包含关系, 不是并列关系

小结

1. 什么是宏任务?
 - ▶ 答案
2. 什么是微任务?
 - ▶ 答案
3. JavaScript 内代码如何执行?
 - ▶ 答案



10.事件循环 - 经典面试题

目标

锻炼事件循环模型的使用

讲解

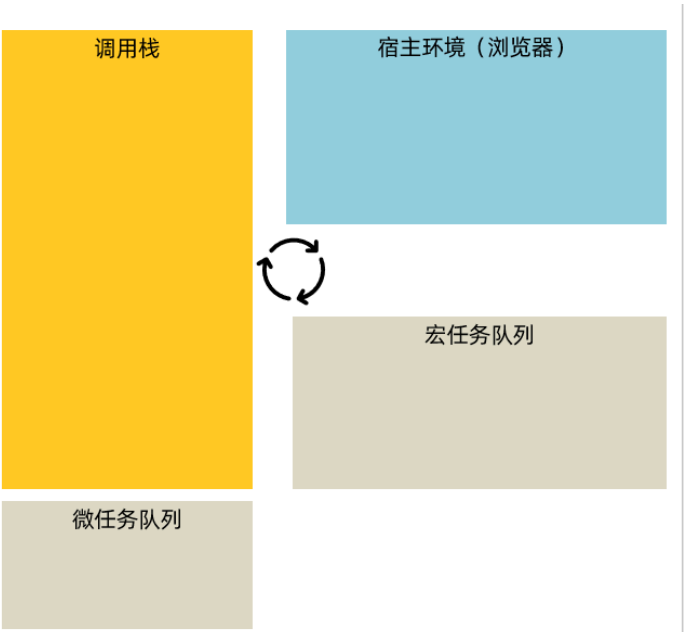
- 1. 需求：请切换到对应配套代码，查看具体代码，并回答打印顺序（锻炼事件循环概念的理解，阅读代码执行顺序）

```
// 目标：回答代码执行顺序
console.log(1)
setTimeout(() => {
  console.log(2)
  const p = new Promise(resolve => resolve(3))
  p.then(result => console.log(result))
}, 0)
const p = new Promise(resolve => {
  setTimeout(() => {
    console.log(4)
  }, 0)
  resolve(5)
})
p.then(result => console.log(result))
const p2 = new Promise(resolve => resolve(6))
p2.then(result => console.log(result))
console.log(7)
```

事件循环 - 经典面试题

请切换到对应配套代码，查看具体代码，并回答打印顺序

```
console.log(1)
setTimeout(() => {
  console.log(2)
  const p = new Promise(resolve => resolve(3))
  p.then(result => console.log(result))
}, 0)
const p = new Promise(resolve => {
  setTimeout(() => {
    console.log(4)
  }, 0)
  resolve(5)
})
p.then(result => console.log(result))
const p2 = new Promise(resolve => resolve(6))
p2.then(result => console.log(result))
console.log(7)
```



小结

暂无

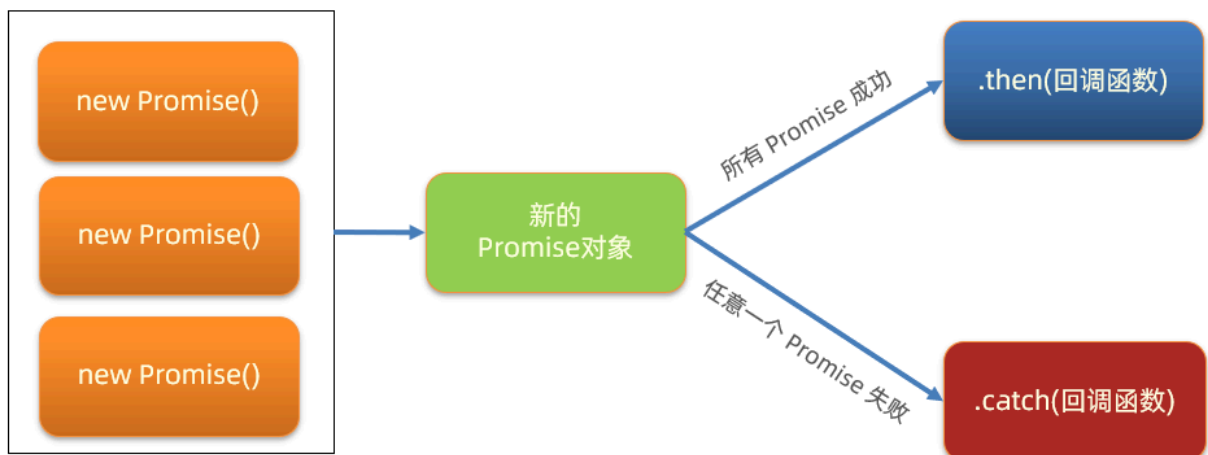
11.Promise.all 静态方法

目标

了解 Promise.all 作用和使用场景

讲解

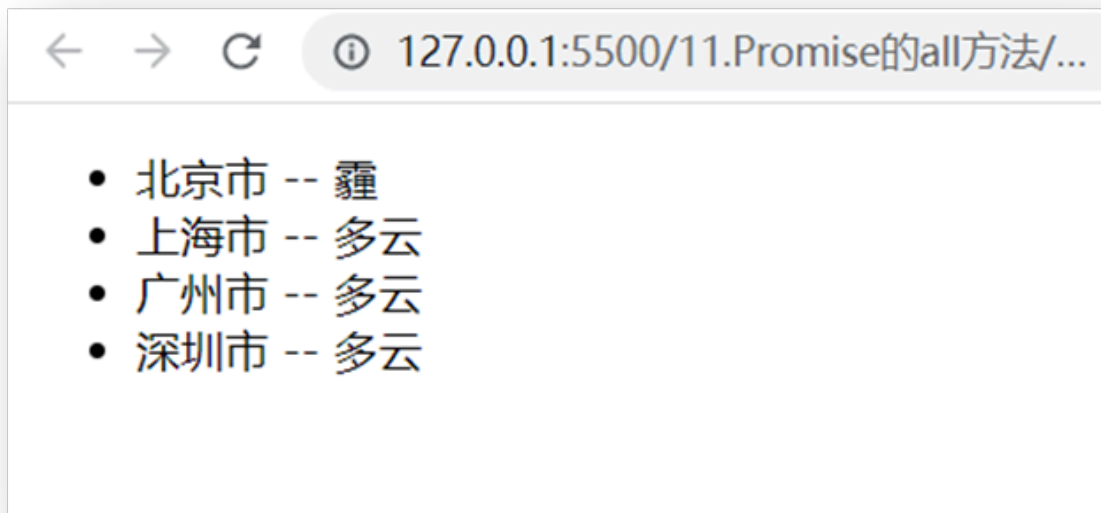
1. 概念：合并多个 Promise 对象，等待所有同时成功完成（或某一个失败），做后续逻辑



2. 语法：

```
const p = Promise.all([Promise对象, Promise对象, ...])
p.then(result => {
  // result 结果: [Promise对象成功结果, Promise对象成功结果, ...]
}).catch(error => {
  // 第一个失败的 Promise 对象, 抛出的异常对象
})
```

3. 需求：同时请求“北京”，“上海”，“广州”，“深圳”的天气并在网页尽可能同时显示



4. 核心代码如下：

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Promise的all方法</title>
</head>

<body>
  <ul class="my-ul"></ul>
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <script>
    /**
     * 目标：掌握Promise的all方法作用，和使用场景
     * 业务：当我需要同一时间显示多个请求的结果时，就要把多请求合并
     * 例如：默认显示"北京"，"上海"，"广州"，"深圳"的天气在首页查看
     * code:
     * 北京-110100
     * 上海-310100
     * 广州-440100
     * 深圳-440300
     */
    // 1. 请求城市天气，得到Promise对象
    const bjPromise = axios({ url: 'http://hmajax.itheima.net/api/weather', params: { city:
'110100' } })
    const shPromise = axios({ url: 'http://hmajax.itheima.net/api/weather', params: { city:
'310100' } })
    const gzPromise = axios({ url: 'http://hmajax.itheima.net/api/weather', params: { city:
'440100' } })
    const szPromise = axios({ url: 'http://hmajax.itheima.net/api/weather', params: { city:
'440300' } })
```

```
// 2. 使用Promise.all, 合并多个Promise对象
const p = Promise.all([bjPromise, shPromise, gzPromise, szPromise])
p.then(result => {
  // 注意: 结果数组顺序和合并时顺序是一致的
  console.log(result)
  const htmlStr = result.map(item => {
    return `<li>${item.data.data.area} --- ${item.data.data.weather}</li>`
  }).join('')
  document.querySelector('.my-ul').innerHTML = htmlStr
}).catch(error => {
  console.dir(error)
})
</script>
</body>

</html>
```

小结

1. Promise.all 什么时候使用?

► 答案

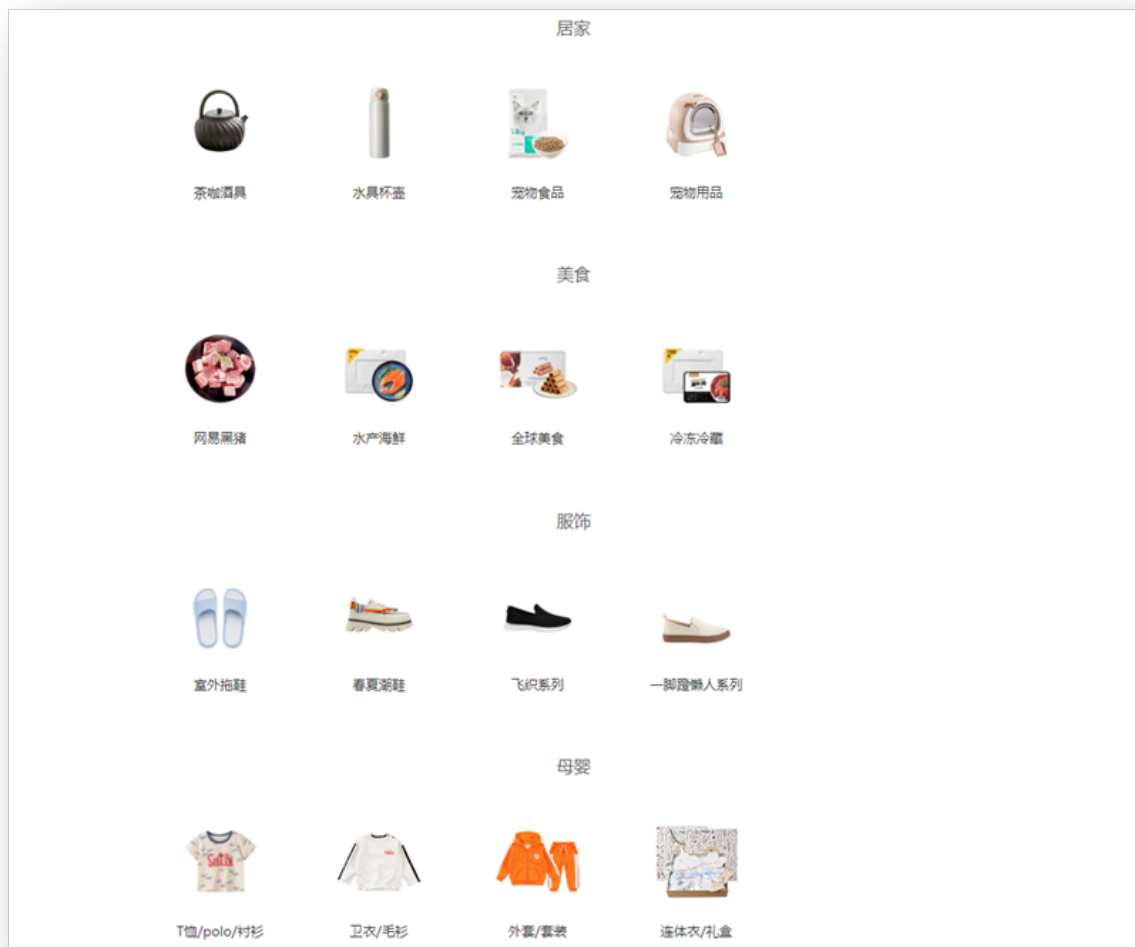
12.案例-商品分类

目标

完成商品分类效果

讲解

1. 需求: 尽可能同时展示所有商品分类到页面上



2. 步骤：

1. 获取所有的一级分类数据
2. 遍历id，创建获取二级分类请求
3. 合并所有二级分类Promise对象
4. 等待同时成功，开始渲染页面

3. 核心代码：

```
/**
 * 目标：把所有商品分类“同时”渲染到页面上
 * 1. 获取所有一级分类数据
 * 2. 遍历id，创建获取二级分类请求
 * 3. 合并所有二级分类Promise对象
 * 4. 等待同时成功后，渲染页面
 */
// 1. 获取所有一级分类数据
axios({
  url: 'http://hmajax.itheima.net/api/category/top'
}).then(result => {
  console.log(result)
  // 2. 遍历id，创建获取二级分类请求
  const secPromiseList = result.data.data.map(item => {
    return axios({
```

```

url: 'http://hmajax.itheima.net/api/category/sub',
params: {
  id: item.id // 一级分类id
}
})
})
console.log(secPromiseList) // 【二级分类请求Promise对象，二级分类请求Promise对象，...】
// 3. 合并所有二级分类Promise对象
const p = Promise.all(secPromiseList)
p.then(result => {
  console.log(result)
  // 4. 等待同时成功后，渲染页面
  const htmlStr = result.map(item => {
    const dataObj = item.data.data // 取出关键数据对象
    return `<div class="item">
<h3>${dataObj.name}</h3>
<ul>
  ${dataObj.children.map(item => {
    return `<li>
<a href="javascript:;">
  
  <p>${item.name}</p>
</a>
</li>`
  }).join('')}
</ul>
</div>`
  }).join('')
  console.log(htmlStr)
  document.querySelector('.sub-list').innerHTML = htmlStr
})
})

```

小结

暂无

13.案例-学习反馈-省市区切换

目标

完成省市区切换效果

讲解

1. 需求：完成省市区切换效果

学习反馈

黑马程序员
www.itheima.com



关注黑马，掌握更多前端资讯



热门校区 北京 上海 广州 深圳

地区选择

北京

北京市

东城区

您的称呼

播仔

宝贵建议

您对AJAX阶段课程宝贵的建议

确定提交

2. 步骤：

1. 设置省份数据到下拉菜单
2. 切换省份，设置城市数据到下拉菜单，并清空地区下拉菜单
3. 切换城市，设置地区数据到下拉菜单

3. 核心代码：

```
/**
 * 目标1：完成省市区下拉列表切换
 * 1.1 设置省份下拉菜单数据
 * 1.2 切换省份，设置城市下拉菜单数据，清空地区下拉菜单
 * 1.3 切换城市，设置地区下拉菜单数据
 */
// 1.1 设置省份下拉菜单数据
axios({
  url: 'http://hmajax.itheima.net/api/province'
}).then(result => {
  const optionStr = result.data.list.map(pname => `<option value="${pname}">${pname}</option>`).join('')
  document.querySelector('.province').innerHTML = `<option value="">省份</option>` + optionStr
})
```

```
// 1.2 切换省份，设置城市下拉菜单数据，清空地区下拉菜单
document.querySelector('.province').addEventListener('change', async e => {
  // 获取用户选择省份名字
  // console.log(e.target.value)
  const result = await axios({ url: 'http://hmajax.itheima.net/api/city', params: { pname:
e.target.value } })
  const optionStr = result.data.list.map(cname => `<option value="${cname}">${cname}
</option>`).join('')
  // 把默认城市选项+下属城市数据插入select中
  document.querySelector('.city').innerHTML = `<option value="">城市</option>` + optionStr

  // 清空地区数据
  document.querySelector('.area').innerHTML = `<option value="">地区</option>`
})

// 1.3 切换城市，设置地区下拉菜单数据
document.querySelector('.city').addEventListener('change', async e => {
  console.log(e.target.value)
  const result = await axios({url: 'http://hmajax.itheima.net/api/area', params: {
    pname: document.querySelector('.province').value,
    cname: e.target.value
  }})
  console.log(result)
  const optionStr = result.data.list.map(aname => `<option value="${aname}">${aname}
</option>`).join('')
  console.log(optionStr)
  document.querySelector('.area').innerHTML = `<option value="">地区</option>` + optionStr
})
```

小结

暂无

14.案例-学习反馈-数据提交

目标

完成学习反馈数据提交

讲解

1. 需求：收集学习反馈数据，提交保存

学习反馈

黑马程序员
www.itheima.com



关注黑马，掌握更多前端资讯



热门校区 北京 上海 广州 深圳

地区选择 北京 北京市 东城区

您的称呼 播仔

宝贵建议
您对AJAX阶段课程宝贵的建议

确定提交

2. 步骤：

1. 监听提交按钮的点击事件
2. 依靠插件收集表单数据
3. 基于 axios 提交保存，显示结果

3. 核心代码如下：

```
/**
 * 目标2：收集数据提交保存
 * 2.1 监听提交的点击事件
 * 2.2 依靠插件收集表单数据
 * 2.3 基于axios提交保存，显示结果
 */
// 2.1 监听提交的点击事件
document.querySelector('.submit').addEventListener('click', async () => {
  // 2.2 依靠插件收集表单数据
  const form = document.querySelector('.info-form')
  const data = serialize(form, { hash: true, empty: true })
  console.log(data)
  // 2.3 基于axios提交保存，显示结果
  try {
    const result = await axios({
      url: 'http://hmajax.itheima.net/api/feedback',
      method: 'POST',
      data
    })
  }
```

```
    console.log(result)
    alert(result.data.message)
  } catch (error) {
    console.dir(error)
    alert(error.response.data.message)
  }
})
```

小结

暂无

今日重点(必须会)

1. 掌握 async 和 await 的使用
2. 理解 EventLoop 和宏任务微任务执行顺序
3. 了解 Promise.all 的作用和使用场景
4. 完成案例-学习反馈

今日作业(必完成)

参考作业文件夹里md文档的要求

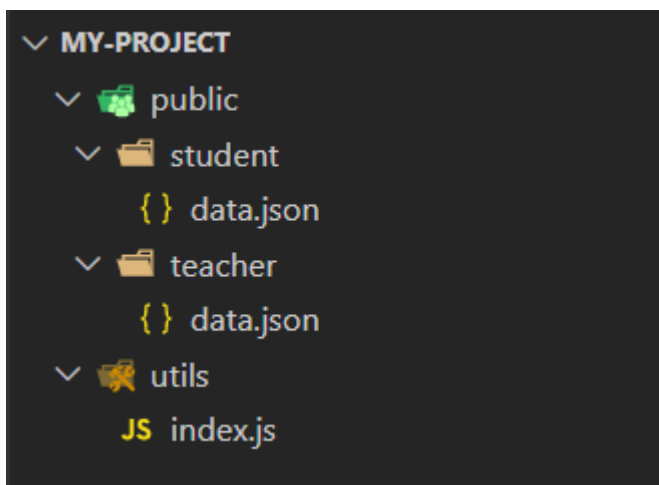
参考文献

1. [async和await的mdn讲解](#)

01_Node.js入门

知识点自测

1. 从 index.js 出发，访问到 student/data.json 的相对路径如何写？



A: `../public/teacher/data.json`

B: `../public/student/data.json`

C: `../student/data.json`

► 答案

2. 如下代码运行结果是多少？

```
const str = `我们都是好人，好人就是我`
const result = str.replace(/好/g, '')
```

A: 我们都是人，好人就是我

B: 我们都是人，人就是我

► 答案

3. 以下哪个正则表达式，能匹配字符串中的小括号？

A: `/(/`

B: `/\(/`

► 答案

4. 如下代码返回的结果是什么？

```
const str = '/api/hello?a=10&b=20'  
const result = str.startsWith('/api/hello')
```

A: true

B: false

► 答案

提前安装软件

在配套的资料文件夹中，先安装好 Node.js 环境，上课可以节省时间

具体安装过程可以参考 PPT 和 配套视频！

01.什么是 Node.js

目标

什么是 Node.js，有什么用，为何能独立执行 JS 代码，演示安装和执行 JS 文件内代码

讲解

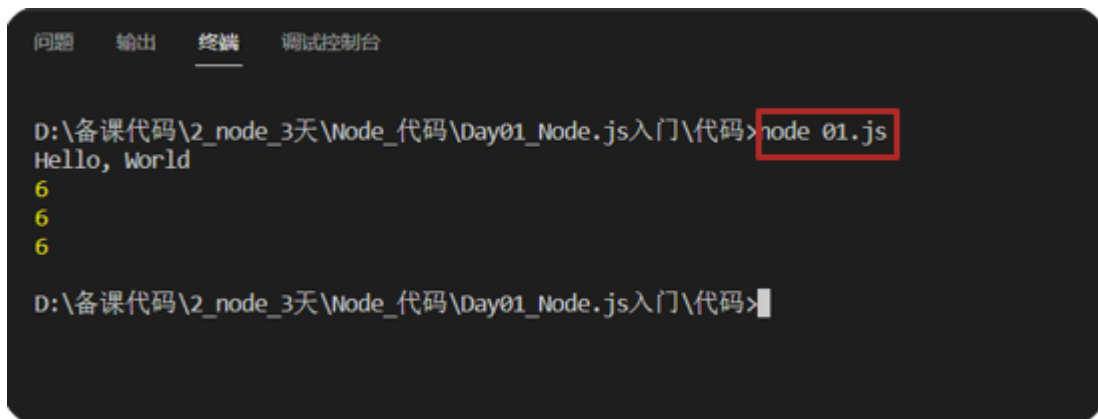
1. Node.js 是一个独立的 JavaScript 运行环境，能独立执行 JS 代码，因为这个特点，它可以用来编写服务器后端的应用程序
2. Node.js 作用除了编写后端应用程序，也可以对前端代码进行压缩，转译，整合等等，提高前端开发和运行效率
3. Node.js 基于 Chrome V8 引擎封装，独立执行 JS 代码，但是语法和浏览器环境的 V8 有所不同，没有 document 和 window 但是都支持 ECMAScript 标准的代码语法
4. 想要得到 Node.js 需要把这个软件安装到电脑，在素材里有安装程序（window 和 mac 环境的）参考 PPT 默认下一步安装即可
5. Node.js 没有图形化界面，需要使用 cmd 终端命令行（利用一些命令来操控电脑执行某些程序软件）输入，node -v 检查是否安装成功

```
node -v
```

6. 需求：新建 index.js 文件，编写打印代码和 for 循环打印 3 个 6

```
/**
 * 目标：编写 js 代码，用 node 命令执行
 * 终端作用：敲击命令，调用对应程序执行
 * 终端打开：目标文件->右键->在集成终端中打开
 * 命令：node xxx.js （注意路径）
 */
console.log('Hello, World')
for (let i = 0; i < 3; i++) {
  console.log(6)
}
```

7. Node.js 执行目标 JS 文件，需要使用 node xxx.js 命令来执行（我们可以借助 VSCode 集成终端使用，好处：可以快速切换到目标 JS 文件所在终端目录，利用相对路径找到要执行的目标 JS 文件）



The image shows a screenshot of the VS Code integrated terminal. At the top, there are tabs for '问题' (Issues), '输出' (Output), '终端' (Terminal), and '调试控制台' (Debug Console). The '终端' tab is active. The terminal shows the command prompt 'D:\备课代码\2_node_3天\Node_代码\Day01_Node.js入门\代码>' followed by the command 'node 01.js' which is highlighted with a red box. The output of the command is 'Hello, World' followed by three lines of the number '6' on separate lines. The prompt is now 'D:\备课代码\2_node_3天\Node_代码\Day01_Node.js入门\代码>'.

小结

1. Node.js 有什么用？
 - ▶ 答案
2. Node.js 为何能执行 JS 代码？
 - ▶ 答案
3. Node.js 与浏览器环境的 JS 最大区别？
 - ▶ 答案
4. Node.js 如何执行代码？
 - ▶ 答案

02.fs模块-读写文件

目标

了解模块概念，使用 fs 模块封装的方法读写文件内容

讲解

1. 模块：类似插件，封装了方法和属性供我们使用
2. fs 模块：封装了与本机文件系统进行交互的，方法和属性
3. fs 模块使用语法如下：

- 加载 fs 模块，得到 fs 对象

```
const fs = require('fs')
```

- 写入文件内容语法：

```
fs.writeFile('文件路径', '写入内容', err => {  
  // 写入后的回调函数  
})
```

- 读取文件内容的语法：

```
fs.readFile('文件路径', (err, data) => {  
  // 读取后的回调函数  
  // data 是文件内容的 Buffer 数据流  
})
```

4. 需求：向 test.txt 文件写入内容并读取打印

```
/**  
 * 目标：使用 fs 模块，读写文件内容  
 * 语法：  
 * 1. 引入 fs 模块  
 * 2. 调用 writeFile 写入内容  
 * 3. 调用 readFile 读取内容  
 */  
// 1. 引入 fs 模块  
const fs = require('fs')  
// 2. 调用 writeFile 写入内容  
// 注意：建议写入字符串内容，会覆盖目标文件所有内容  
fs.writeFile('./text.txt', '欢迎使用 fs 模块读写文件内容', err => {  
  if (err) console.log(err)  
  else console.log('写入成功')  
})  
// 3. 调用 readFile 读取内容  
fs.readFile('./text.txt', (err, data) => {  
  if (err) console.log(err)
```

```
else console.log(data.toString()) // 把 Buffer 数据流转成字符串类型
})
```

小结

1. 什么是模块?
▶ 答案
2. fs 模块的作用?
▶ 答案

03.path模块-路径处理

目标

使用 path 模块来得到绝对路径

讲解

1. 为什么在 Node.js 待执行的 JS 代码中要用绝对路径：

Node.js 执行 JS 代码时，代码中的路径都是以终端所在文件夹出发查找相对路径，而不是以我们认为的从代码本身出发，会遇到问题，所以在 Node.js 要执行的代码中，访问其他文件，建议使用绝对路径

2. 新建 03 文件夹编写待执行的 JS 代码，访问外层相对路径下的文件，**然后在最外层终端路径来执行目标文件，造成问题**



```
03 > JS index.js > ...
1  /**
2  * 目标: 读取 test.txt 文件内容
3  */
4  const fs = require('fs')
5  fs.readFile('../text.txt', (err, data) => {
6    if (err) console.log(err)
7    else console.log(data.toString()) // 把 Buffer 数据流转成字符串类型
8  })
```

```
问题 输出 终端 调试控制台

Microsoft Windows [版本 10.0.19044.2604]
(c) Microsoft Corporation. 保留所有权利。

D:\各课代码\2_node_3天\Node_代码\Day01_Node.js入门\代码>node 03/index.js
[Error: ENOENT: no such file or directory, open 'D:\各课代码\2_node_3天\Node_代码\Day01_Node.js入门\text.txt'] {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'D:\\各课代码\\2_node_3天\\Node_代码\\Day01_Node.js入门\\text.txt'
}

D:\各课代码\2_node_3天\Node_代码\Day01_Node.js入门\代码>
```

3. 问题原因：就是从代码文件夹出发，使用 `../text.txt` 解析路径，找不到目标文件，报错了！

4. 解决方案：使用模块内置变量 `__dirname` 配合 `path.join()` 来得到绝对路径使用

```
const fs = require('fs')
console.log(__dirname) // D:\各课代码\2_node_3天\Node_代码\Day01_Node.js入门\代码\03

// 1. 加载 path 模块
const path = require('path')
// 2. 使用 path.join() 来拼接路径
const pathStr = path.join(__dirname, '..', 'text.txt')
console.log(pathStr)

fs.readFile(pathStr, (err, data) => {
  if (err) console.log(err)
  else console.log(data.toString())
})
```

5. 再次执行查看问题就被修复了！以后在 Node.js 要执行的 JS 代码中访问其他文件的路径，都建议使用绝对路径

小结

1. `path.join()` 方法有什么用？

► 答案

2. `__dirname` 模块内置变量的值是多少？

► 答案

04.案例-压缩前端html

目标

压缩前端代码，体验前端工程化的例子

讲解

1. 前端工程化：前端代码压缩，整合，转译，测试，自动部署等等工具的集成统称，为了提高前端开发项目的效率
2. 需求：把准备好的 html 文件里的回车符（\r）和换行符（\n）去掉进行压缩，写入到新 html 中
3. 步骤：
 1. 读取源 html 文件内容
 2. 正则替换字符串
 3. 写入到新的 html 文件中，并运行查看是否能正常打开网页
4. 代码如下：

```
/**
 * 目标一：压缩 html 里代码
 * 需求：把 public/index.html 里的，回车/换行符去掉，写入到 dist/index.html 中
 * 1.1 读取 public/index.html 内容
 * 1.2 使用正则替换内容字符串里的，回车符\r 换行符\n
 * 1.3 确认后，写入到 dist/index.html 内
 */
const fs = require('fs')
const path = require('path')
// 1.1 读取 public/index.html 内容
fs.readFile(path.join(__dirname, 'public', 'index.html'), (err, data) => {
  const htmlStr = data.toString()
  // 1.2 使用正则替换内容字符串里的，回车符\r 换行符\n
  const resultStr = htmlStr.replace(/[\r\n]/g, '')
  // 1.3 确认后，写入到 dist/index.html 内
  fs.writeFile(path.join(__dirname, 'dist', 'index.html'), resultStr, err => {
    if (err) console.log(err)
    else console.log('压缩成功')
  })
})
```

小结

1. 前端工程化的理解？

▶ 答案

05.认识URL中的端口号

目标

认识 URL 中端口号的作用，以及 Web 服务的作用

讲解

1. URL 是统一资源定位符，简称网址，用于访问网络上的资源
2. 端口号的作用：标记服务器里对应的服务程序，值为（0-65535 之间的任意整数）
3. 注意：http 协议，默认访问的是 80 端口
4. Web服务：一个程序，用于提供网上信息浏览功能
5. 注意：0-1023 和一些特定的端口号被占用，我们自己编写服务程序请避开使用

小结

1. 端口号的作用？
▶ 答案
2. 什么是 Web 服务？
▶ 答案

06.http模块-创建Web服务

目标

基于 Node.js 环境，使用内置 http 模块，创建 Web 服务程序

讲解

1. 需求：引入 http 模块，使用相关语法，创建 Web 服务程序，响应返回给请求方一句提示 'hello, world'
2. 步骤：
 1. 引入 http 模块，创建 Web 服务对象
 2. 监听 request 请求事件，对本次请求，做一些响应处理
 3. 启动 Web 服务监听对应端口号
 4. 运行本服务在终端进程中，用浏览器发起请求
3. 注意：本机的域名叫做 localhost
4. 代码如下：

```
/**
 * 目标：基于 http 模块创建 Web 服务程序
 * 1.1 加载 http 模块，创建 Web 服务对象
 * 1.2 监听 request 请求事件，设置响应头和响应体
 * 1.3 配置端口号并启动 Web 服务
 * 1.4 浏览器请求（http://localhost:3000）测试
 */
// 1.1 加载 http 模块，创建 Web 服务对象
const http = require('http')
const server = http.createServer()
// 1.2 监听 request 请求事件，设置响应头和响应体
server.on('request', (req, res) => {
  // 设置响应头-内容类型-普通文本以及中文编码格式
  res.setHeader('Content-Type', 'text/plain;charset=utf-8')
  // 设置响应体内容，结束本次请求与响应
  res.end('欢迎使用 Node.js 和 http 模块创建的 Web 服务')
})
// 1.3 配置端口号并启动 Web 服务
server.listen(3000, () => {
  console.log('Web 服务启动成功了')
})
```

小结

1. 如何访问本机里运行的 Web 服务？
► 答案

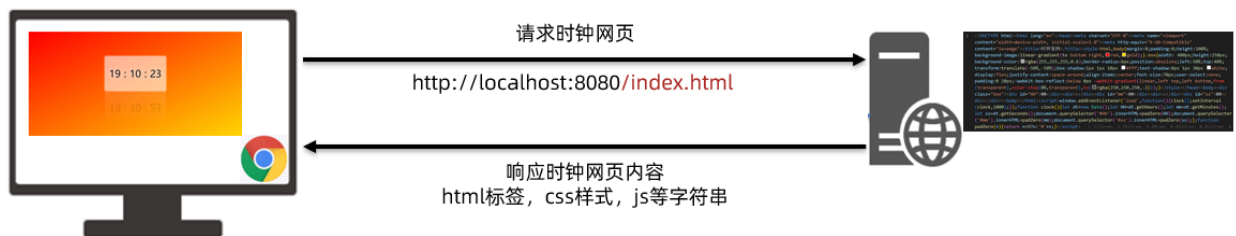
07.案例-浏览时钟

目标

体验 Web 服务除了接口数据以外，还能返回网页资源等

讲解

1. 需求：基于 Web 服务，开发提供网页资源的功能，了解下后端的代码工作过程



2. 步骤：

1. 基于 http 模块，创建 Web 服务
2. 使用 req.url 获取请求资源路径为 /index.html 的时候，读取 index.html 文件内容字符串返回给请求方
3. 其他路径，暂时返回不存在的提示
4. 运行 Web 服务，用浏览器发起请求

3. 代码如下：

```
/**
 * 目标：编写 web 服务，监听请求的是 /index.html 路径的时候，返回 dist/index.html 时钟案例页面内容
 * 步骤：
 * 1. 基于 http 模块，创建 Web 服务
 * 2. 使用 req.url 获取请求资源路径，并读取 index.html 里字符串内容返回给请求方
 * 3. 其他路径，暂时返回不存在提示
 * 4. 运行 Web 服务，用浏览器发起请求
 */
const fs = require('fs')
const path = require('path')
// 1. 基于 http 模块，创建 Web 服务
const http = require('http')
const server = http.createServer()
server.on('request', (req, res) => {
  // 2. 使用 req.url 获取请求资源路径，并读取 index.html 里字符串内容返回给请求方
  if (req.url === '/index.html') {
    fs.readFile(path.join(__dirname, 'dist/index.html'), (err, data) => {
      res.setHeader('Content-Type', 'text/html;charset=utf-8')
      res.end(data.toString())
    })
  } else {
    // 3. 其他路径，暂时返回不存在提示
    res.setHeader('Content-Type', 'text/html;charset=utf-8')
    res.end('你要访问的资源路径不存在')
```

```
    }  
  })  
  server.listen(8080, () => {  
    console.log('Web 服务启动了')  
  })  
})
```

小结

1. Web 服务程序都有什么功能？

► 答案

02_Node.js模块化

知识点自测

1. 以下代码运行的结果是多少？

```
const arr = [10, 20, 30]  
const result = arr.map(val => val + 1).reduce((sum, val) => sum += val, 0)  
console.log(result)
```

A: 60

B: 63

► 答案

2. 以下代码运行的结果是多少？

```
const obj = { username: 'itheima', password: '777777' }  
const { uname } = obj  
console.log(uname)
```

A: itheima

B: undefined

► 答案

01.模块化简介

目标

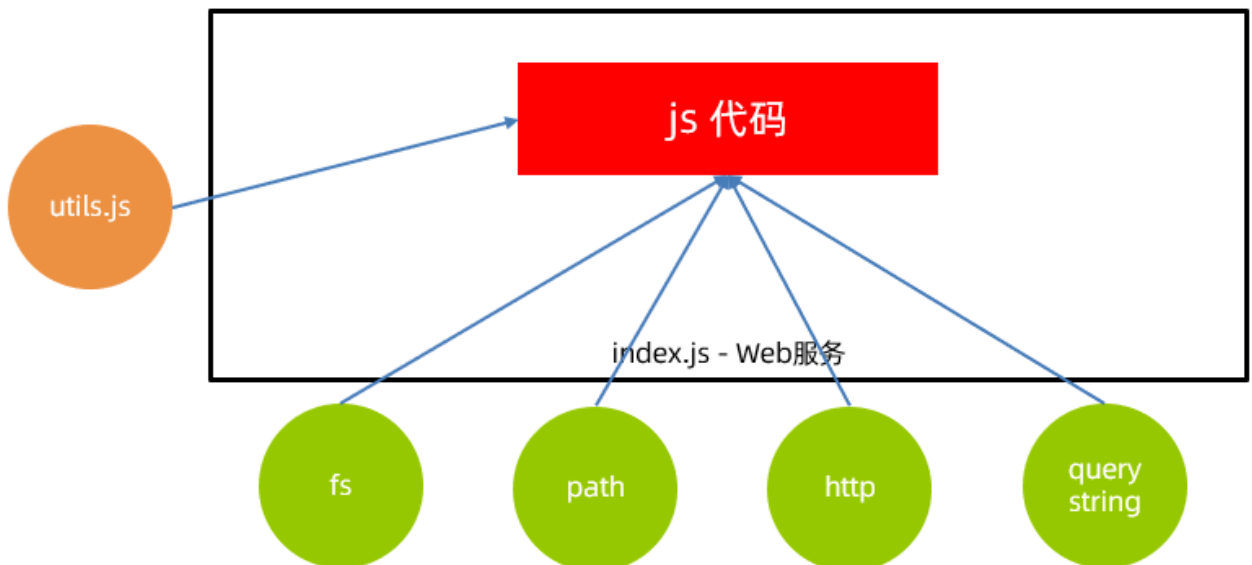
了解模块化概念和好处，以及 CommonJS 标准语法导出和导入

讲解

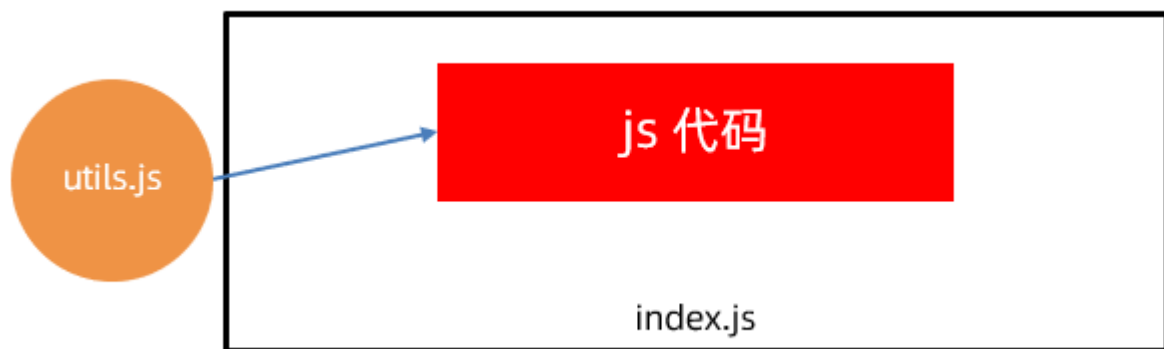
1. 在 Node.js 中每个文件都被当做是一个独立的模块，模块内定义的变量和函数都是独立作用域的，因为 Node.js 在执行模块代码时，将使用如下所示的函数封装器对其进行封装

```
(function(exports, require, module, __filename, __dirname) {  
  // 模块代码实际存在于此处  
});
```

2. 而且项目是由多个模块组成的，每个模块之间都是独立的，而且提高模块代码复用性，按需加载，独立作用域



3. 但是因为模块内的属性和函数都是私有的，如果对外使用，需要使用标准语法导出和导入才可以，而这个标准叫 CommonJS 标准，接下来我们在一个需求中，体验下模块化导出和导入语法的使用
4. 需求：定义 utils.js 模块，封装基地址和求数组总和的函数，导入到 index.js 使用查看效果



5. 导出语法:

```
module.exports = {  
  对外属性名: 模块内私有变量  
}
```

6. 导入语法:

```
const 变量名 = require('模块名或路径')  
// Node.js 环境内置模块直接写模块名 (例如: fs, path, http)  
// 自定义模块: 写模块文件路径 (例如: ./utils.js)
```

变量名的值接收的就是目标模块导出的对象

7. 代码实现

◦ utils.js: 导出

```
/**  
 * 目标: 基于 CommonJS 标准语法, 封装属性和方法并导出  
 */  
const baseUrl = 'http://hmajax.itheima.net'  
const getArraySum = arr => arr.reduce((sum, item) => sum += item, 0)  
  
// 导出  
module.exports = {  
  url: baseUrl,  
  arraySum: getArraySum  
}
```

◦ index.js: 导入使用

```
/**  
 * 目标: 基于 CommonJS 标准语法, 导入工具属性和方法使用  
 */  
// 导入  
const obj = require('./utils.js')  
console.log(obj)  
const result = obj.arraySum([5, 1, 2, 3])  
console.log(result)
```

小结

1. Node.js 中什么是模块化?
 - ▶ 答案
2. 模块之间如何联系呢?
 - ▶ 答案
3. CommonJS 标准规定如何导出和导入模块?
 - ▶ 答案
4. 模块名/路径如何选择?
 - ▶ 答案

02.ECMAScript标准-默认导出和导入

目标

掌握 ECMAScript 标准语法中，默认导出和导入的使用

讲解

1. CommonJS 规范是 Node.js 环境中默认的，后来官方推出 ECMAScript 标准语法，我们接下来在一个需求中，体验下这个标准中默认导出和导入的语法要如何使用
2. 需求：封装并导出地址和求数组元素和的函数，导入到 index.js 使用查看效果
3. 导出语法：

```
export default {  
  对外属性名：模块内私有变量  
}
```

4. 导入语法：

```
import 变量名 from '模块名或路径'
```

变量名的值接收的就是目标模块导出的对象

5. 注意：Node.js 默认只支持 CommonJS 标准语法，如果要在当前项目环境下使用 ECMAScript 标准语法，请新建 package.json 文件设置 type: 'module' 来进行设置

```
{ "type": "module" }
```

6. 代码实现：

- utils.js：导出

```
/**
 * 目标：基于 ECMAScript 标准语法，封装属性和方法并"默认"导出
 */
const baseUrl = 'http://hmajax.itheima.net'
const getArraySum = arr => arr.reduce((sum, item) => sum += item, 0)

// 默认导出
export default {
  url: baseUrl,
  arraySum: getArraySum
}
```

- index.js：导入

```
/**
 * 目标：基于 ECMAScript 标准语法，"默认"导入，工具属性和方法使用
 */
// 默认导入
import obj from './utils.js'
console.log(obj)
const result = obj.arraySum([10, 20, 30])
console.log(result)
```

小结

1. ECMAScript 标准规定如何默认导出和导入模块？

► 答案

2. 如何让 Node.js 切换模块标准为 ECMAScript？

► 答案

03.ECMAScript标准-命名导出和导入

目标

掌握 ECMAScript 标准语法中，命名导出和导入的使用

讲解

1. ECMAScript 标准的语法有很多，常用的就是默认和命名导出和导入，这节课我们来学习下命名导出和导入的使用
2. 需求：封装并导出基地址和数组求和函数，导入到 index.js 使用查看效果
3. 命名导出语法：

```
export 修饰定义语句
```

4. 命名导入语法：

```
import { 同名变量 } from '模块名或路径'
```

注意：同名变量指的是模块内导出的变量名

5. 代码示例：

- utils.js 导出

```
/**
 * 目标：基于 ECMAScript 标准语法，封装属性和方法并"命名"导出
 */
export const baseUrl = 'http://hmajax.itheima.net'
export const getArraySum = arr => arr.reduce((sum, item) => sum += item, 0)
```

- index.js 导入

```
/**
 * 目标：基于 ECMAScript 标准语法，"命名"导入，工具属性和方法使用
 */
// 命名导入
import {baseUrl, getArraySum} from './utils.js'
console.log(obj)
console.log(baseUrl)
console.log(getArraySum)
const result = getArraySum([10, 21, 33])
console.log(result)
```

6. 与默认导出如何选择：

- 按需加载，使用命名导出和导入
- 全部加载，使用默认导出和导入

小结

1. Node.js 支持哪 2 种模块化标准？
 - 答案
2. ECMAScript 标准，命名导出和导入的语法？
 - 答案

3. ECMAScript 标准，默认导出和导入的语法？

► 答案

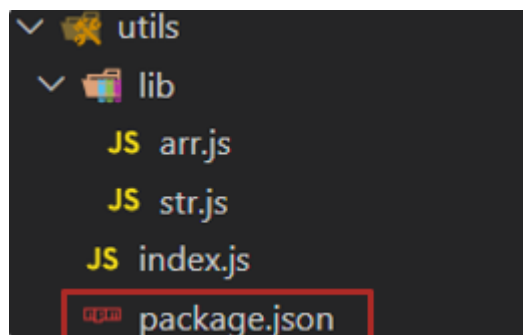
04.包的概念

目标

了解 Node.js 环境中包的概念

讲解

1. 包：将模块，代码，其他资料整合成一个文件夹，这个文件夹就叫包
2. 包分类：
 - 项目包：主要用于编写项目和业务逻辑
 - 软件包：封装工具和方法进行使用
3. 包要求：根目录中，必须有 package.json 文件（记录包的清单信息）



```
package.json > ...
1  {
2    "name": "cz_utils",    软件包名称
3    "version": "1.0.0",    软件包当前版本
4    "description": "一个数组和字符串常用工具方法的包",  软件包简短描述
5    "main": "index.js",    软件包入口点
6    "author": "itheima",    软件包作者
7    "license": "MIT"        软件包许可证（商用后可以用作者名字宣传）
8  }
```

4. 包使用：在引入一个包文件夹到代码中，默认引入的是包文件夹下的 index.js 模块文件里导出的对象，如果没有 index.js 文件，则会引入 package.json 里 main 属性指定的文件模块导出的对象
5. 需求：封装数组求和函数的模块，封装判断用户名和密码长度函数的模块，形成一个软件包，并导入到 index.js 中使用查看效果

6. 代码示例：

- utils/lib 相关代码在素材里准备好了，只需要自己在 utils/index.js 统一出口进行导出

```
/**
 * 本文件是，utils 工具包的唯一出口
 * 作用：把所有工具模块方法集中起来，统一向外暴露
 */
const { getArraySum } = require('./lib/arr.js')
const { checkUser, checkPwd } = require('./lib/str.js')

// 统一导出所有函数
module.exports = {
  getArraySum,
  checkUser,
  checkPwd
}
```

- index.js 导入软件包文件夹使用（注意：这次导入的是包文件夹，不是模块文件）

```
/**
 * 目标：导入 utils 软件包，使用里面封装的工具函数
 */
const obj = require('./utils')
console.log(obj)
const result = obj.getArraySum([10, 20, 30])
console.log(result)
```

小结

- 什么是包？
 - ▶ 答案
- 包分为哪 2 类呢？
 - ▶ 答案
- package.json 文件的作用？
 - ▶ 答案
- 导入一个包文件夹的时候，导入的是哪个文件？
 - ▶ 答案

05.npm软件包管理器

目标

掌握使用 npm 管理软件包

讲解

- 1. npm 简介 [链接](#)： 软件包管理器，用于下载和管理 Node.js 环境中的软件包

npm 简介

npm 是 Node.js 标准的软件包管理器。

在 2017 年 1 月时，npm 仓库中就有超过 350000 个软件包，这使其成为世界上最大的单一语言代码仓库，并且可以确定几乎有可用于一切的软件包。

它起初是作为下载和管理 Node.js 包依赖的方式，但其现在也已成为前端 JavaScript 中使用的工具。

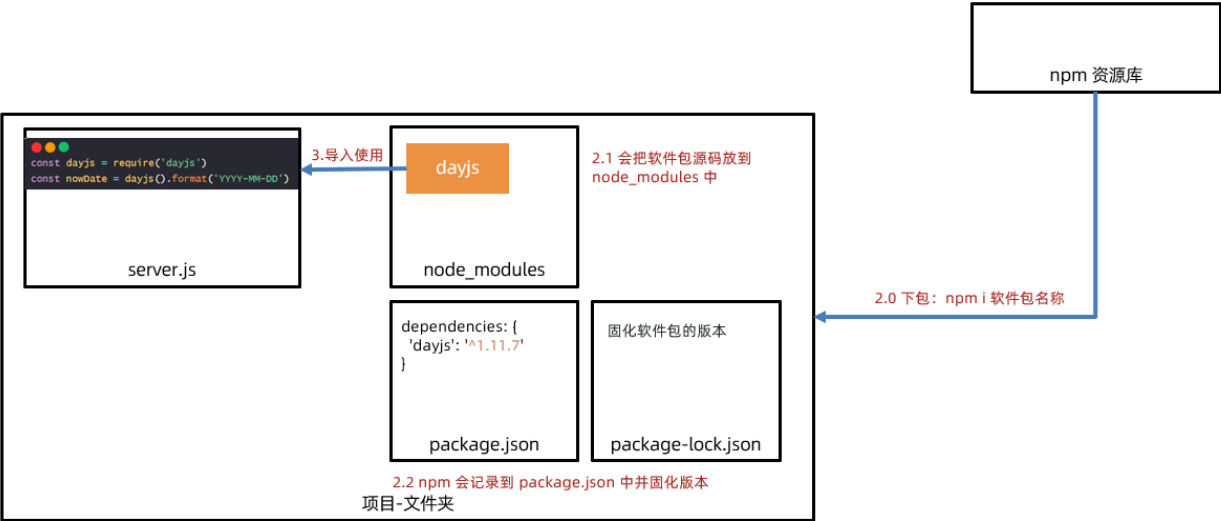
- 2. npm 使用步骤：

- 1. 初始化清单文件： npm init -y （得到 package.json 文件，有则跳过此命令）

注意 -y 就是所有选项用默认值，所在文件夹不要有中文/特殊符号，建议英文和数字组成，因为 npm 包名限制建议用英文和数字或者下划线中划线

- 2. 下载软件包： npm i 软件包名称
 - 3. 使用软件包

- 3. 需求：使用 npm 下载 dayjs 软件包到本地项目文件夹中，引入到 index.js 中格式化日期打印，运行观察效果
- 4. 具体使用流程图：



小结

1. npm 软件包管理器作用?
▶ 答案
2. 初始化项目清单文件 package.json 命令?
▶ 答案
3. 下载软件包的命令?
▶ 答案
4. 下载的包会存放在哪里?
▶ 答案

06.npm安装所有依赖

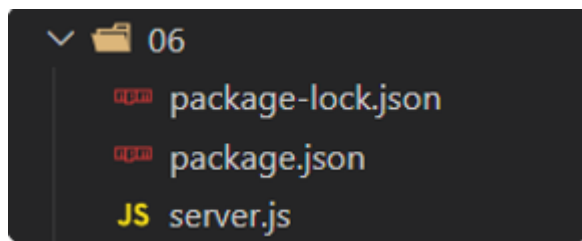
目标

掌握 npm 安装所有依赖功能

讲解

1. 我们拿到了一个别人编写的项目，但是没有 node_modules，项目能否正确运行？

不能，因为缺少了项目需要的依赖软件包，比如要使用 dayjs 和 lodash 但是你项目里没有这个对应的源码，项目会报错的

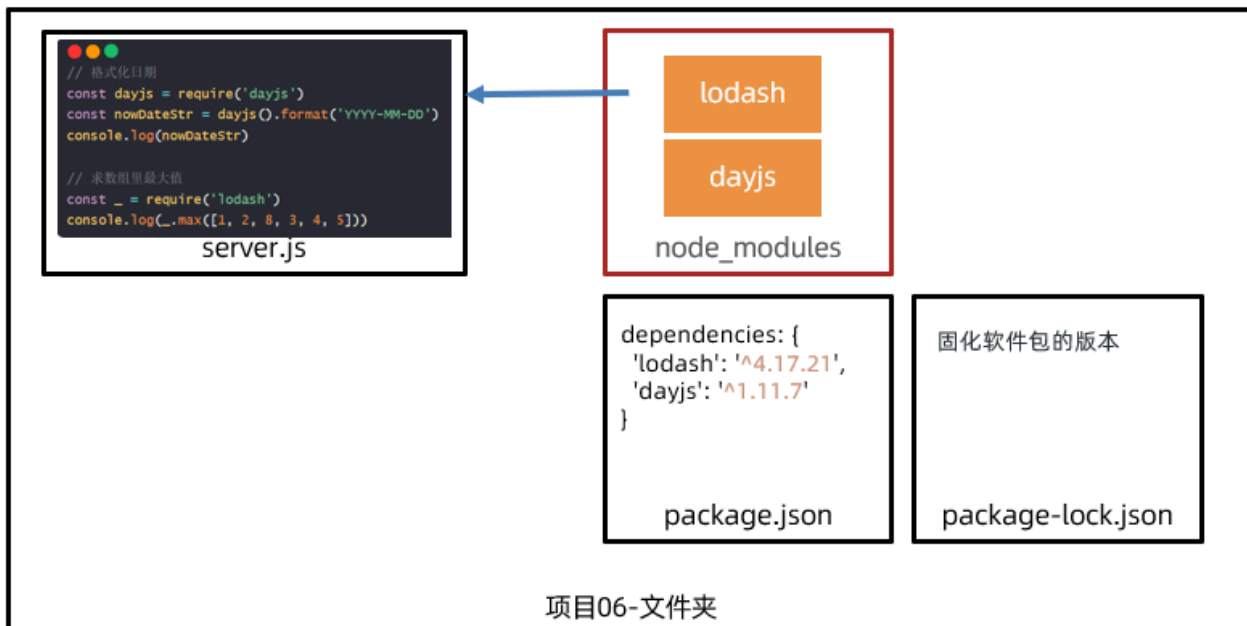


2. 为何没有给我 node_modules？

因为每个人在自己的本机使用 npm 下载，要比磁盘间传递要快（npm 有缓存在本机）

3. 如何得到需要的所有依赖软件包呢？

直接在项目目录下，运行终端命令：npm i 即可安装 package.json 里记录的所有包和对应版本到本项目中的 node_modules



4. 需求：请在准备好的素材项目中，安装项目所有需要的依赖软件包，并运行 index.js 代码看是否正常！

小结

1. 当前项目中只有 package.json 没有 node_modules 怎么办？
 - ▶ 答案
2. 为什么 node_modules 不进行传递？
 - ▶ 答案

07.npm全局软件包-nodemon

目标

掌握本地软件包和全局软件包区别，体验 nodemon 的使用

讲解

1. 软件包区别：
 - 本地软件包：当前项目内使用，**封装属性和方法**，存在于 node_modules
 - 全局软件包：本机所有项目使用，**封装命令和工具**，存在于系统设置的位置
2. nodemon 作用：替代 node 命令，检测代码更改，自动重启程序
3. 使用：
 1. 安装：npm i nodemon -g （-g 代表安装到全局环境中）

2. 运行：nodemon 待执行的目标 js 文件
4. 需求：使用 nodemon 命令来启动素材里准备好的项目，然后修改代码保存后，观察终端重启应用程序

小结

1. 本地软件包和全局软件包区别?
▶ 答案
2. nodemon 作用?
▶ 答案
3. nodemon 怎么用?
▶ 答案

08.Node.js概念和常用命令总结

目标

把上面学的模块化语法，包的概念，常用命令进行总结

讲解

1. Node.js 模块化：把每个文件当做一个模块，独立作用域，按需加载，使用特定标准语法导出和导入使用

CommonJS 标准：一般应用在 Node.js 项目环境中

ECMAScript 标准：一般应用在前端工程化项目中

CommonJS 标准		
	导出	导入
语法	<code>module.exports = {}</code>	<code>require('模块名或路径')</code>

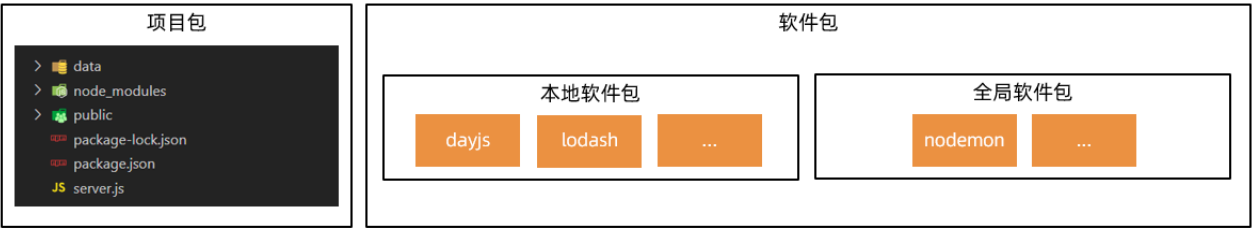
ECMAScript 标准		
	导出	导入
默认	export default {}	import 变量名 from '模块名或路径'
命名	export 修饰定义语句	import { 同名变量 } from '模块名或路径'

2. Node.js 包：把模块文件，代码文件，其他资料聚合成一个文件夹就是包

项目包：编写项目需求和业务逻辑的文件夹

软件包：封装工具和方法进行使用的文件夹（一般使用 npm 管理）

- 本地软件包：作用在当前项目，封装的属性/方法，供项目调用编写业务需求
- 全局软件包：作用在所有项目，一般封装的命令/工具，支撑项目运行



3. Node.js 常用命令：

功能	命令
执行 js 文件	node xxx
初始化 package.json	npm init -y
下载本地软件包	npm i 软件包名
下载全局软件包	npm i 软件包名 -g
删除软件包	npm uni 软件包名

小结

1. 安装本地软件包的命令是什么？

► 答案

03_Webpack模块打包工具

知识点自测

1. 以下哪个选项是 ECMAScript 默认导出和导入的语法？

- A: export 和 require
- B: module.exports = {} 和 import 变量名
- C: export default 和 import 变量名
- D: export 和 import { 同名变量名 }

► 答案

2. 对以下 less 样式代码描述正确的是？

```
.title {  
  span {  
    color: red;  
  }  
}
```

- A: 查找 class 名字叫 title 元素范围内，所有 span 标签
- B: 查找 class 名字叫 title 和 span 标签

► 答案

3. 以下代码运行结果是多少？

```
const arr = [1, 2, 3, 4].map(val => val * 2)  
console.log(arr)
```

- A: [undefined, undefined, undefined, undefined]
- B: [2, 4, 6, 8]

► 答案

4. 以下针对 axios 函数使用时的配置项描述正确的选项？


```
axios({  
  // 这里  
})
```

A: url: 请求的方法, method: 传递参数

B: data: 传递查询参数, params: 传递请求体参数

C: url: 请求的网址, method: 请求的方法

D: data: 传递请求体参数, params: 传递请求体参数

► 答案

01.Webpack 简介以及体验

目标

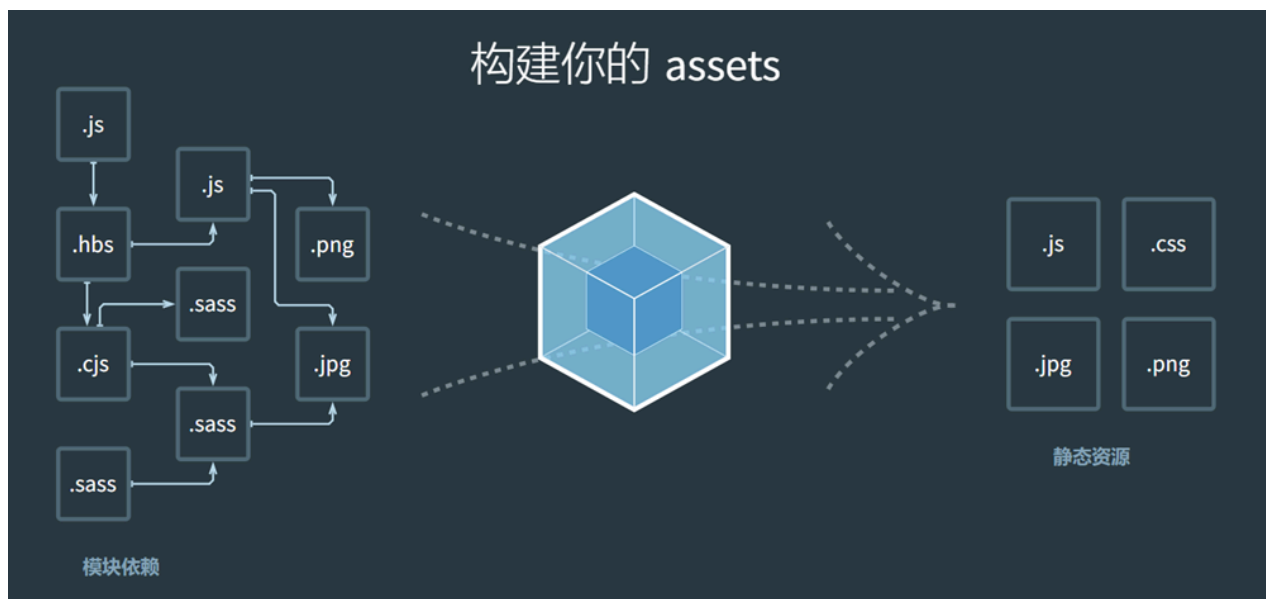
了解 Webpack 的概念和作用, 以及使用

讲解

1. Webpack 是一个静态模块打包工具, 从入口构建依赖图, 打包有关的模块, 最后用于展示你的内容

本质上, **webpack** 是一个用于现代 JavaScript 应用程序的 静态模块打包工具。当 webpack 处理应用程序时, 它会在内部从一个或多个入口点构建一个 依赖图(dependency graph), 然后将你项目中所需的每一个模块组合成一个或多个 *bundles*, 它们均为静态资源, 用于展示你的内容。

2. 静态模块: 编写代码过程中的, html, css, js, 图片等固定内容的文件
3. 打包过程, 注意: 只有和入口有直接/间接引入关系的模块, 才会被打包



4. Webpack 的作用：把静态模块内容，压缩，这个和，转译等（前端工程化）

- 把 less/sass 转成 css 代码
- 把 ES6+ 降级成 ES5 等
- 支持多种模块文件类型，多种模块标准语法

5. 为何不学 vite?

现在很多项目还是基于 Webpack 来进行构建的，所以还是要掌握 Webpack 的使用

6. 体验 Webpack 打包 2 个 JS 文件内容

7. 需求：封装 utils 包，校验手机号和验证码长度，在 src/index.js 中使用，使用 Webpack 打包

8. 步骤：

1. 新建项目文件夹 Webpack_study，初始化包环境，得到 package.json 文件

```
npm init -y
```

2. 新建 src 源代码文件夹（书写代码）包括 utils/check.js 封装用户名和密码长度函数，引入到 src/index.js 进行使用

▪ src/utils/check.js

```
// 封装校验手机号长度和校验验证码长度的函数
export const checkPhone = phone => phone.length === 11
export const checkCode = code => code.length === 6
```

▪ src/index.js

```
/**
 * 目标1：体验 webpack 打包过程
 */
// 1.1 准备项目和源代码
import { checkPhone, checkCode } from '../utils/check.js'
console.log(checkPhone('13900002020'))
console.log(checkCode('123123123123'))
// 1.2 准备 webpack 打包的环境
// 1.3 运行自定义命令打包观察效果（npm run 自定义命令）
```

3. 下载 webpack webpack-cli 到项目（版本独立）

```
npm i webpack webpack-cli --save-dev
```

注意：虽然 webpack 是全局软件包，封装的是命令工具，但是为了保证项目之间版本分别独立，所以这次比较特殊，下载到某个项目环境下，但是需要把 webpack 命令配置到 package.json 的 scripts 自定义命令，作为局部命令使用

```
"scripts": {
  "build": "webpack"
},
```

4. 项目中运行工具命令，采用自定义命令的方式（局部命令）

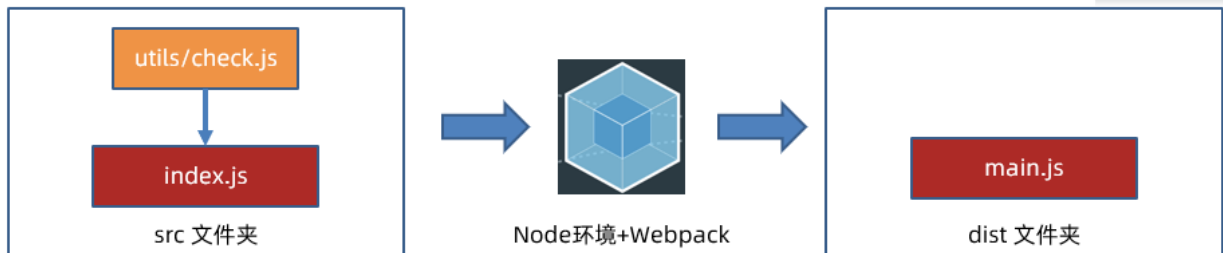
```
npm run build
```

npm run 自定义命令名字

注意：实际上在终端运行的是 build 右侧的具体命名

5. 自动产生 dist 分发文件夹（压缩和优化后，用于最终运行的代码）

9. 需求最终流程图：



小结

1. Webpack 有什么用?
 - ▶ 答案
2. Webpack 怎么用?
 - ▶ 答案
3. 如何运行 package.json 里的自定义命令?
 - ▶ 答案
4. Webpack 默认入口和出口?
 - ▶ 答案

02.Webpack 修改入口和出口

目标

了解 Webpack 配置文件使用，影响 Webpack 打包过程和结果

讲解

1. **Webpack 配置**：影响 Webpack 打包过程
2. 步骤：
 1. 项目根目录，新建 Webpack.config.js 配置文件
 2. 导出配置对象，配置入口，出口文件路径（别忘了修改磁盘文件夹和文件的名字）

```
const path = require('path')

module.exports = {
  entry: path.resolve(__dirname, 'src/login/index.js'),
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: './login/index.js'
  }
}
```

3. 重新打包观察

3. 图解：



小结

1. 如何影响 Webpack 打包过程?

► 答案

03.案例-用户登录-长度判断

目标

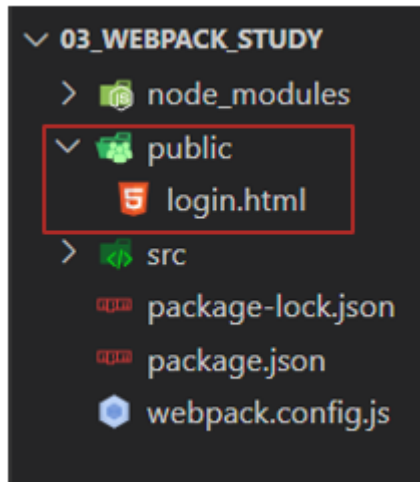
体验前端项目代码，如何被 Webpack 打包和使用

讲解

1. 需求：点击登录按钮，判断手机号和验证码长度是否符合要求

2. 步骤：

1. 新建 public/login.html 准备网页模板（方便查找标签和后期自动生成 html 文件做准备）



2. 核心 JS 代码写在 src/login/index.js 文件

```
/**
 * 目标3: 用户登录-长度判断案例
 * 3.1 准备用户登录页面
 * 3.2 编写核心 JS 逻辑代码
 * 3.3 打包并手动复制网页到 dist 下, 引入打包后的 js, 运行
 */
// 3.2 编写核心 JS 逻辑代码
document.querySelector('.btn').addEventListener('click', () => {
  const phone = document.querySelector('.login-form [name=mobile]').value
  const code = document.querySelector('.login-form [name=code]').value

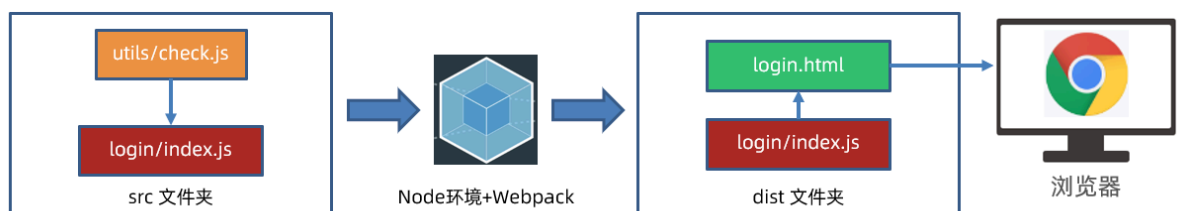
  if (!checkPhone(phone)) {
    console.log('手机号长度必须是11位')
    return
  }

  if (!checkCode(code)) {
    console.log('验证码长度必须是6位')
    return
  }

  console.log('提交到服务器登录...')
})
```

3. 运行自定义命令, 让 Webpack 打包 JS 代码

4. 手动复制 public/login.html 到 dist 下, 手动引入打包后的 JS 代码文件, 运行 dist/login.html 在浏览器查看效果



小结

1. Webpack 打包后的前端代码是如何运行的?

► 答案

04.Webpack 自动生成 html 文件

目标

让 Webpack 拥有自动生成 html 文件能力，并引入打包后的其他资源

讲解

1. **插件 html-webpack-plugin 作用**：在 Webpack 打包时生成 html 文件，并引入其他打包后的资源
2. 步骤：
 1. 下载 html-webpack-plugin 本地软件包到项目中

```
npm i html-webpack-plugin --save-dev
```

2. 配置 webpack.config.js 让 Webpack 拥有插件功能

```
// ...
const HtmlWebpackPlugin = require('html-webpack-plugin')

module.exports = {
  // ...
  plugins: [
    new HtmlWebpackPlugin({
      template: './public/login.html', // 模板文件
      filename: './login/index.html' // 输出文件
    })
  ]
}
```

3. 指定以 public/login.html 为模板复制到 dist/login/index.html，并自动引入其他打包后资源
3. 运行打包命令，观察打包后 dist 文件夹下内容并运行查看效果

小结

1. html-webpack-plugin 插件怎么用?

► 答案

05.Webpack-打包 css 代码

目标

让 Webpack 能够打包 css 代码

讲解

1. 注意：Webpack 默认只识别 JS 和 JSON 文件内容，所以想要让 Webpack 识别更多不同内容，需要使用加载器
2. 介绍需要的 2 个加载器来辅助 Webpack 才能打包 css 代码
 - **加载器 css-loader**：解析 css 代码
 - **加载器 style-loader**：把解析后的 css 代码插入到 DOM（style 标签之间）
3. 步骤：
 1. 准备 css 文件引入到 src/login/index.js 中（压缩转译处理等）

```
/**
 * 目标5: 打包 css 代码
 * 5.1 准备 css 代码，并引入到 js 中
 * 5.2 下载 css-loader 和 style-loader 本地软件包
 * 5.3 配置 webpack.config.js 让 Webpack 拥有该加载器功能
 * 5.4 打包后观察效果
 */
// 5.1 准备 css 代码，并引入到 js 中
import 'bootstrap/dist/css/bootstrap.min.css'
import './index.css'
```

注意：这里只是引入代码内容让 Webpack 处理，不需定义变量接收在 JS 代码中继续使用，所以没有定义变量接收

2. 下载 css-loader 和 style-loader 本地软件包

```
npm i css-loader style-loader --save-dev
```

3. 配置 webpack.config.js 让 Webpack 拥有该加载器功能

```
// ...

module.exports = {
  // ...
  module: { // 加载器
    rules: [ // 规则列表
      {
        test: /\.css$/i, // 匹配 .css 结尾的文件
        use: ['style-loader', 'css-loader'], // 使用从后到前的加载器来解析 css 代码和插入到
        DOM
      }
    ]
  }
};
```

4. 打包后运行 dist/login/index.html 观察效果，看看准备好的样式是否作用在网页上

小结

1. 加载器的作用是什么?

► 答案

06.优化-提取 css 代码

目标

让 Webpack 能够提取 css 代码到独立的 css 文件中

讲解

1. 需求：让 webpack 把 css 代码内容字符串单独提取到 dist 下的 css 文件中
2. 需要：mini-css-extract-plugin 插件来实现
3. 步骤：
 1. 下载 mini-css-extract-plugin 插件软件包到本地项目中

```
npm i --save-dev mini-css-extract-plugin
```

2. 配置 webpack.config.js 让 Webpack 拥有该插件功能

```
// ...
const MiniCssExtractPlugin = require("mini-css-extract-plugin")

module.exports = {
  // ...
  module: {
```



```
rules: [
  {
    test: /\.css$/i,
    // use: ['style-loader', 'css-loader']
    use: [MiniCssExtractPlugin.loader, "css-loader"],
  },
],
},
plugins: [
  // ...
  new MiniCssExtractPlugin()
]
};
```

3. 打包后观察效果
4. 注意：不能和 style-loader 一起使用
5. 好处：css 文件可以被浏览器缓存，减少 JS 文件体积，让浏览器并行下载 css 和 js 文件

小结

07.优化压缩过程

目标

把单独提取的 css 文件内代码压缩

讲解

1. 需求：把提出的 css 文件内样式代码压缩
2. 需要：css-minimizer-webpack-plugin 插件来实现
3. 步骤：
 1. 下载 mini-css-extract-plugin 插件软件包到本地项目中

```
npm i css-minimizer-webpack-plugin --save-dev
```

2. 配置 webpack.config.js 让 Webpack 拥有该插件功能

```
// ...
const CssMinimizerPlugin = require("css-minimizer-webpack-plugin");

module.exports = {
```

```
// ...
// 优化
optimization: {
  // 最小化
  minimizer: [
    // 在 webpack@5 中，你可以使用 ``...` 语法来扩展现有的 minimizer（即
    // `terser-webpack-plugin`），将下一行取消注释（保证 JS 代码还能被压缩处理）
    `...`,
    new CssMinimizerPlugin(),
  ],
}
};
```

3. 打包后观察 css 文件内自己代码是否被压缩了

小结

08.Webpack-打包 less 代码

目标

让 Webpack 拥有打包 less 代码功能

讲解

1. **加载器 less-loader**：把 less 代码编译为 css 代码，还需要依赖 less 软件包
2. 步骤：
 1. 新建 login/index.less 文件，设置背景图样式（图片在配套资料-素材文件夹中）

```
html {
  body {
    background: url('./assets/login-bg.png') no-repeat center/cover;
  }
}
```

2. less 样式引入到 src/login/index.js 中

```
/**
 * 目标8: 打包 less 代码
 * 8.1 新建 less 代码（设置背景图）并引入到 src/login/index.js 中
 * 8.2 下载 less 和 less-loader 本地软件包
 * 8.3 配置 webpack.config.js 让 Webpack 拥有功能
 * 8.4 打包后观察效果
 */
// 8.1 新建 less 代码（设置背景图）并引入到 src/login/index.js 中
import './index.less'
```

3. 下载 less 和 less-loader 本地软件包

```
npm i less less-loader --save-dev
```

4. 配置 webpack.config.js 让 Webpack 拥有功能

```
// ...

module.exports = {
  // ...
  module: {
    rules: [
      // ...
      {
        test: /\.less$/,
        use: [MiniCssExtractPlugin.loader, "css-loader", "less-loader"]
      }
    ]
  }
}
```

5. 打包后运行 观察效果

小结

1. Webpack 支持 less 代码打包需要哪 2 个软件包?

► 答案

09.Webpack-打包图片

目标

让 Webpack 支持图片等资源打包

讲解

1. **资源模块**：Webpack 内置了资源模块的打包，无需下载额外 loader

2. 步骤：

1. 配置 webpack.config.js 让 Webpack 拥有打包图片功能

占位符 **【hash】** 对模块内容做算法计算，得到映射的数字字母组合的字符串

占位符 **【ext】** 使用当前模块原本的占位符，例如：.png / .jpg 等字符串

占位符 **【query】** 保留引入文件时代码中查询参数（只有 URL 下生效）

2. 注意：判断临界值默认为 8KB

大于 8KB 文件：发送一个单独的文件并导出 URL 地址

小于 8KB 文件：导出一个 data URI（base64字符串）

3. 在 src/login/index.js 中给 img 标签添加 logo 图片

```
/**
 * 目标9: 打包资源模块（图片处理）
 * 9.1 创建 img 标签并动态添加到页面，配置 webpack.config.js
 * 9.2 打包后观察效果和区别
 */
// 9.1 创建 img 标签并动态添加到页面，配置 webpack.config.js
// 注意：js 中引入本地图片资源要用 import 方式（如果是网络图片http地址，字符串可以直接写）
import imgObj from './assets/logo.png'
const theImg = document.createElement('img')
theImg.src = imgObj
document.querySelector('.login-wrap').appendChild(theImg)
```

4. 配置 webpack.config.js 让 Webpack 拥有打包图片功能

```
// ...

module.exports = {
  // ...
  module: {
    rules: [
      // ...
      {
        test: /\.?(png|jpg|jpeg|gif)$/i,
        type: 'asset',
        generator: {
          filename: 'assets/[hash][ext][query]'
        }
      }
    ]
  }
}
```

5. 打包后运行观察效果

小结

1. 资源模块指的是什么?

► 答案

10.案例-用户登录-完成功能

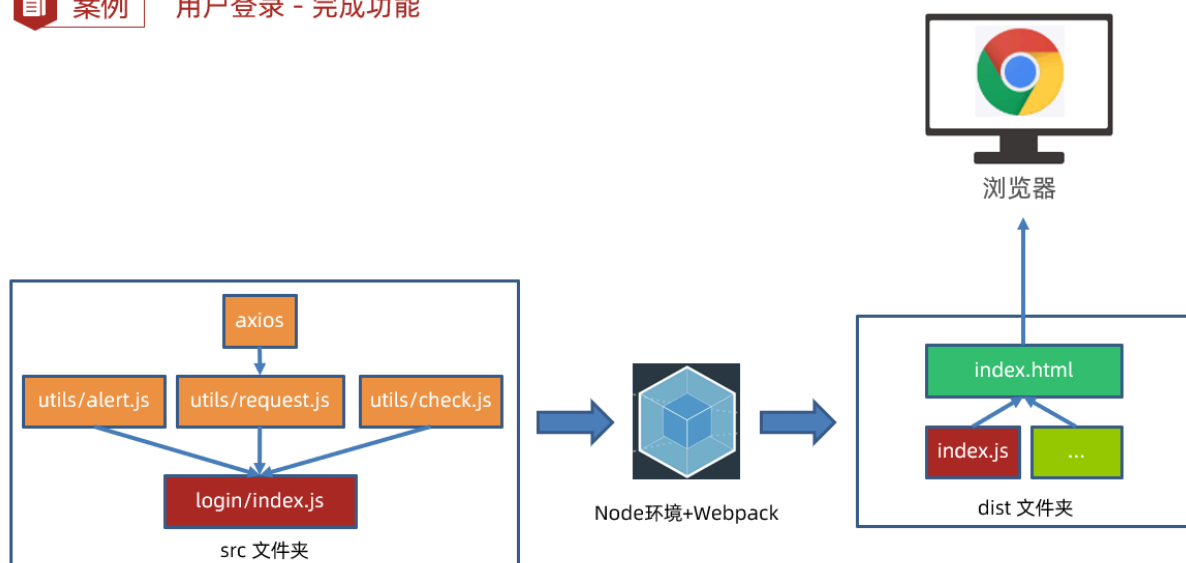
目标

在 Webpack 环境下，使用 npm 下包作用在前端项目

讲解

1. 需求：点击登录按钮，基于 npm 下载 axios 包，完成验证码登录功能

案例 用户登录 - 完成功能



2. 步骤：

1. 使用 npm 下载 axios

```
npm i axios
```

2. 引入到 src/login/index.js 中编写业务实现

```
/**
 * 目标10：完成登录功能
 * 10.1 使用 npm 下载 axios（体验 npm 作用在前端项目中）
 * 10.2 准备并修改 utils 工具包源代码导出实现函数
 * 10.3 导入并编写逻辑代码，打包后运行观察效果
 */
// 10.3 导入并编写逻辑代码，打包后运行观察效果
import myAxios from '../utils/request.js'
```

```

import { myAlert } from '../utils/alert.js'
document.querySelector('.btn').addEventListener('click', () => {
  const phone = document.querySelector('.login-form [name=mobile]').value
  const code = document.querySelector('.login-form [name=code]').value

  if (!checkPhone(phone)) {
    myAlert(false, '手机号长度必须是11位')
    console.log('手机号长度必须是11位')
    return
  }

  if (!checkCode(code)) {
    myAlert(false, '验证码长度必须是6位')
    console.log('验证码长度必须是6位')
    return
  }

  myAxios({
    url: '/v1_0/authorizations',
    method: 'POST',
    data: {
      mobile: phone,
      code: code
    }
  }).then(res => {
    myAlert(true, '登录成功')
    localStorage.setItem('token', res.data.token)
    location.href = '../content/index.html'
  }).catch(error => {
    myAlert(false, error.response.data.message)
  })
})

```

3. 打包后运行观察效果

小结

1. npm 下载的包如何作用在前端项目上?

► 答案

10.Webpack 搭建开发环境

目标

体验 webpack-dev-server 开发服务器，快速开发应用程序

讲解

1. 每次改动代码，都要重新打包，很麻烦，所以这里给项目集成 webpack-dev-server 开发服务器
2. 作用：启动 Web 服务，打包输出源码在内存，并会自动检测代码变化热更新到网页
3. 步骤；

1. 下载 webpack-dev-server 软件包到当前项目

```
npm i webpack-dev-server --save-dev
```

2. 配置自定义命令，并设置打包的模式为开发模式

```
// ...  
  
module.exports = {  
  // ...  
  mode: 'development'  
}
```

```
"scripts": {  
  // ...  
  "dev": "webpack serve --mode=development"  
},
```

3. 使用 npm run dev 来启动开发服务器，访问提示的域名+端口号，在浏览器访问打包后的项目网页，修改代码后试试热更新效果

在 js / css 文件中修改代码保存后，会实时反馈到浏览器

小结

1. webpack-dev-server 的作用？

► 答案

11.Webpack 打包模式

目标

了解不同打包模式对代码和环境的影响

讲解

- 1. **打包模式**：告知 Webpack 使用相应模式的内置优化
- 2. 分类：

模式名称	模式名字	特点	场景
开发模式	development	调试代码，实时加载，模块热替换等	本地开发
生产模式	production	压缩代码，资源优化，更轻量等	打包上线

- 3. 如何设置影响 Webpack呢？
 - 方式1：在 webpack.config.js 配置文件设置 mode 选项

```
// ...  
  
module.exports = {  
  // ...  
  mode: 'production'  
}
```

- 方式2：在 package.json 命令行设置 mode 参数

```
"scripts": {  
  "build": "webpack --mode=production",  
  "dev": "webpack serve --mode=development"  
},
```

- 4. 注意：命令行设置的优先级高于配置文件中的，推荐用命令行设置
- 5. 体验：在 build 命令后 修改 mode 的值，打包输出观察打包后的 js 文件内容

小结

- 1. 两种模式的区别？
 - ▶ 答案

12.Webpack 打包模式的应用

目标

了解 Webpack 打包模式的应用

讲解

1. 需求：在开发模式下用 style-loader 内嵌更快，在生产模式下提取 css 代码
2. **方案 1**：webpack.config.js 配置导出函数，但是局限性大（只接受 2 种模式）

方案2：借助 cross-env （跨平台通用）包命令，设置参数区分环境

方案 3：配置不同的 webpack.config.js （适用多种模式差异性较大情况）

3. 主要使用方案 2 尝试，其他方案可以结合点击跳转的官方文档查看尝试
4. 步骤：

1. 下载 cross-env 软件包到当前项目

```
npm i cross-env --save-dev
```

2. 配置自定义命令，传入参数名和值（会绑定到 process.env 对象下）

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "build": "cross-env NODE_ENV=production webpack --mode=production",  
  "dev": "cross-env NODE_ENV=development webpack serve --open --mode=development"  
},
```

3. 在 webpack.config.js 区分不同环境使用不同配置

```
module: {  
  rules: [  
    {  
      test: /\.css$/i,  
      // use: ['style-loader', 'css-loader'],  
      use: [process.env.NODE_ENV === 'development' ? 'style-loader' :  
MiniCssExtractPlugin.loader, "css-loader"]  
    },  
    {  
      test: /\.less$/i,  
      use: [  
        // compiles Less to CSS  
        process.env.NODE_ENV === 'development' ? 'style-loader' :  
MiniCssExtractPlugin.loader,  
        'css-loader',  
        'less-loader',  
      ],  
    },  
  ],  
}
```

```
},
```

4.重新打包观察两种配置区别

小结

13.Webpack 前端注入环境变量

目标

前端项目中，开发模式下打印语句生效，生产模式下打印语句失效

讲解

1. 需求：前端项目中，开发模式下打印语句生效，生产模式下打印语句失效
2. 问题：cross-env 设置的只在 Node.js 环境生效，前端代码无法访问 process.env.NODE_ENV
3. **解决**：使用 Webpack 内置的 DefinePlugin 插件
4. 作用：在编译时，将前端代码中匹配的变量名，替换为值或表达式
5. 配置 webpack.config.js 中给前端注入环境变量

```
// ...
const webpack = require('webpack')

module.exports = {
  // ...
  plugins: [
    // ...
    new webpack.DefinePlugin({
      // key 是注入到打包后的前端 JS 代码中作为全局变量
      // value 是变量对应的值（在 cross-env 注入在 node.js 中的环境变量字符串）
      'process.env.NODE_ENV': JSON.stringify(process.env.NODE_ENV)
    })
  ]
}
```

小结

14.Webpack 开发环境调错 source map

目标

在开发环境如何精准定位到报错源码位置

讲解

1. **source map**：可以准确追踪 error 和 warning 在原始代码的位置
2. 问题：代码被压缩和混淆，无法正确定位源代码位置（行数和列数）
3. 设置：webpack.config.js 配置 devtool 选项

```
// ...  
  
module.exports = {  
  // ...  
  devtool: 'inline-source-map'  
}
```

inline-source-map 选项：把源码的位置信息一起打包在 JS 文件内

4. 注意：source map 适用于开发环境，不要在生产环境使用（防止被轻易查看源码位置）

小结

1. 为何打包后，在控制台无法准确定位到源码的位置信息？
▶ 答案

15.Webpack 设置解析别名路径

目标

设置 Webpack 如何设置路径别名，方便我们引入目标模块

讲解

1. **解析别名**：配置模块如何解析，创建 import 或 require 的别名，来确保模块引入变得更简单
2. 例如：

1. 原来路径如下：

```
import { checkPhone, checkCode } from '../src/utils/check.js'
```

2. 配置解析别名：在 webpack.config.js 中设置

```
// ...  
  
const config = {  
  // ...  
  resolve: {  
    alias: {  
      '@': path.resolve(__dirname, 'src')  
    }  
  }  
}
```

3. 这样我们以后，引入目标模块写的路径就更简单了

```
import { checkPhone, checkCode } from '@/utils/check.js'
```

3. 修改代码的路径后，重新打包观察效果是否正常！

小结

1. 路径中的 '@' 符号代表什么意思？

► 答案

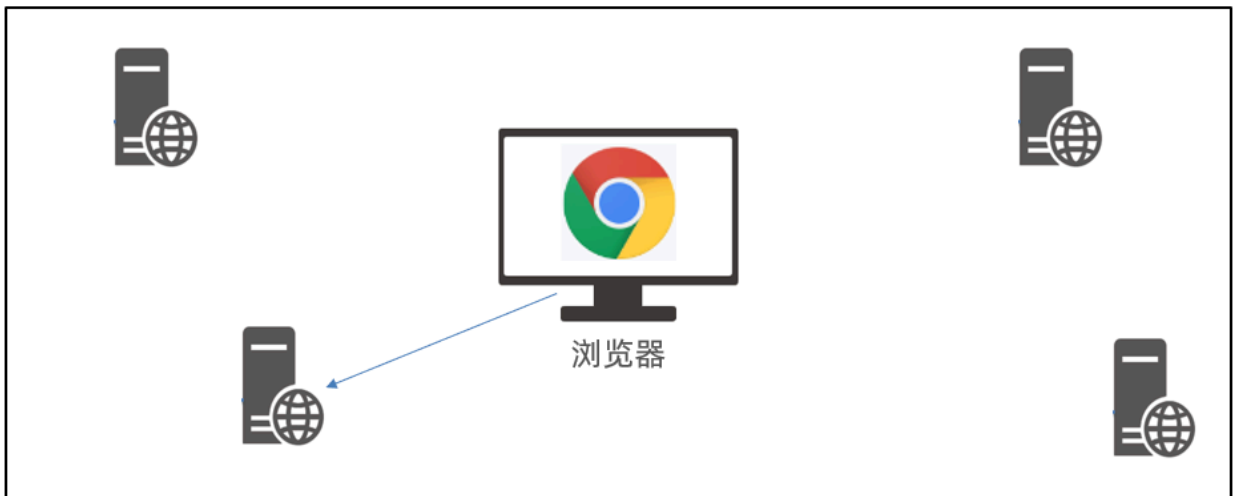
16. 优化-CDN使用

目标

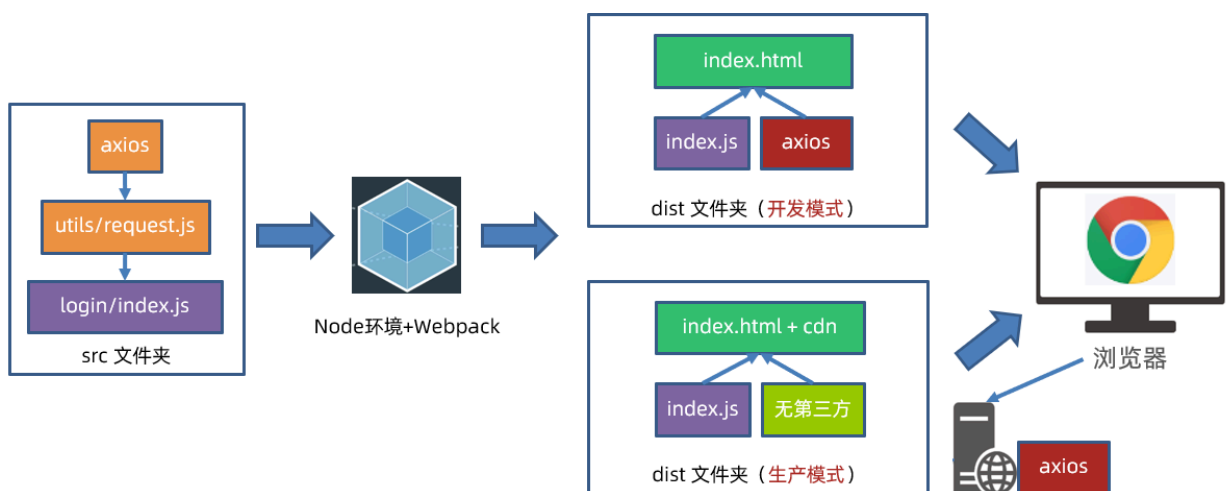
开发模式使用本地第三方库，生产模式下使用 CDN 加载引入

讲解

1. 需求：开发模式使用本地第三方库，生产模式下使用 CDN 加载引入
2. **CDN 定义**：内容分发网络，指的是一组分布在各个地区的服务器
3. 作用：把静态资源文件/第三方库放在 CDN 网络中各个服务器中，供用户就近请求获取
4. 好处：减轻自己服务器请求压力，就近请求物理延迟低，配套缓存策略



5. 实现需求的思路图：



6. 步骤：

1. 在 html 中引入第三方库的 **CDN 地址** 并用模板语法判断

```
<% if(htmlWebpackPlugin.options.useCdn){ %>
  <link href="https://cdn.bootcdn.net/ajax/libs/twitter-bootstrap/5.2.3/css/bootstrap.min.css" rel="stylesheet">
<% } %>
```

2. 配置 webpack.config.js 中 **externals** 外部扩展选项（防止某些 import 的包被打包）

```
// 生产环境下使用相关配置
if (process.env.NODE_ENV === 'production') {
  // 外部扩展（让 webpack 防止 import 的包被打包进来）
  config.externals = {
    // key: import from 语句后面的字符串
    // value: 留在原地的全局变量（最好和 cdn 在全局暴露的变量一致）
    'bootstrap/dist/css/bootstrap.min.css': 'bootstrap',
    'axios': 'axios'
  }
}
```

```
// ...
const config = {
  // ...
  plugins: [
    new HtmlWebpackPlugin({
      // ...
      // 自定义属性, 在 html 模板中 <%=htmlWebpackPlugin.options.useCdn%> 访问使用
      useCdn: process.env.NODE_ENV === 'production'
    })
  ]
}
```

3.两种模式下打包观察效果

小结

17.Webpack 多页面打包

目标

让 Webpack 同时打包登录和内容列表页面

讲解

- 概念：**单页面**：单个 html 文件，切换 DOM 的方式实现不同业务逻辑展示，后续 Vue/React 会学到
多页面：多个 html 文件，切换页面实现不同业务逻辑展示
- 需求：把黑马头条-数据管理平台-内容页面一起引入打包使用
- 步骤：
 - 准备源码（html，css，js）放入相应位置，并改用模块化语法导出
 - 下载 form-serialize 包并导入到核心代码中使用
 - 配置 webpack.config.js 多入口和多页面的设置

```
// ...
const config = {
  entry: {
    '模块名1': path.resolve(__dirname, 'src/入口1.js'),
    '模块名2': path.resolve(__dirname, 'src/入口2.js'),
  },
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: './[name]/index.js'
  }
}
```

```
}
plugins: [
  new HtmlWebpackPlugin({
    template: './public/页面2.html', // 模板文件
    filename: './路径/index.html', // 输出文件
    chunks: ['模块名2']
  })
  new HtmlWebpackPlugin({
    template: './public/页面2.html', // 模板文件
    filename: './路径/index.html', // 输出文件
    chunks: ['模块名2']
  })
]
}
```

4. 重新打包观察效果

小结

18.案例-发布文章页面打包

目标

案例-发布文章页面打包

讲解

1. 需求：把发布文章页面一起打包

步骤：

- 1.准备发布文章页面源代码，改写成模块化的导出和导入方式
- 2.修改 webpack.config.js 的配置，增加一个入口和出口
- 3.打包观察效果

小结

19.优化-分割公共代码

目标

优化-分割功能代码

讲解

1. 需求：把 2 个以上页面引用的公共代码提取

步骤：

- 1.配置 webpack.config.js 的 splitChunks 分割功能

```
// ...
const config = {
  // ...
  optimization: {
    // ...
    splitChunks: {
      chunks: 'all', // 所有模块动态非动态移入的都分割分析
      cacheGroups: { // 分隔组
        commons: { // 抽取公共模块
          minSize: 0, // 抽取的chunk最小大小字节
          minChunks: 2, // 最小引用数
          reuseExistingChunk: true, // 当前 chunk 包含已从主 bundle 中拆分出的模块，则它将被重用
          name(module, chunks, cacheGroupKey) { // 分离出模块文件名
            const allChunksNames = chunks.map((item) => item.name).join('~') // 模块名1~模块名
            2
            return `./js/${allChunksNames}` // 输出到 dist 目录下位置
          }
        }
      }
    }
  }
}
```

- 2.打包观察效果

小结

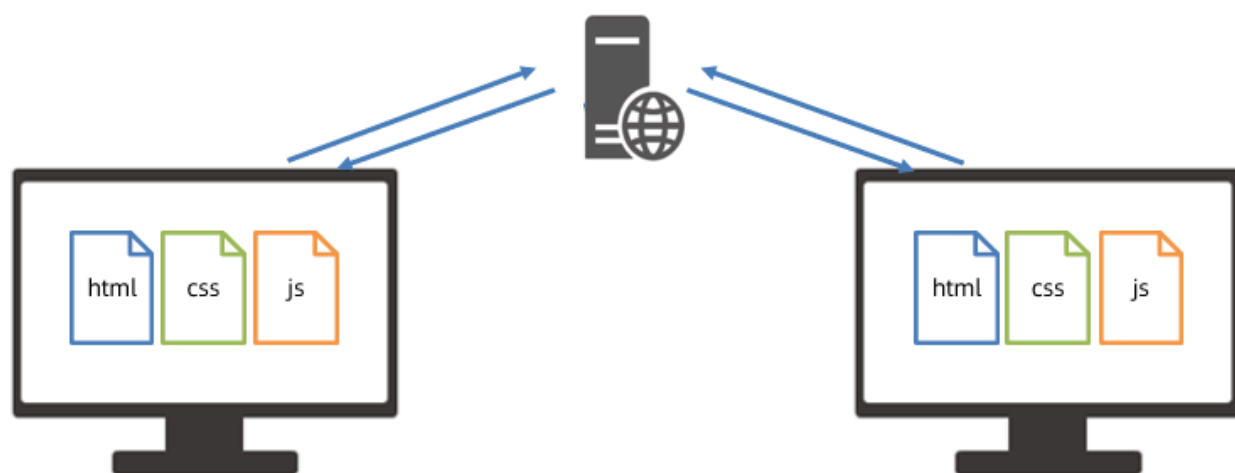
黑马头条-数据管理平台

01.Git 初识

1. **概念**：一个免费开源，分布式的代码版本控制系统，帮助开发团队维护代码
2. 作用：记录代码内容，切换代码版本，多人开发时高效合并代码内容
3. 如何学：

个人本机使用：Git 基础命令和概念

多人共享使用：团队开发同一个项目的代码版本管理



Windows 系统：exe 程序，默认下一步即可

Mac系统：dmg 程序，默认下一步即可

检验成功：

1. 打开 bash 终端（git 专用）
2. 命令：git -v（查看版本号）

```
lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study
$ git -v
git version 2.39.2.windows.1

lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study
$
```

配置：用户名和邮箱，应用在每次提交代码版本时表明自己身份

命令：

```
git config --global user.name "itheima"
```

```
git config --global user.email "itheima@itcast.cn"
```

```
问题 输出 终端 调试控制台

lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study
$ git config --global user.name "itheima"

lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study
$ git config --global user.email "itheima@itcast.cn"

lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study
$
```

1.为何学习 Git？

- ü 管理代码版本，记录，切换，合并代码

2.Git 学习：

- ü 现在本机自己使用

- ü 再学习多人共享使用

3.如何安装使用？

- ü 程序双击安装

- ü 在 VSCode 中使用 bash 终端以及 git 命令

02.Git 仓库

Git 仓库（repository）：记录文件状态内容的地方，存储着修改的历史记录



创建：

- 1.把本地文件夹转换成 Git 仓库：命令 git init
- 2.从其他服务器上克隆 Git 仓库



需求：创建一个空白的 Git 仓库

- 1.什么是 Git 仓库？
 - ü 记录文件状态内容和历史记录的地方（.git 文件夹）
- 2.如何创建 Git 仓库？
 - ü 把本地文件夹转换成 Git 仓库：命令 git init
 - ü 从其他服务器上克隆 Git 仓库

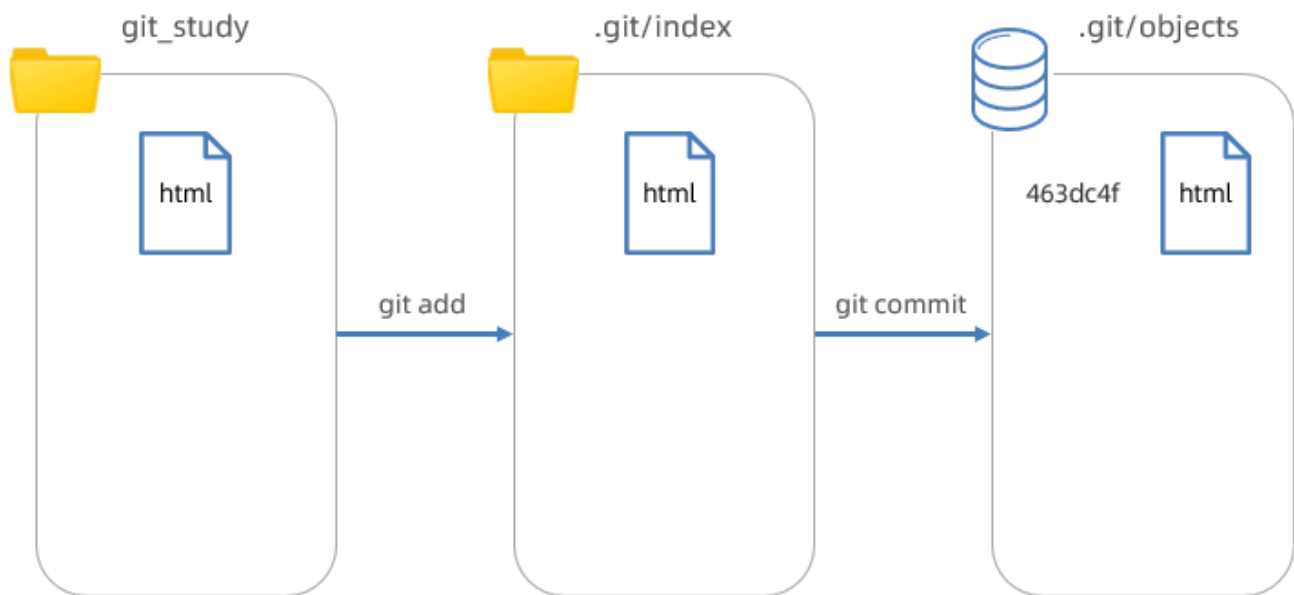
03.Git 的三个区域

Git 使用时：

- 工作区：实际开发时操作的文件夹
- 暂存区：保存之前的准备区域（暂存改动过的文件）
- 版本库：提交并保存暂存区中的内容，产生一个版本快照

命令	作用
git add 文件名	暂存指定文件
git add .	暂存所有改动的文件
git commit -m "注释说明"	提交并保存，产生版本快照

需求：把登录页面新增后，暂存并提交



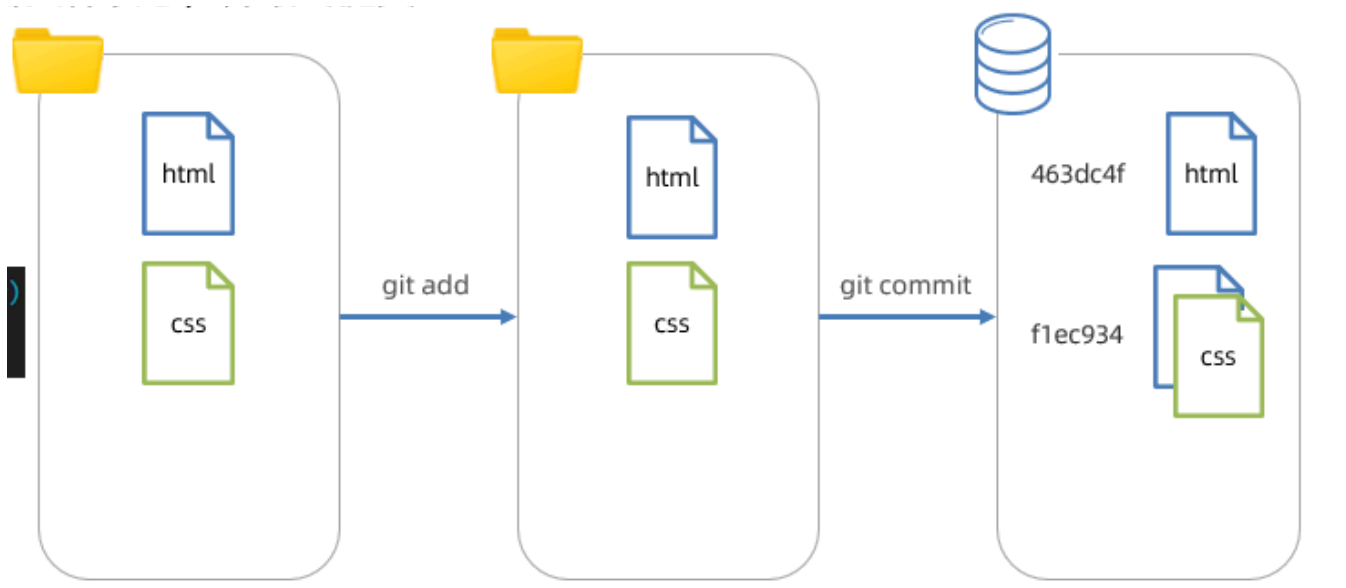
- 1.Git 使用时有哪些区域？
 - ü 工作区，暂存区，版本库
- 2.工作区的内容，最终要如何保存在版本库中？
 - ü git add 添加到暂存区
 - ü等待时机后 git commit 提交保存到版本库，产生一次版本快照记录

04.Git 文件状态

- Git 文件 2 种状态：
- ü未跟踪：新文件，从未被 Git 管理过
 - ü已跟踪：Git 已经知道和管理的文件

文件状态	概念	场景
未跟踪（U）	从未被 Git 管理过	新文件
新添加（A）	第一次被 Git 暂存	之前版本记录无此文件
未修改（"）	三个区域统一	提交保存后
已修改（M）	工作区内容变化	修改了内容产生

使用：修改文件，暂存，提交保存记录，如此反复



需求：新增 css 文件，并使用 `git status -s` 查看文件状态，并最终提交

第一列是暂存区状态

第二列是工作区状态

```
lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study (master)
$ git status -s
MM login/index.css
```

1. Git 文件状态分为哪 2 种？

ü 未跟踪和已跟踪（新添加，未修改，已修改）

2. 如何查看暂存区和工作区文件状态？

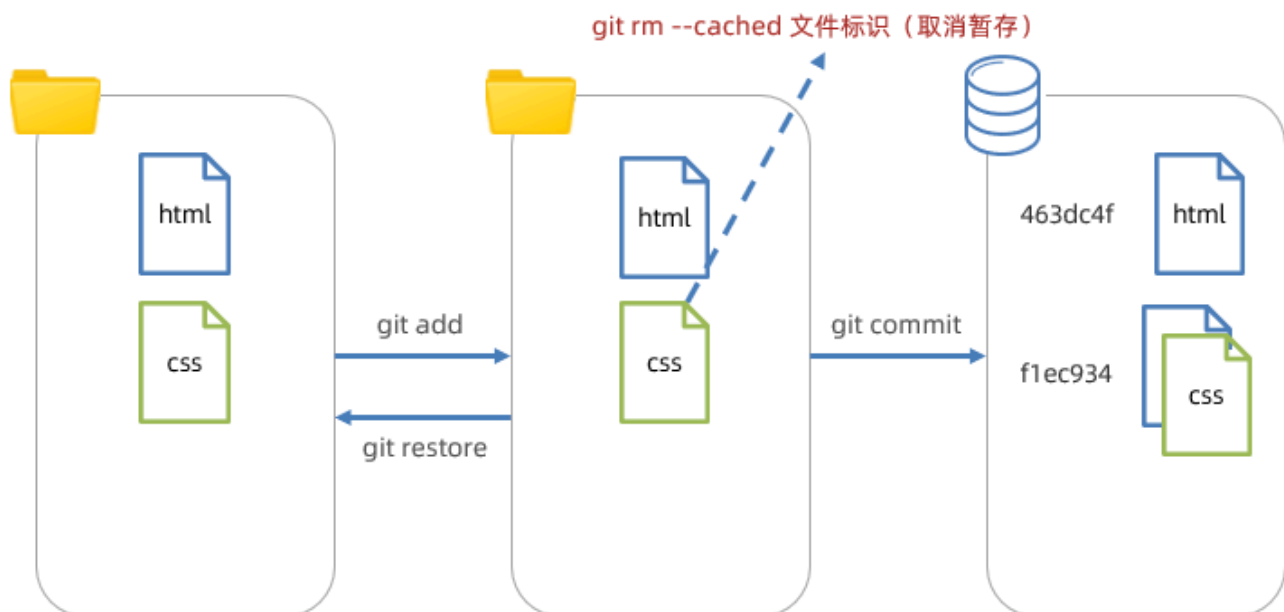
ü `git status -s`

05. Git 暂存区作用

暂存区：暂时存储，可以临时恢复代码内容，与版本库解耦

暂存区 -> 覆盖 -> 工作区，命令：`git restore 目标文件`（注意：完全确认覆盖时使用）

从暂存区移除文件，命令：`git rm --cached 目标文件`



1.如何移除暂存区已暂存的文件?

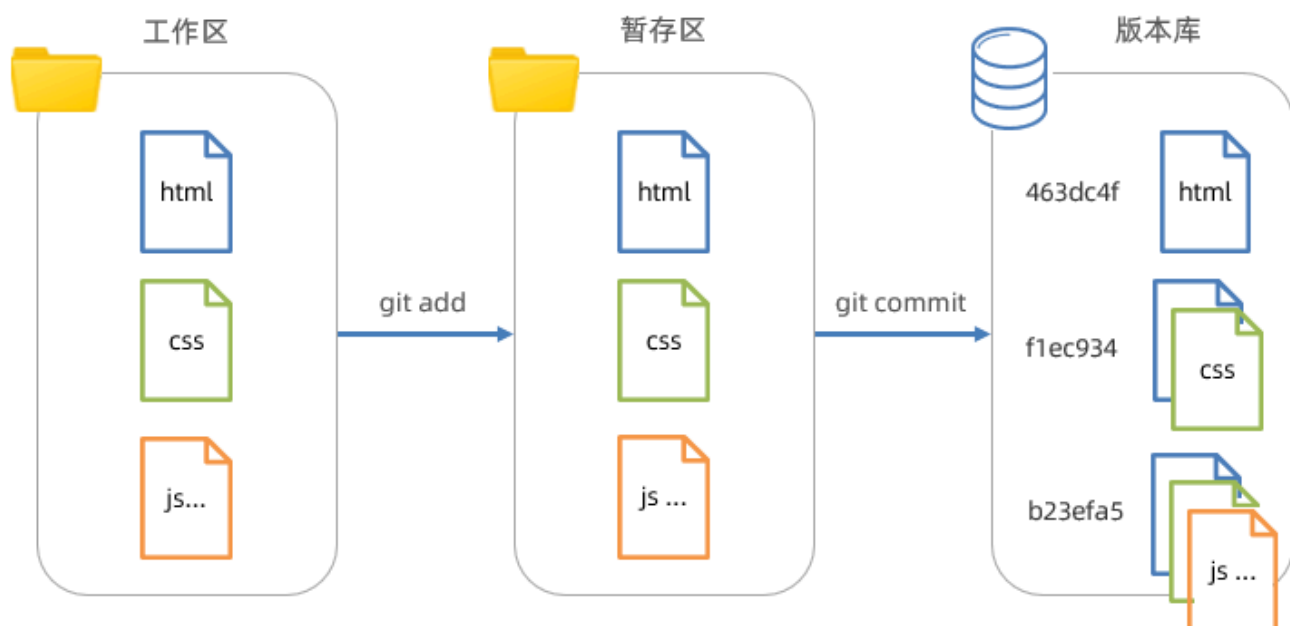
ü `git rm --cached` 目标文件

06.练习-登录页面

需求：新增 JS 代码并暂存提交产生新的版本快照

步骤

- 1.新增 js 文件和内容
- 2.临时存放在暂存区
- 3.提交保存到版本库



07.Git-切换版本

概念：把版本库某个版本对应的内容快照，恢复到工作区/暂存区

查看提交历史：git log --oneline

```
lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study (master)
$ git log --oneline
e486866 (HEAD -> master) 3.登录页面-开发完毕
d7ee31a 2.登录页面-css样式
a219584 1.登录页面-标签部分准备
```

回退命令：

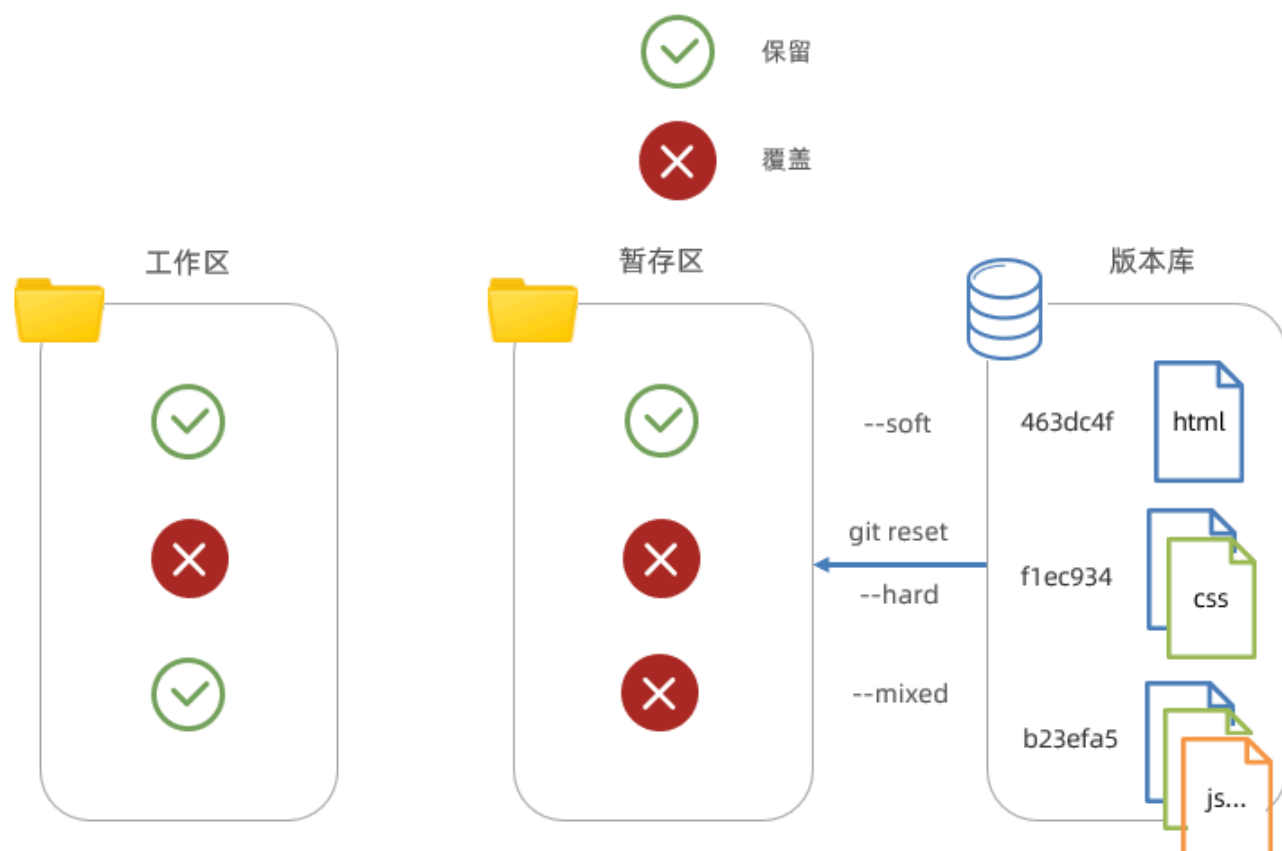
git reset --soft 版本号（其他文件未跟踪）

git reset --hard 版本号

git reset --mixed 版本号（与 git reset 等价）

注意1：只有记录在版本库的提交记录才能恢复

注意2：回退后，继续修改->暂存->提交操作即可（产生新的提交记录过程）



1.什么是 Git 回退版本？

ü 把版本库某个版本对应的内容快照，恢复到工作区/暂存区

2.强制覆盖暂存区和工作区的命令？

ü git reset --hard 版本号

3.如何查看提交历史？

ü git log --oneline

ü git reflog --oneline

08.删除文件

需求：删除 editor.js 文件，并产生一次版本记录

步骤：

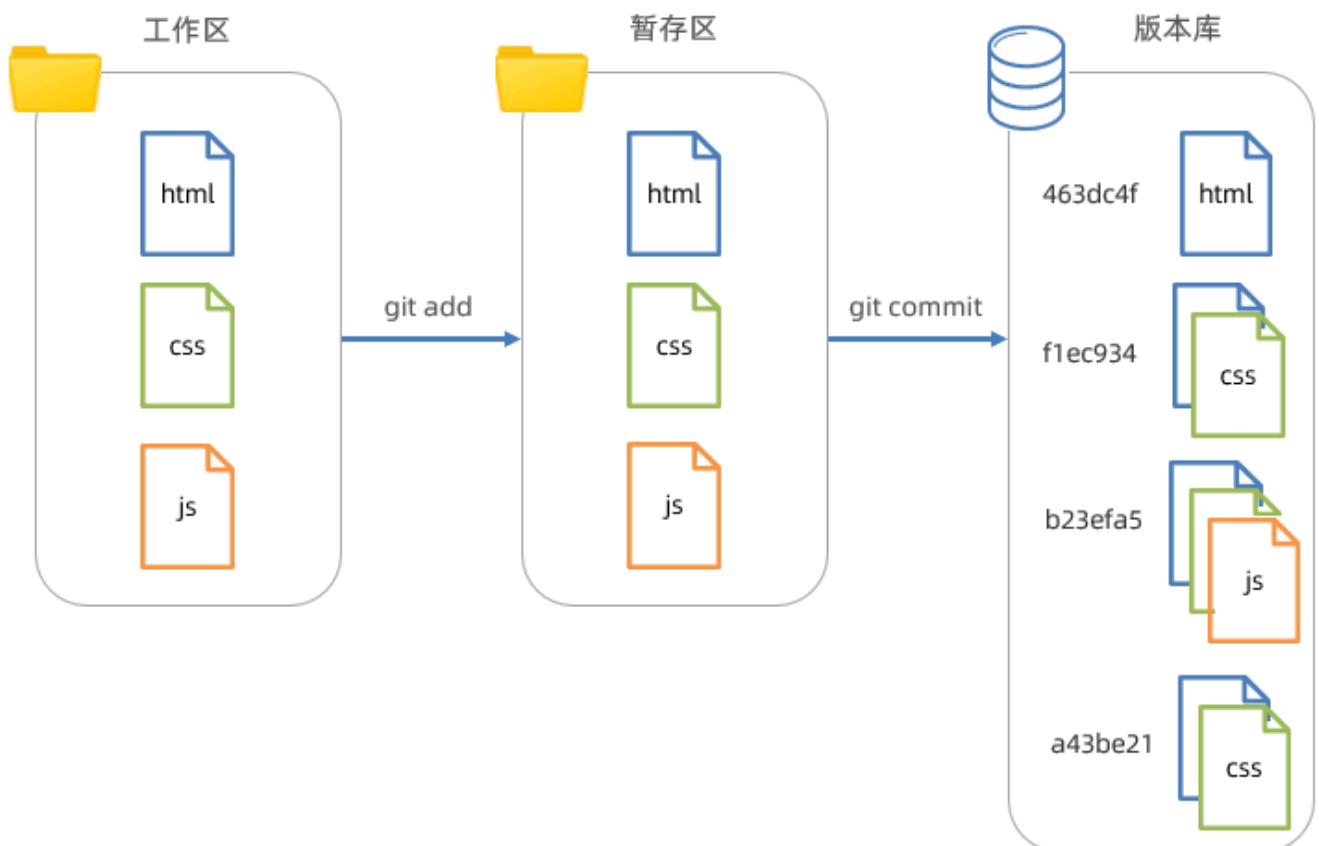
1.手动删除工作区文件

2.暂存变更/手动删除暂存区文件造成变更

3.提交保存

总结：

工作区只要改变，都可以暂存提交产生新记录



09.忽略文件

概念：.gitignore 文件可以让 git 彻底忽略跟踪指定文件

目的：让 git 仓库更小更快，避免重复无意义的文件管理

例如：

- 1.系统或软件自动生成的文件
- 2.编译产生的结果文件
- 3.运行时生成的日志文件，缓存文件，临时文件等
- 4.涉密文件，密码，秘钥等文件

创建：

- 1.项目根目录新建 .gitignore 文件
- 2.填入相应配置来忽略指定文件

注意：如果文件已经被暂存区跟踪过，可以从暂存区移除即可



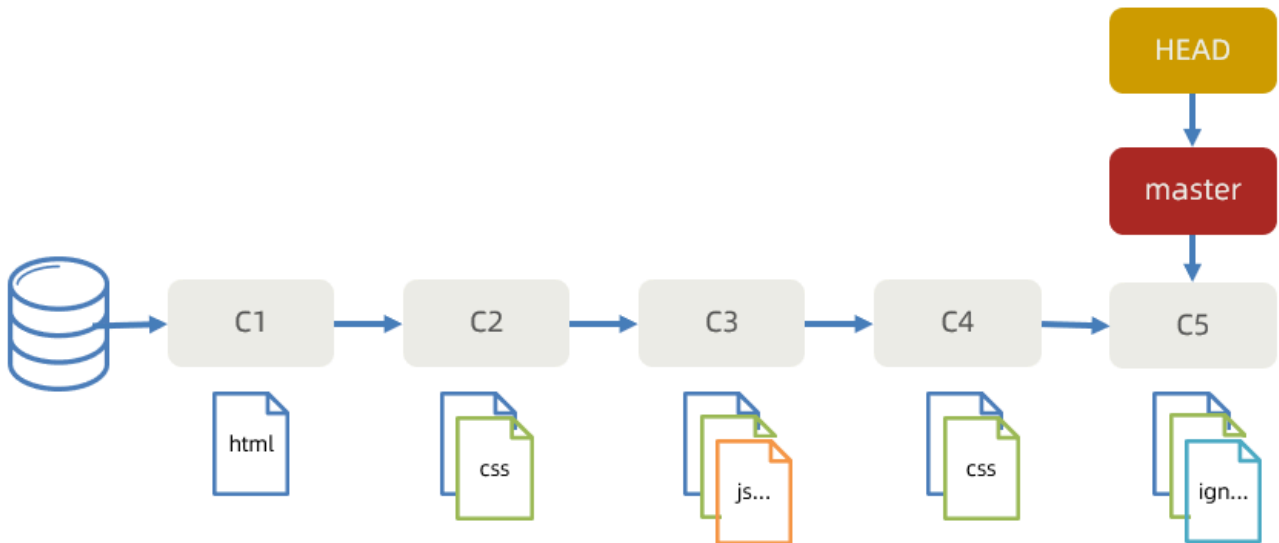
```
.gitignore
1  # 忽略 npm 下载的第三方包
2  node_modules
3  # 忽略分发文件夹
4  dist
5  # 忽略 VSCode 配置文件
6  .vscode
7  # 忽略秘钥文件
8  *.pem
9  *.cer
10 # 忽略日志文件
11 *.log
```

10.分支的概念

概念：本质上是指向提交节点的可变指针，默认名字是 master

注意：HEAD 指针影响工作区/暂存区的代码状态

```
lenovo@LAPTOP-4DIK6LSC MINGW64 /d/备课代码/git_study (master)
$ git log --oneline
fbb472a (HEAD -> master) 5.忽略文件增加
e4b8f45 4.登录页面-删除editor.js
e486866 3.登录页面-开发完毕
d7ee31a 2.登录页面-css样式
a219584 1.登录页面-标签部分准备
```

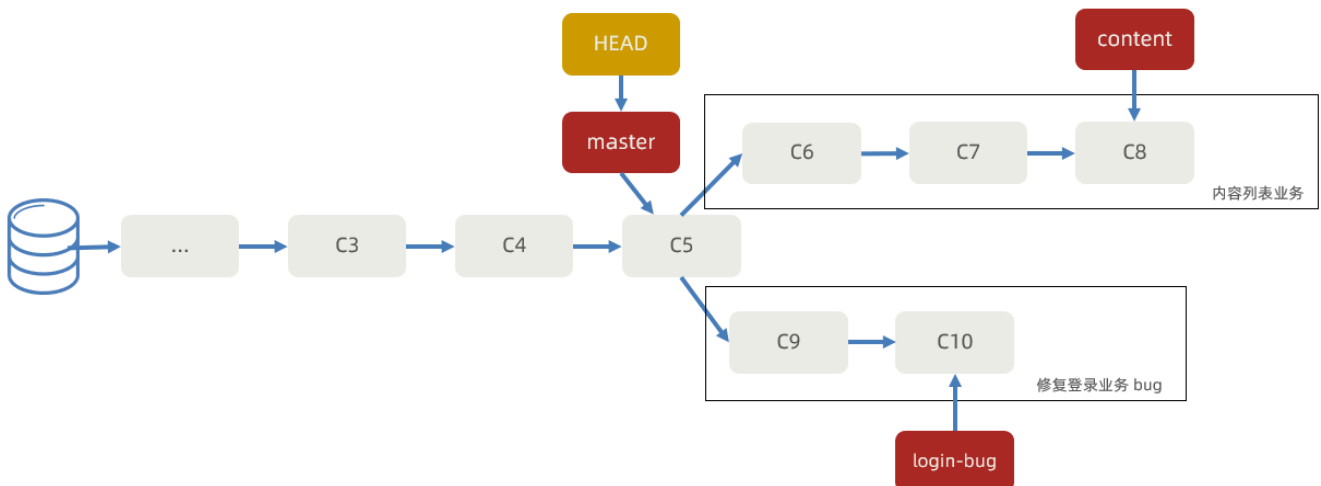


场景：开发新需求 / 修复 Bug，保证主线代码随时可用，多人协同开发提高效率

例如：

在现有代码上创建新分支完成内容列表业务

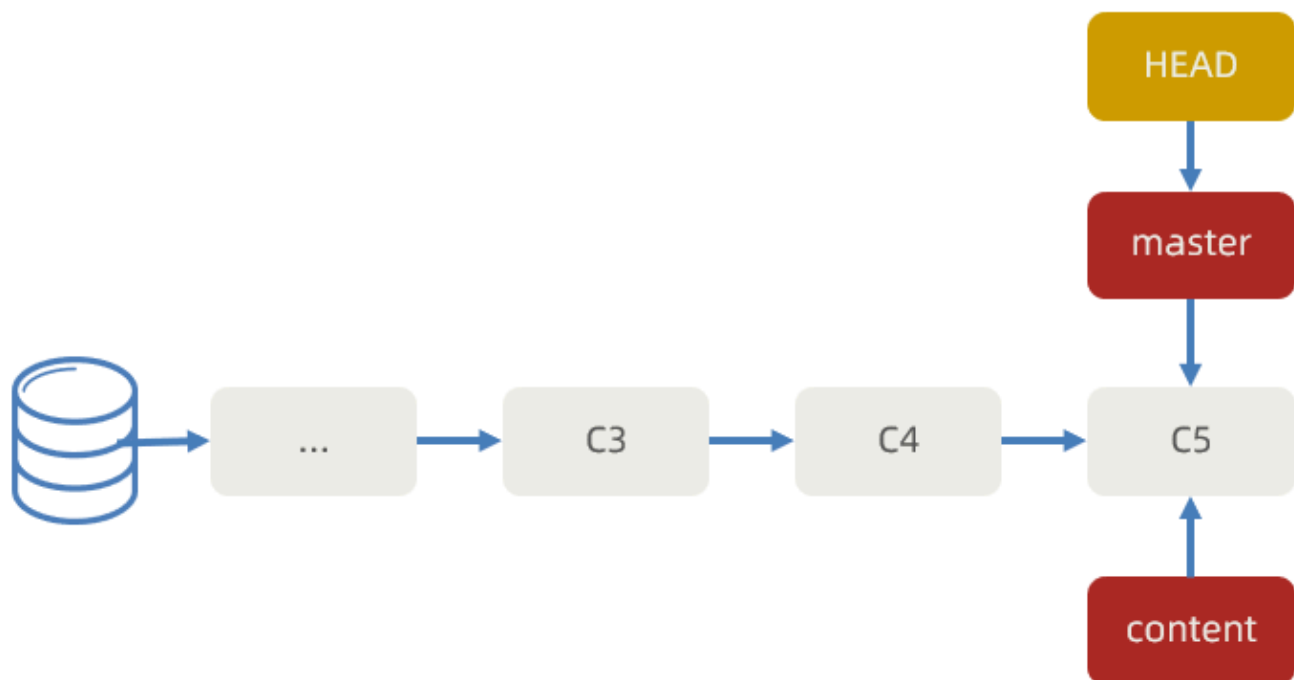
突然需要紧急修复 Bug - 单独创建分支解决 Bug



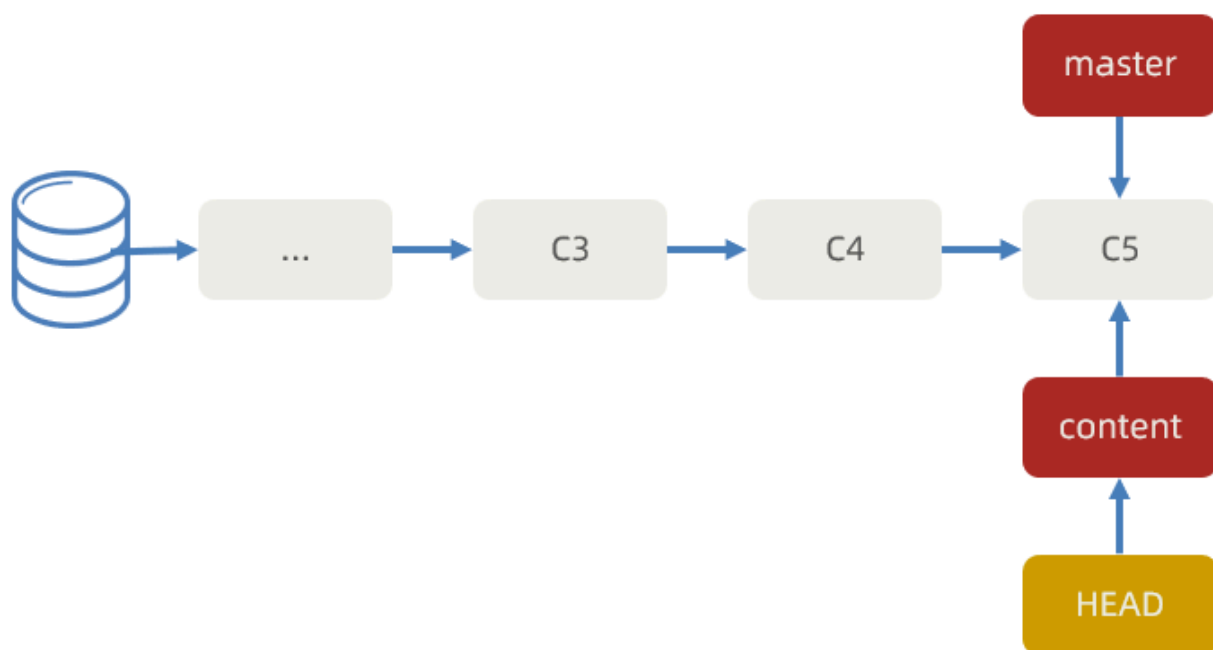
需求：创建内容列表 content 分支，并产生 3 次提交记录

步骤：

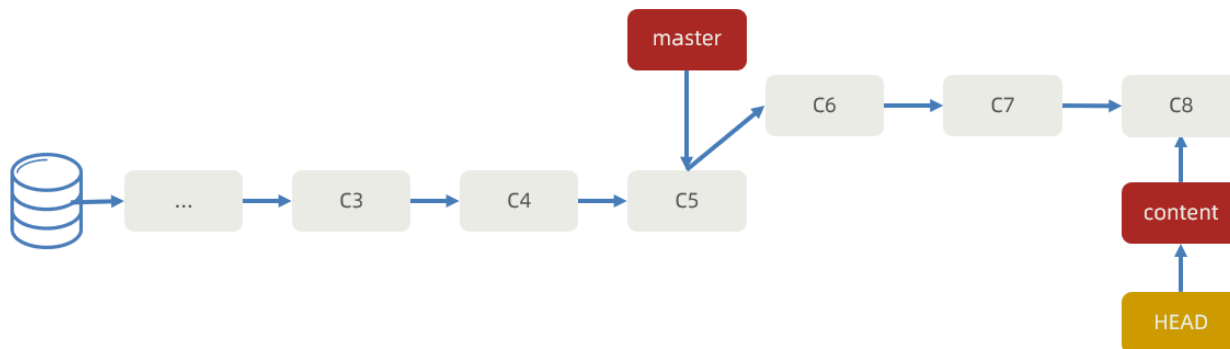
1. 创建分支命令：git branch 分支名



2. 切换分支命令：git checkout 分支名



3. 工作区准备代码并暂存提交，重复 3 次



1.什么是 Git 分支？

ü 指针，指向提交记录

2.HEAD 指针的作用？

ü 影响暂存区和工作区的代码

3.如何创建和切换指针？

ü git branch 分支名

ü git checkout 分支名

11.练习-登录 bug 修复

需求：新建 login-bug 分支，做 2 次提交记录（对手机号长度，验证码长度做判断）

步骤：

1.切回到主分支：git checkout master

2.创建新分支：git branch login-bug

3.切换新分支：git checkout login-bug

4.修改代码，暂存，提交产生版本记录

12.分支-合并与删除

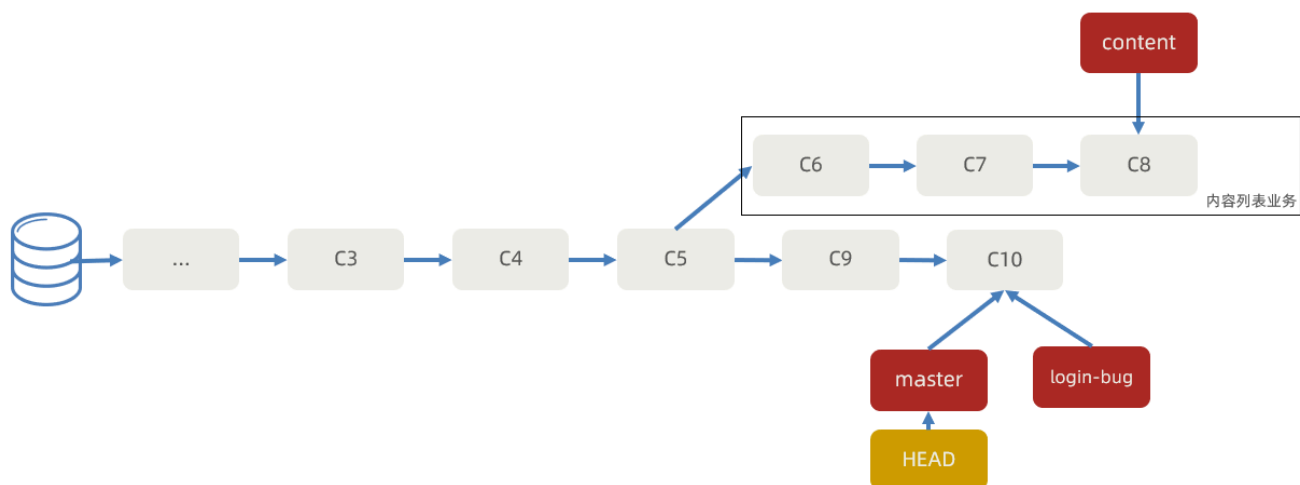
需求：把 login-bug 合并回到 master 分支并删除 login-bug 分支

步骤：

1.切回到要合入的分支上：git checkout master

2.合并其他分支过来：git merge login-bug

3.删除合并后的分支指针：git branch -d login-bug

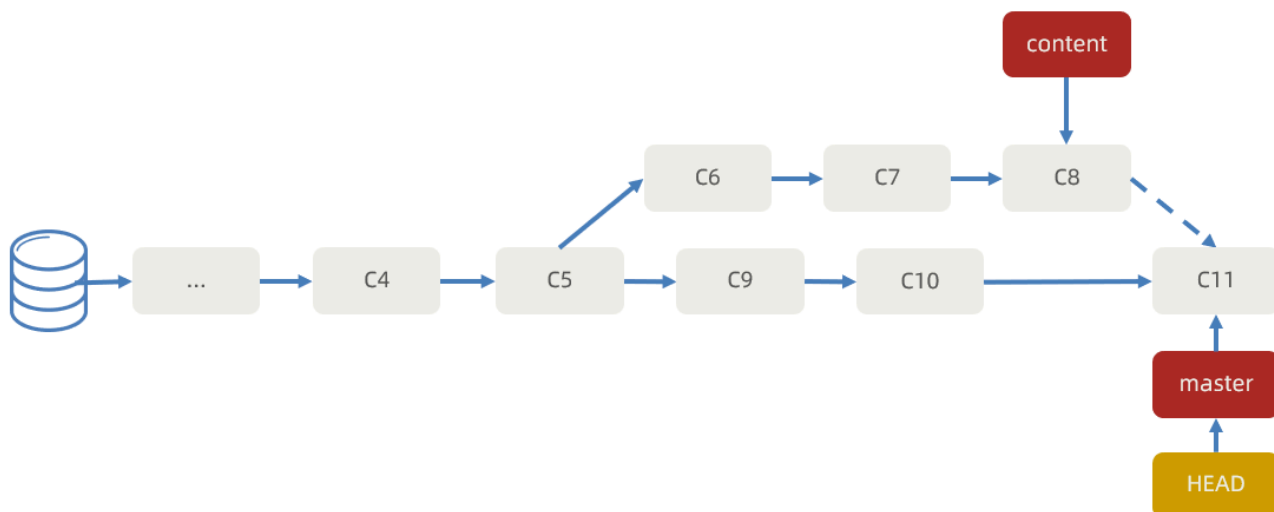


13.分支-合并与提交

合并提交：发生于原分支产生了新的提交记录后，再合并回去时发生，自动使用多个快照记录合并后产生一次新的提交

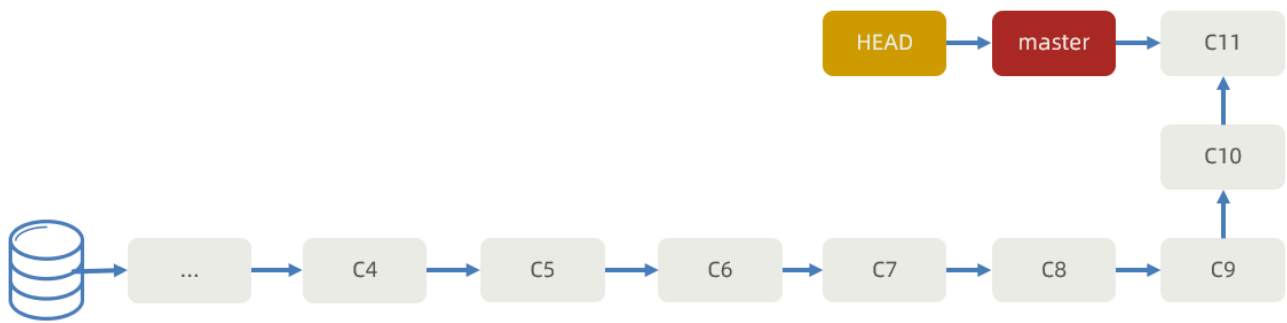
步骤：

- 1.切回到要合入的分支上：git checkout master
- 2.合并其他分支过来：git merge content
- 3.删除合并后的分支：git branch -d content



最后合并回到主分支上时，提交记录流程图：

注意：提交记录的顺序按照产生的先后顺序排列，而非合并的先后顺序



14.分支-合并冲突

需求1：基于 master 新建 publish 分支，完成发布文章业务，然后修改内容页面的 html 文件的 title 标签，并提交一次

需求2：切换到 master，也在修改内容页面的 html 文件的 title 标签，并提交一次

冲突：把 publish 分支合并到 master 回来，产生合并冲突

概念：不同分支中，对同一个文件的同一部分修改，Git 无法干净的合并，产生合并冲突

解决：

1.打开 VSCode 找到冲突文件并手动解决

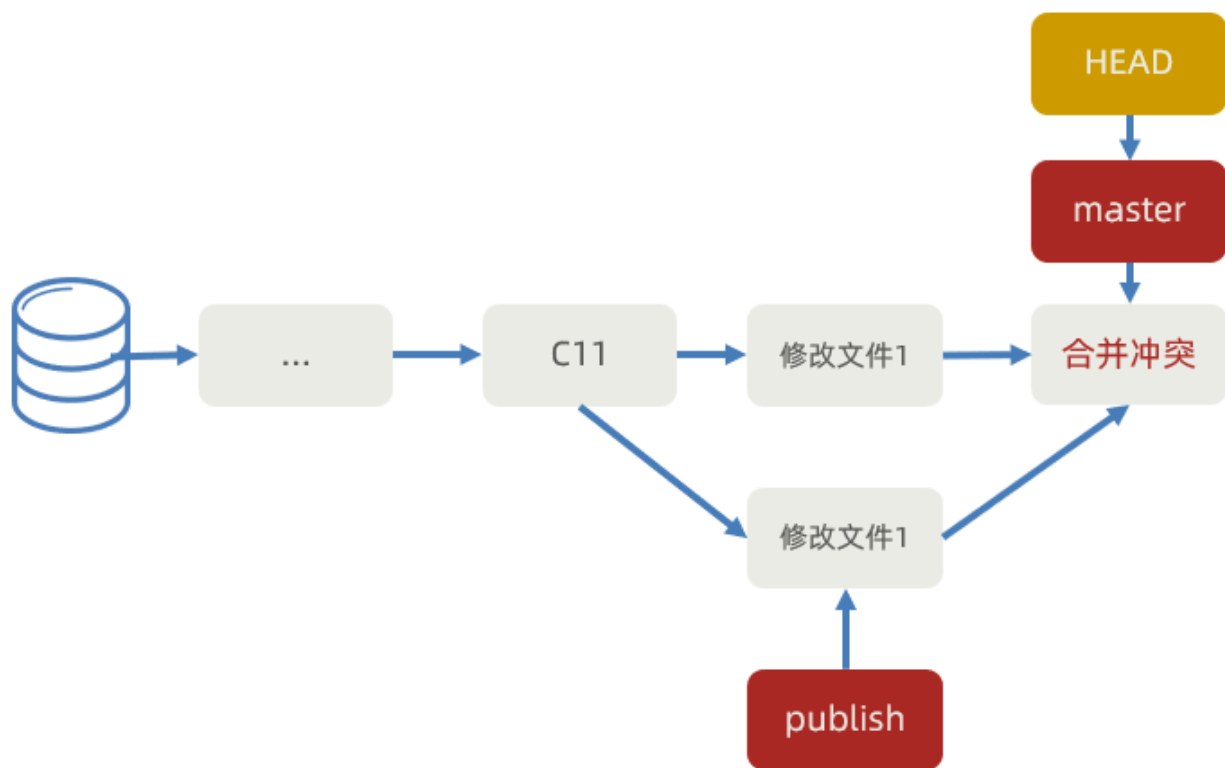
2.解决后需要提交一次记录

避免：（多交流）

1.按页面划分不同分支开发

2.公共代码在统一文件夹维护

3.Node等软件版本统一，npm 包统一下载



15.Git 常用命令

命令	作用	注意
git -v	查看 git 版本	
git init	初始化 git 仓库	
git add 文件标识	暂存某个文件	文件标识以终端为起始的相对路径
git add .	暂存所有文件	
git commit -m '说明注释'	提交产生版本记录	每次提交，把暂存区内容快照一份
git status	查看文件状态 - 详细信息	
git status -s	查看文件状态 - 简略信息	第一列是暂存区状态，第二列是工作区状态
git ls-files	查看暂存区文件列表	
git restore 文件标识	从暂存区恢复到工作区	如果文件标识为 . 则恢复所有文件
git rm --cached 文件标识	从暂存区移除文件	不让 git 跟踪文件变化
git log	查看提交记录 - 详细信息	
git log --oneline	查看提交记录 - 简略信息	版本号 分支指针 提交时说明注释

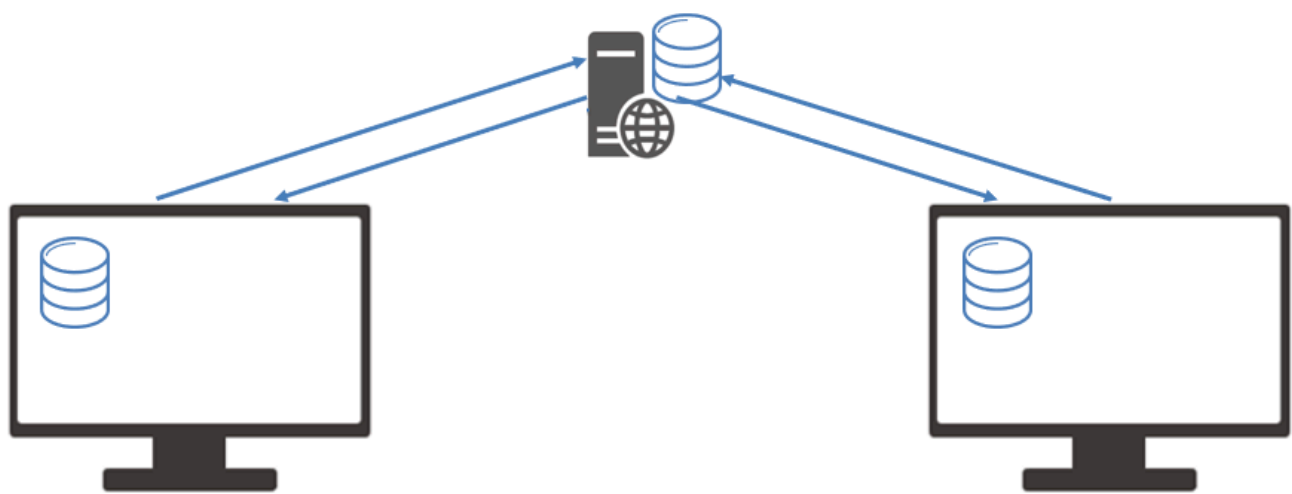
命令	作用	注意
git reflog --oneline	查看完整历史 - 简略消息	包括提交，切换，回退等所有记录
git reset 版本号	切换版本代码到暂存区和工作区	--soft 模式保留暂存区和工作区原本内容 --hard 模式不保留暂存区和工作区原本内容 --mixed 模式不保留暂存区，工作区保留（默认）先覆盖到暂存区，再用暂存区对比覆盖工作区
git branch 分支名	创建分支	
git branch	查看本地分支	
git branch -d 分支名	删除分支	请确保记录已经合并到别的分支下，再删除分支
git checkout 分支名	切换分支	
git checkout -b 分支名	创建并立刻切换分支	
git merge 分支名	把分支提交历史记录合并到当前所在分支	

16.Git 远程仓库

概念：托管在因特网或其他网络中的你的项目的版本库

作用：保存版本库的历史记录，多人协作

创建：公司自己服务器 / 第三方托管平台（Gitee，GitLab，GitHub...）



需求：创建远程版本库，并把本地 Git 仓库推送上去保存

步骤：

1.注册第三方托管平台网站账号

2.新建仓库得到远程仓库 Git 地址

3.本地 Git 仓库添加远程仓库原点地址

命令：git remote add 远程仓库别名 远程仓库地址

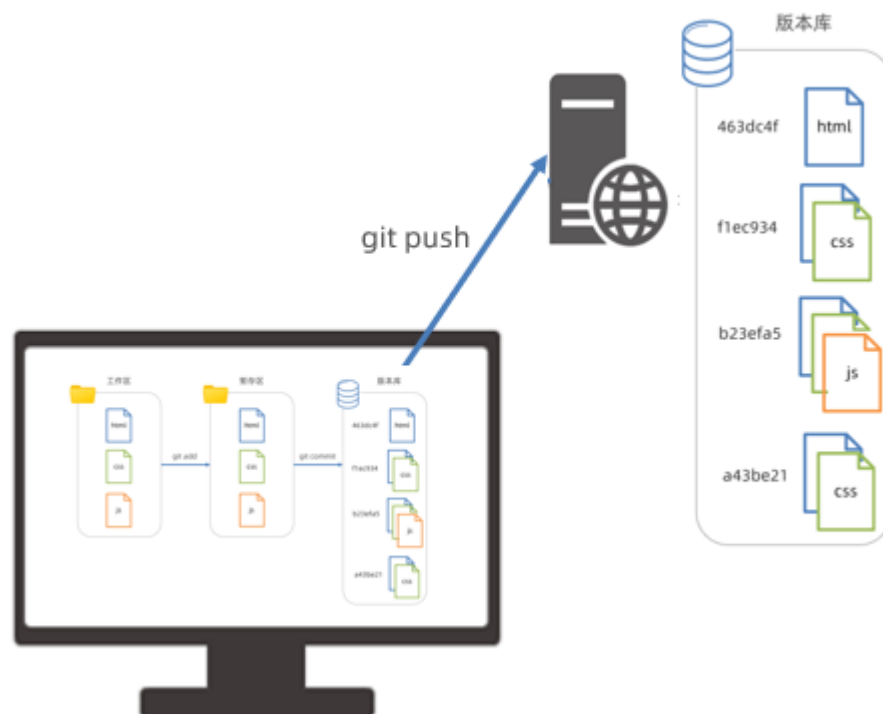
例如：git remote add origin <https://gitee.com/lidongxu/work.git>

4.本地 Git 仓库推送版本记录到远程仓库

命令：git push -u 远程仓库别名 本地和远程分支名

例如：git push -u origin master

完整写法：git push --set-upstream origin master:master



1.远程版本库的作用？

ü 保存提交历史记录，多人共享

2.远程版本库使用步骤？

ü 创建远程版本库（自己服务器/第三方托管平台）

ü 本地版本库设置远程地址

ü 推送本地版本库到远程

3.推送的命令？

ü git push -u origin master

17.Git 远程仓库 - 克隆

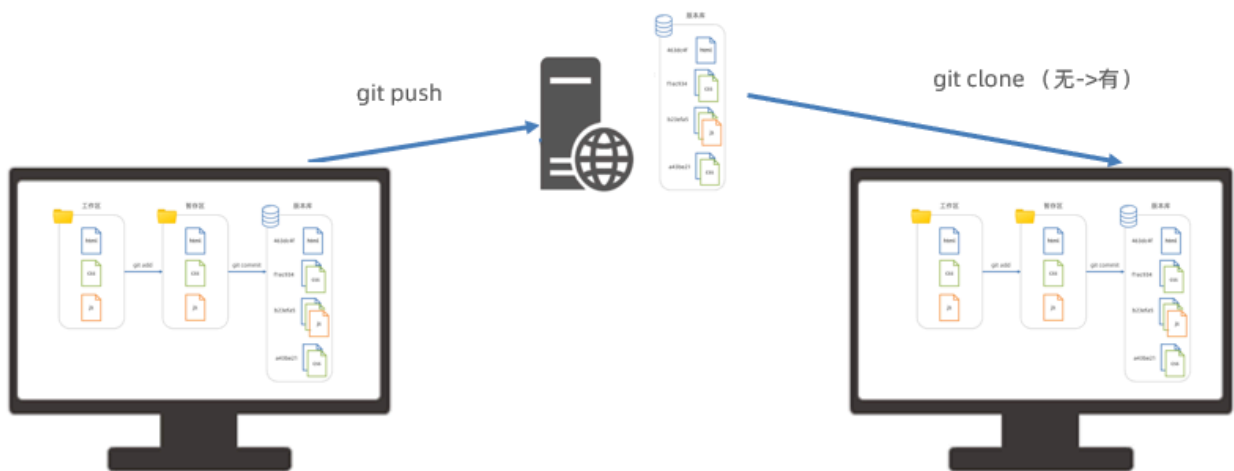
克隆：拷贝一个 Git 仓库到本地，进行使用

命令：git clone 远程仓库地址，例如：git clone <https://gitee.com/lidongxu/work.git>

效果：在运行命令所在文件夹，生成 work 项目文件夹（包含版本库，并映射到暂存区和工作区）

注意1：Git 本地仓库已经建立好和远程仓库的链接

注意2：仓库公开随意克隆，推送需要身为仓库团队成员

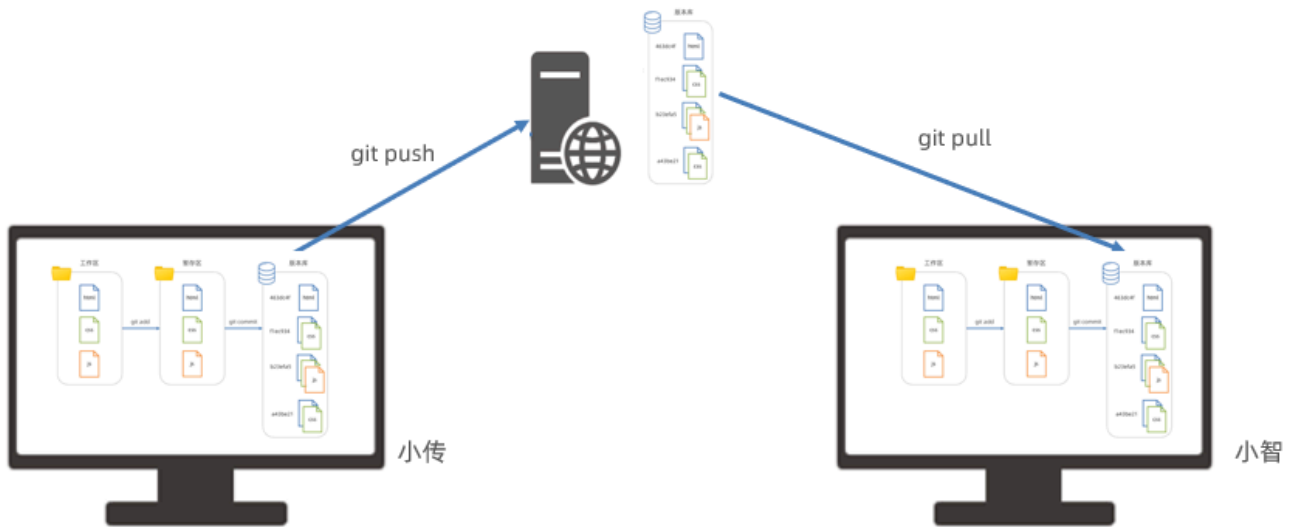


18.多人协同开发

需求：小传新代码共享给小智

步骤：

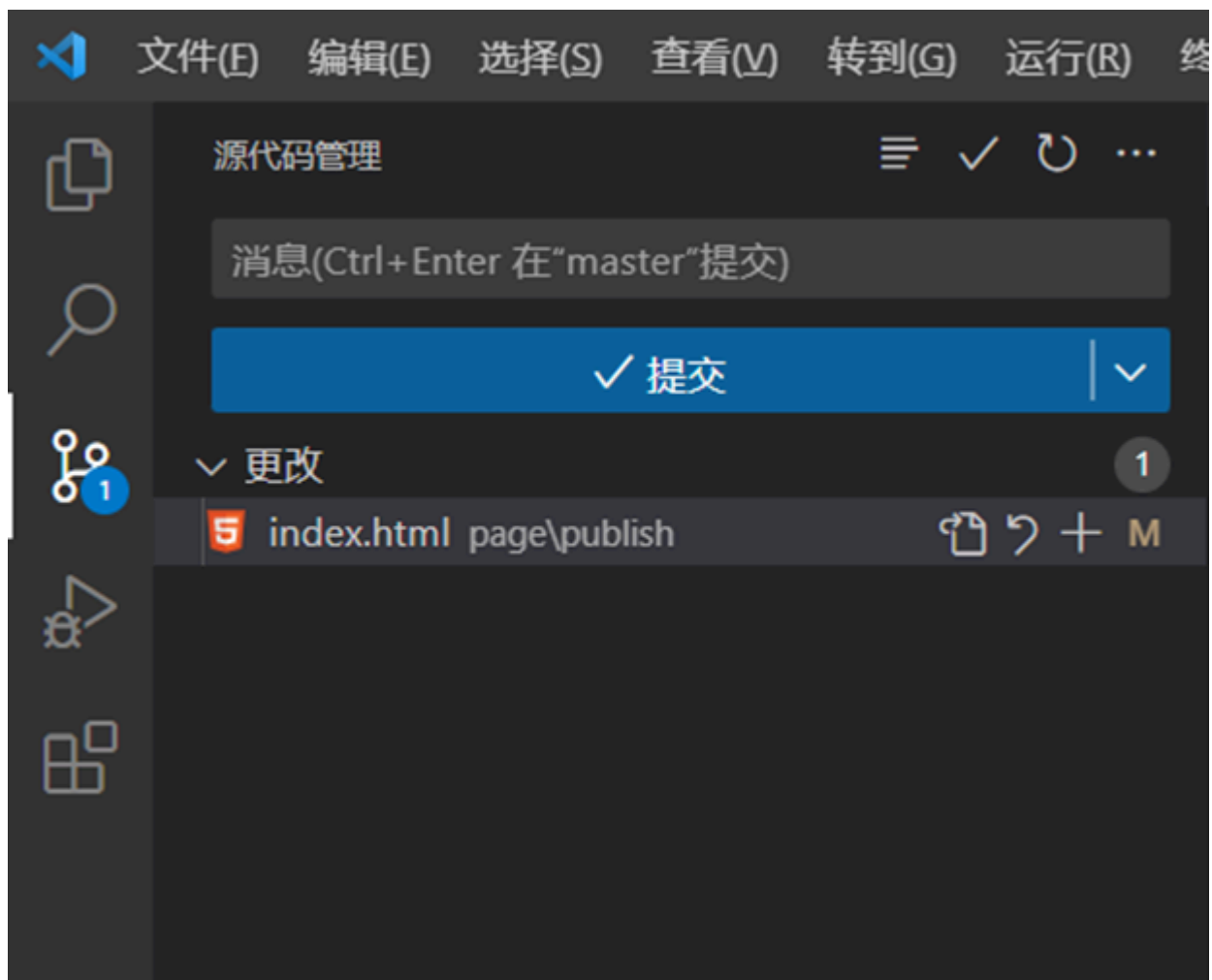
- 1.小传开发代码 -> 工作区 -> 暂存区 -> 提交 -> 拉取（可选） -> 推送
- 2.小智 -> 拉取（后续也可以开发代码 -> ... -> 推送）
- 3.想要看到别人同步上去的最新内容：git pull origin master 等价于
git fetch origin master:master （获取远程分支记录到本地，未合并）
git merge origin/master (把远程分支记录合并到所在分支下)



19.VSCode 中使用 Git

使用：源代码管理，进行暂存，撤销，对比差异，提交等操作

注意：VSCode 打开的项目文件夹下需要有 .git 仓库



20.案例-发布黑马头条数据管理平台

需求：把 Webpack 压缩好的 dist 分发文件夹网页，部署到码云上，开启 Page 服务在互联网中浏览

步骤：

- 1.初始化本地 Git 仓库（这次是非空文件夹-配套素材 dist 文件夹）
- 2.初始化远程 Git 仓库（这一次也是非空的）
- 3.本地配置远程仓库链接
- 4.本地拉取合并一下（确认本地要包含远程内容时使用）
- 5.本地推送到远程 Git 仓库
- 6.开启 page 网页服务得到地址浏览

21.Git 常用命令

命令	作用	注意
git remote add 远程仓库别名 远程仓库地址	添加远程仓库地址	别名唯一，地址是 .git 结尾的网址
git remote -v	查看远程仓库地址	
git remote remove 远程仓库别名	删除远程仓库地址	
git pull 远程仓库别名 分支名	拉取	完整写法：git pull 远程仓库别名 远程分支名:本地分支名 等价于：git fetch 和 git merge
git push 远程仓库别名 分支名	推送	完整写法：git push 远程仓库别名 本地分支名:远程分支名 -u：建立通道后以后可以简写 git push
git pull --rebase 远程仓库别名 分支名	拉取合并	合并没有关系的记录
git clone 远程仓库地址	克隆	从0得到一个远程的Git仓库到本地使用